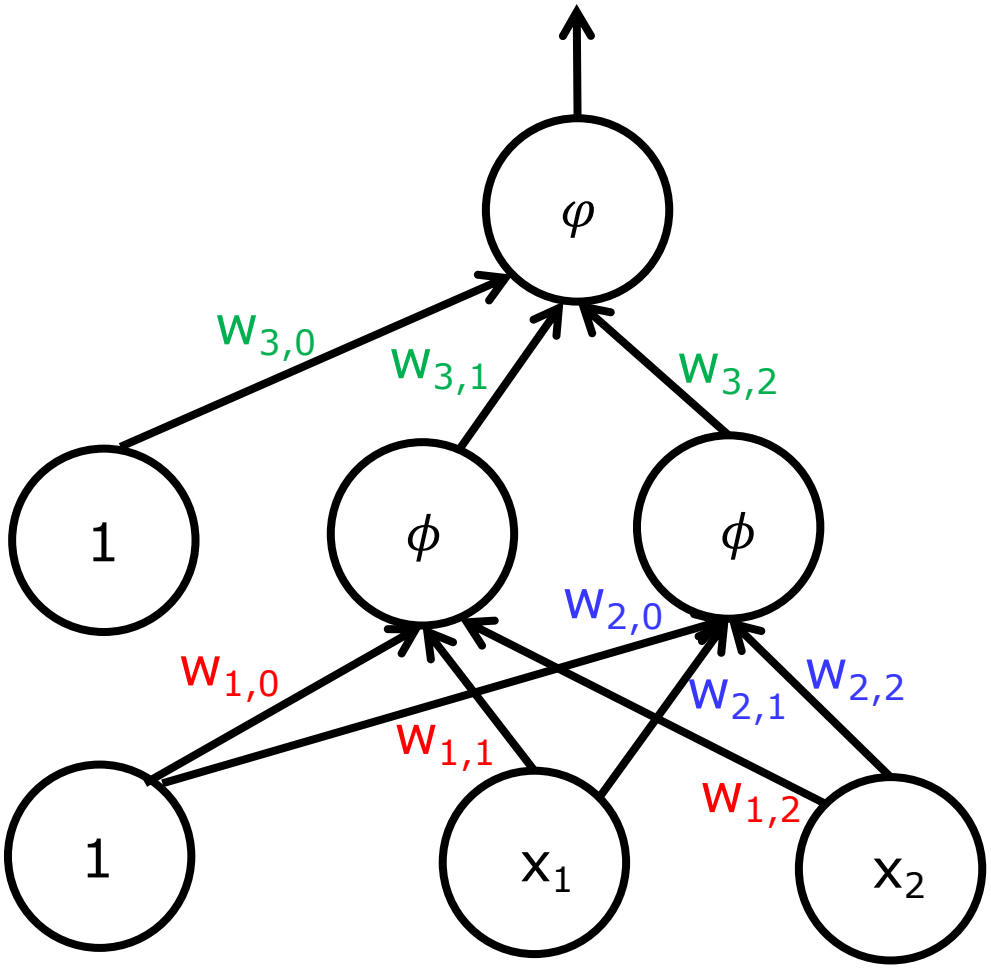
Neural Networks

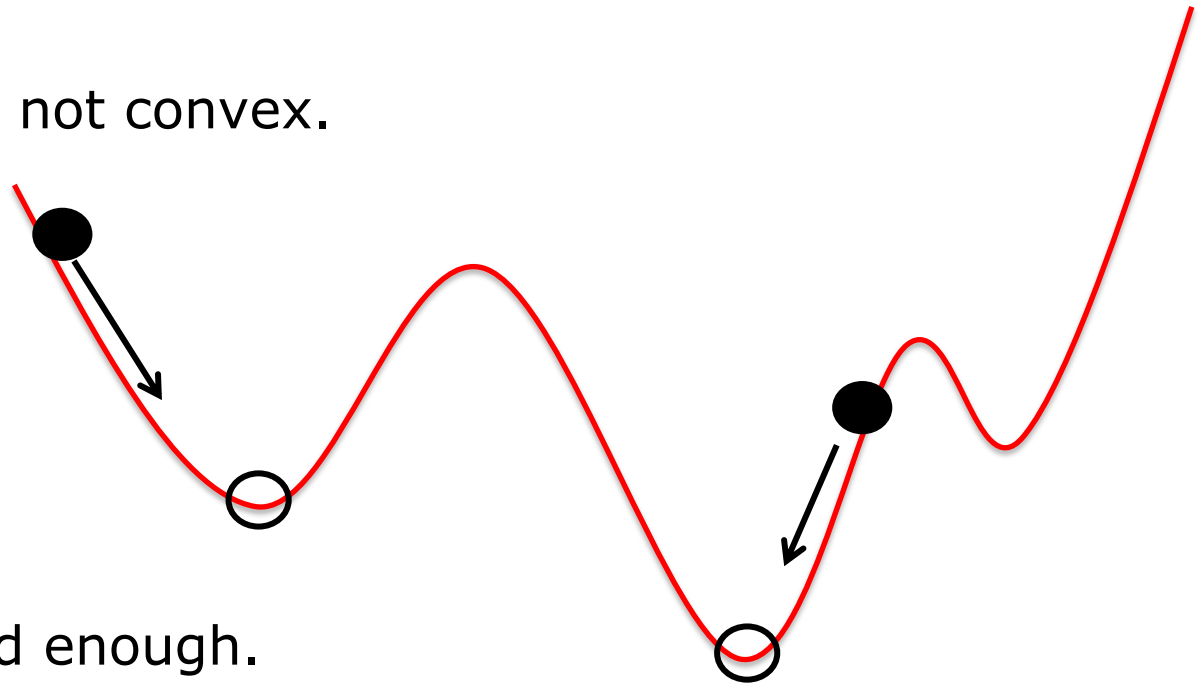
Feed Forward Neural Net



Training Neural Nets

- (Mini-Batch) Stochastic Gradient Descent
  - Algorithm only needs gradients.
    - Gradient of loss function wrt. weights.

- Concern:
  - Loss of neural net not convex.



- Don't care:
  - Local minima good enough.

Stochastic Gradient Descent

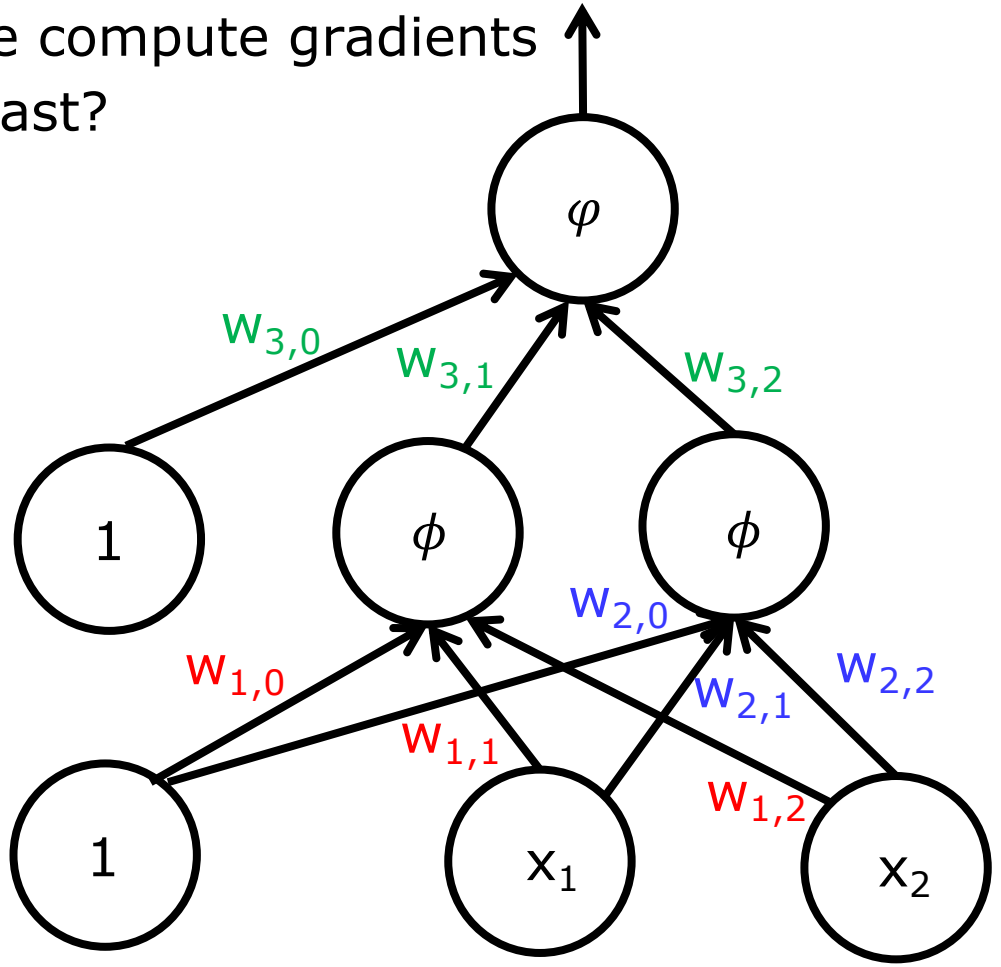
- Any point-wise loss:
  - $E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n} L(w(x_i), y_i)$
- Mini-batch SGD:
  - Initialize w (e.g. random N(0,1) distributed entrie
  - While more time:
    - Permute $(x_1,y_1),\ldots,(x_n,y_n)$ randomly.
    - For i=1,...,n/B:
      - Let $(x_1',y_1'),\ldots,(x_B',y_B')$ be next B data items.
      - $g = \frac{1}{B}\sum_{i=1}^{B} \nabla L(w(x_i'), y_i')$
      - $w = w - \eta g$
  - Return w.

$w(x_i)$ output of neural net with weights and biases w

w all weights and biases in neural net.

Feed Forward Neural Net

- Gradients:
  - How do we compute gradients of this beast?



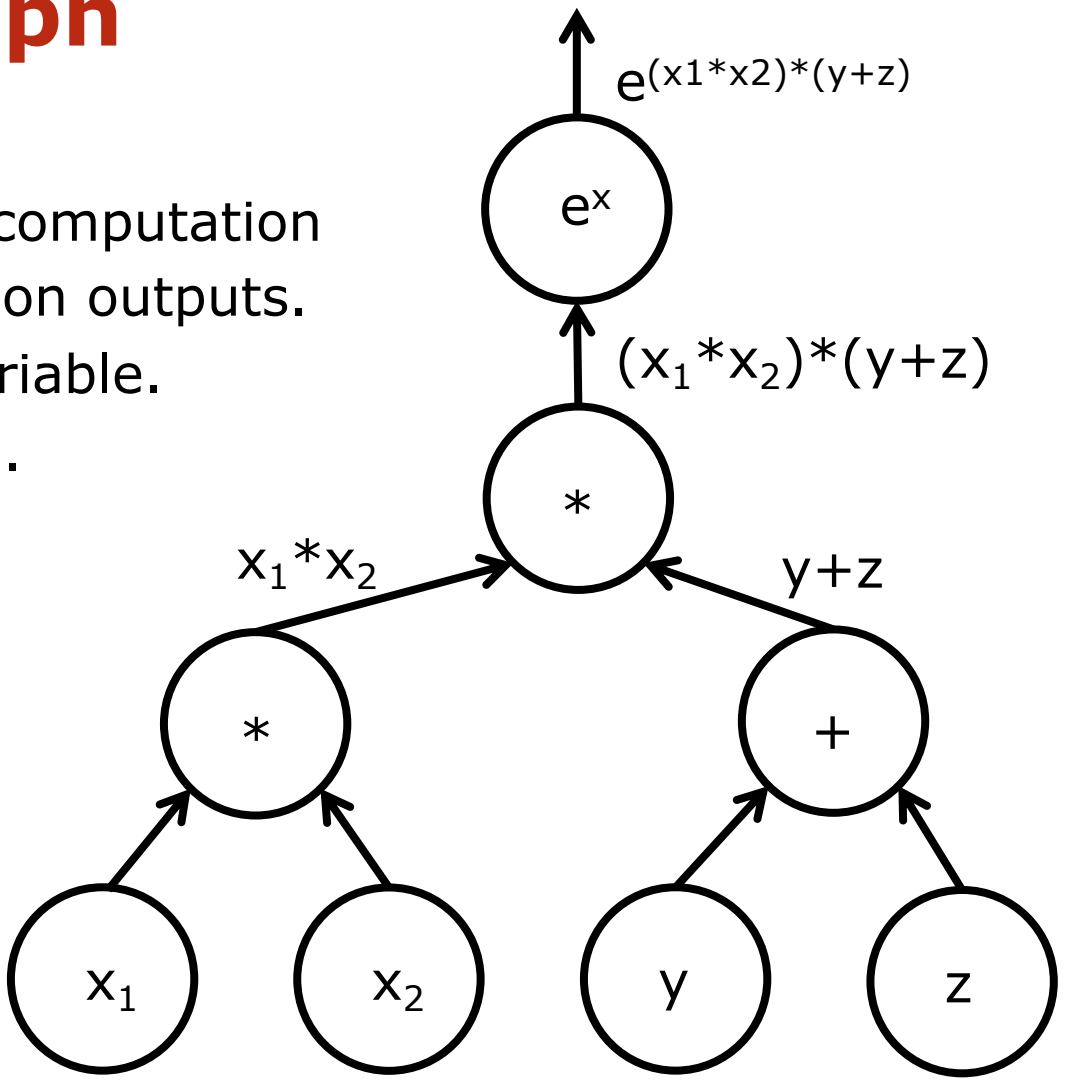
Backpropagation

- Backprop:
  - Efficient algorithm for computing gradients of computations.

- Computation Graph:
  - First we define a computation graph.

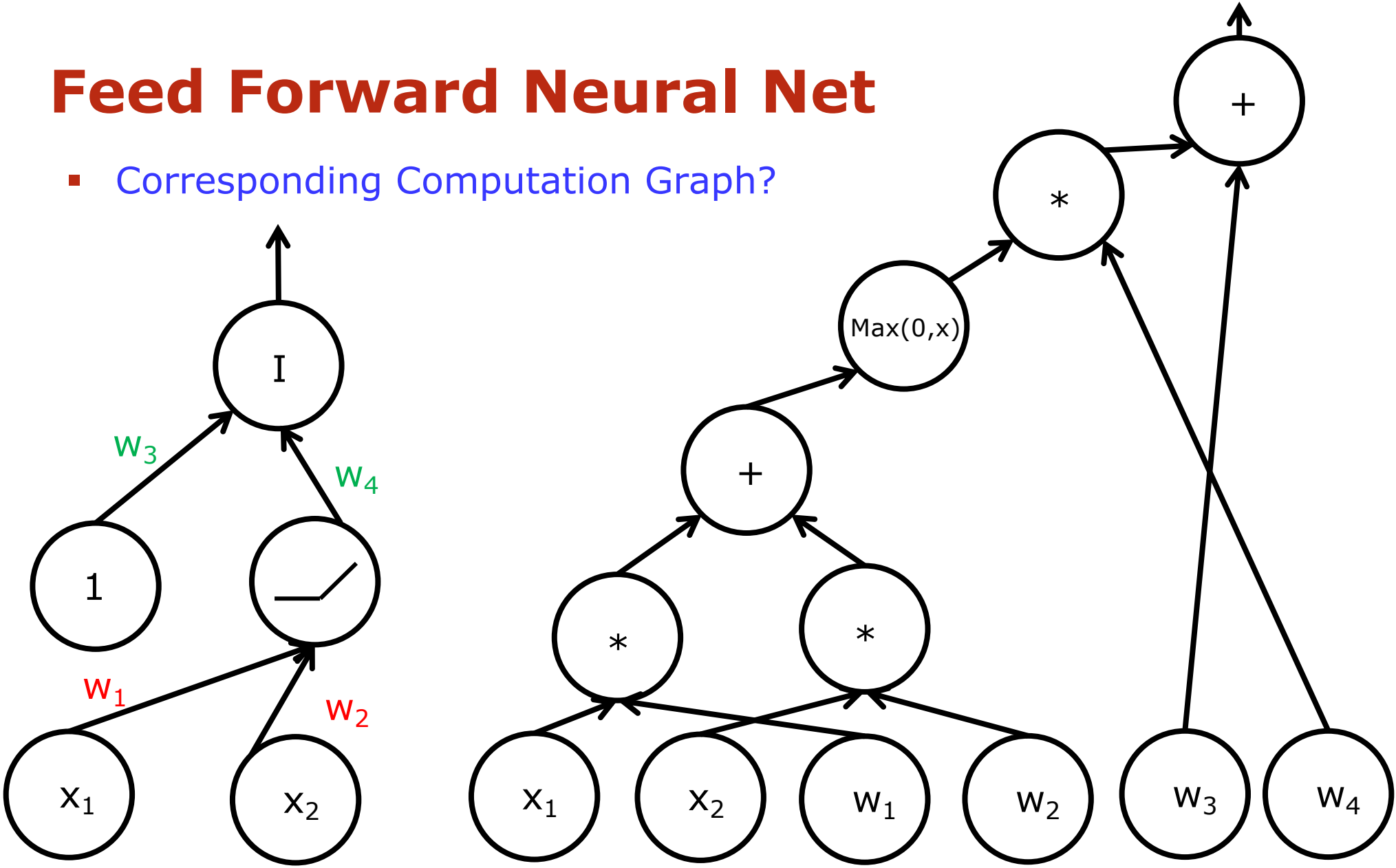
Computation Graph

- Directed Acyclic Graph
  - Each node performs a computation on inputs and forwards it on outputs.
  - Input node for each variable.
  - All edges point forward.



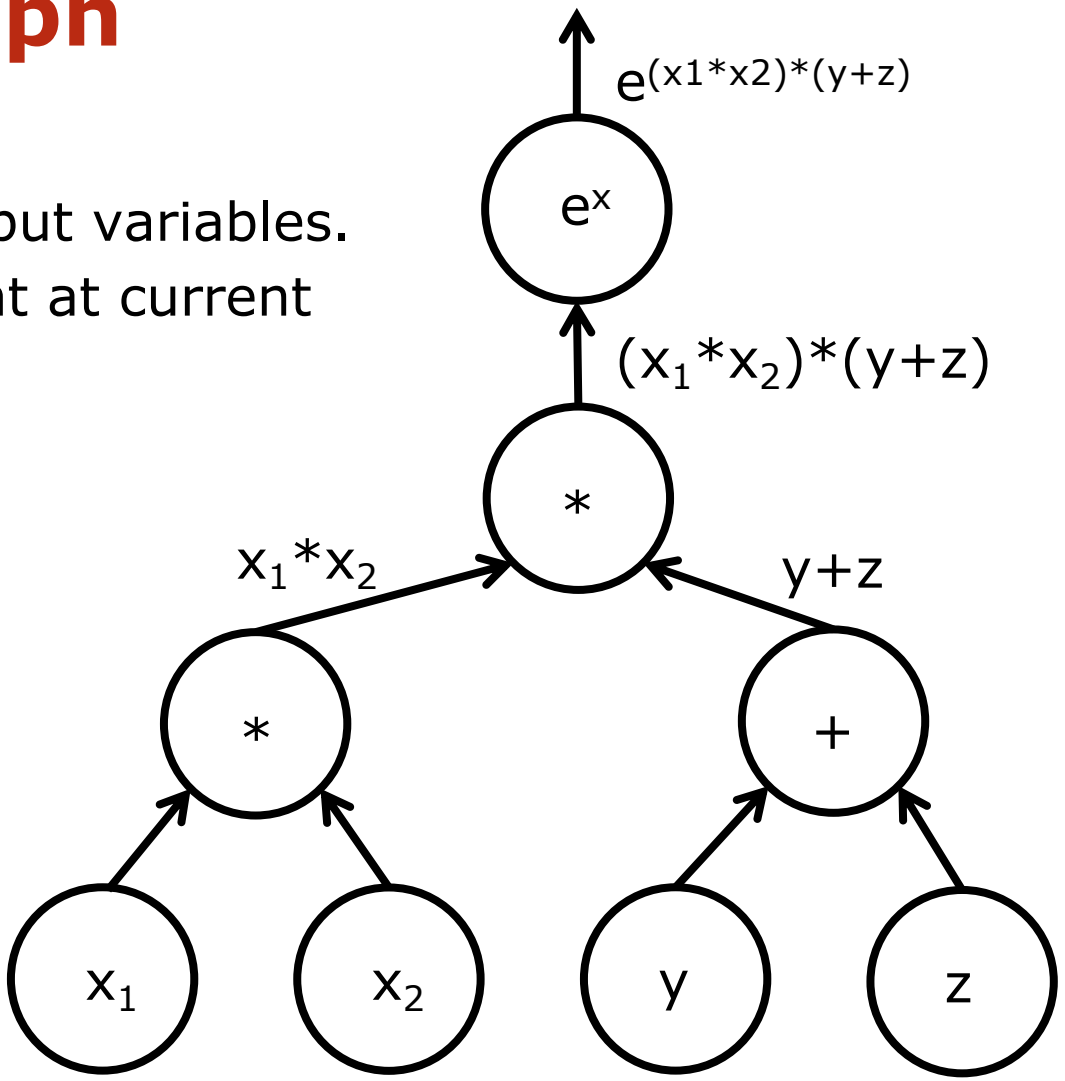
Feed Forward Neural Net

- Corresponding Computation Graph?



Computation Graph

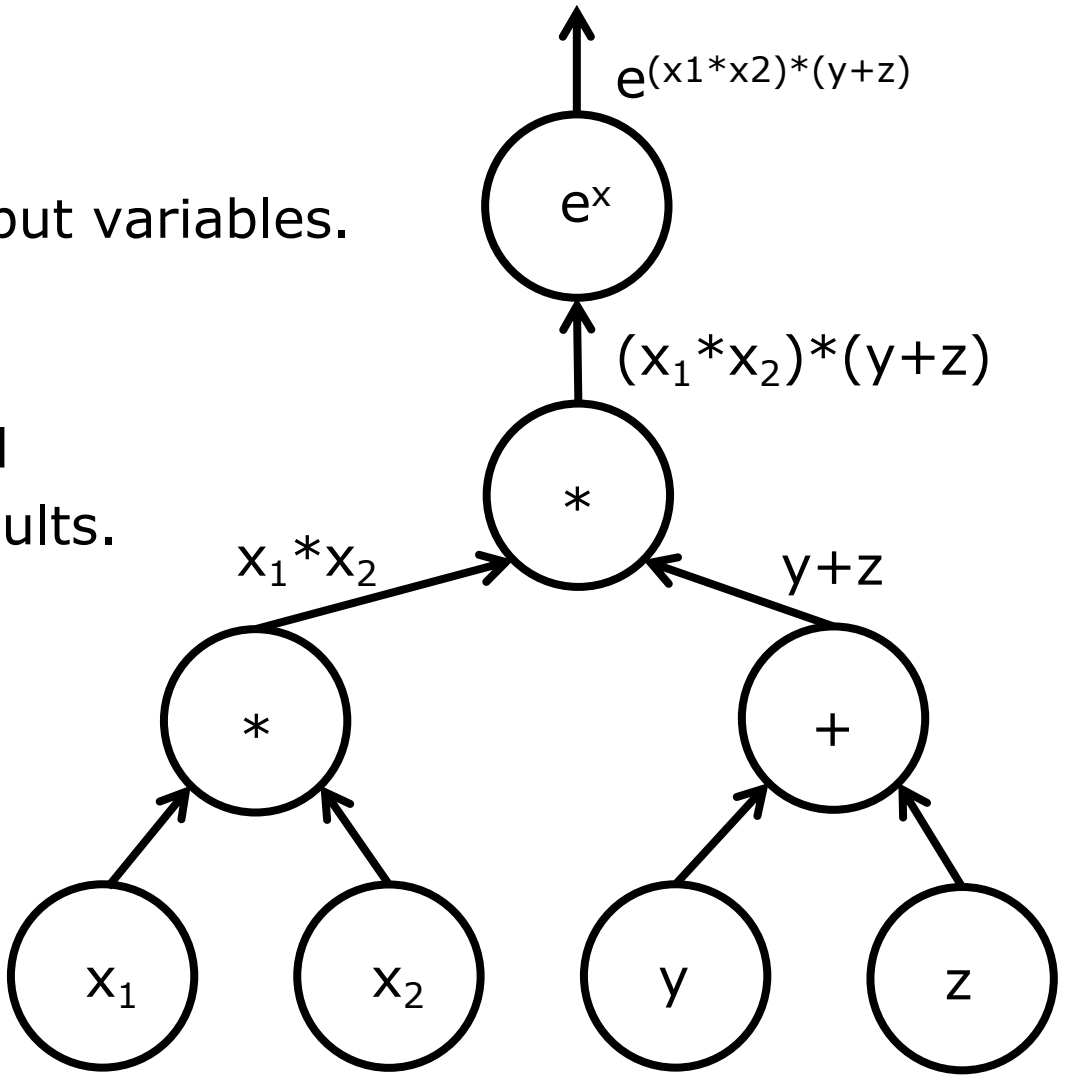
- Computing Gradients
  - Input: Values for all input variables.
  - Goal: Compute gradient at current values.



Forward Pass

- Computing Gradients
  - Input: Values for all input variables.

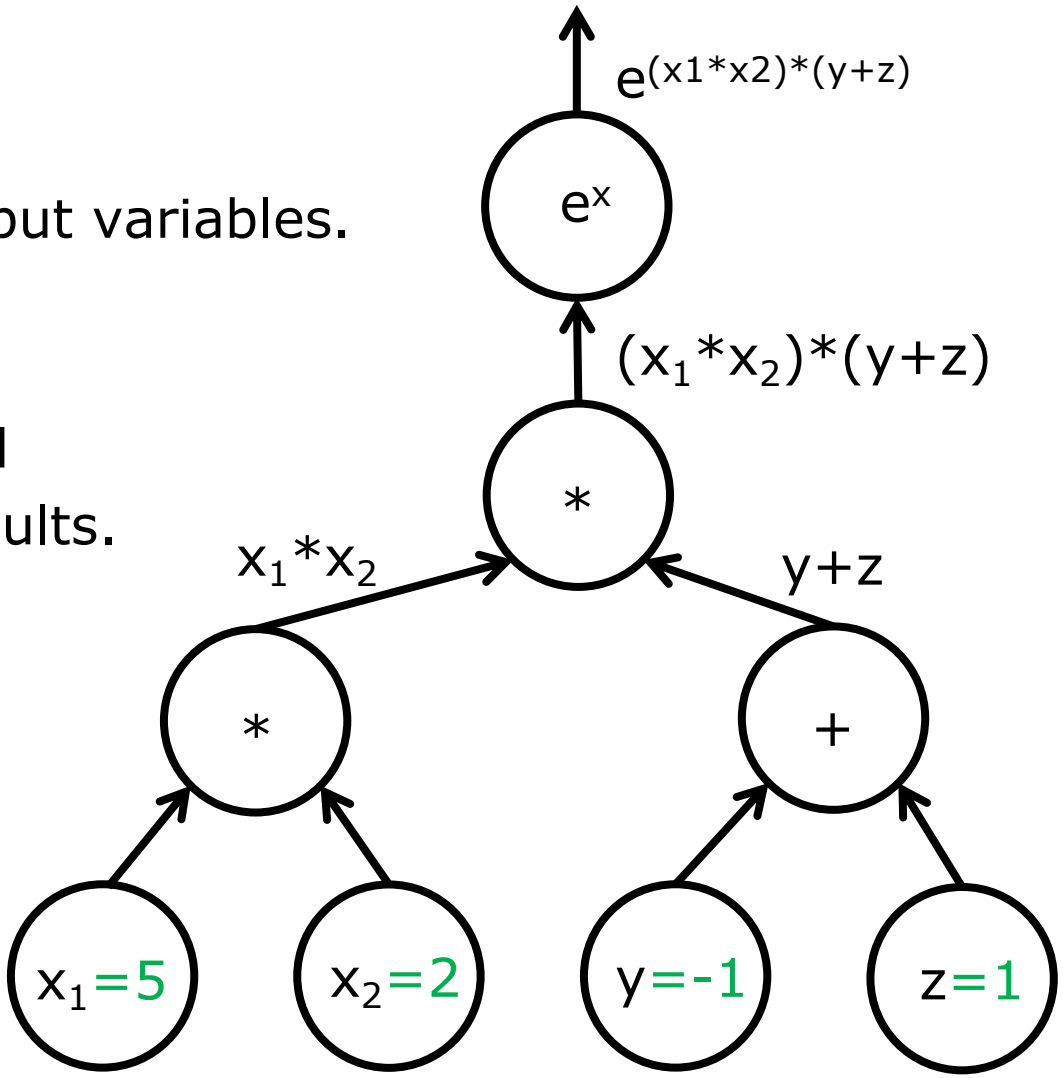
- Forward Pass:
  - Evaluate the graph and store intermediate results.



Forward Pass

- Computing Gradients
  - Input: Values for all input variables.

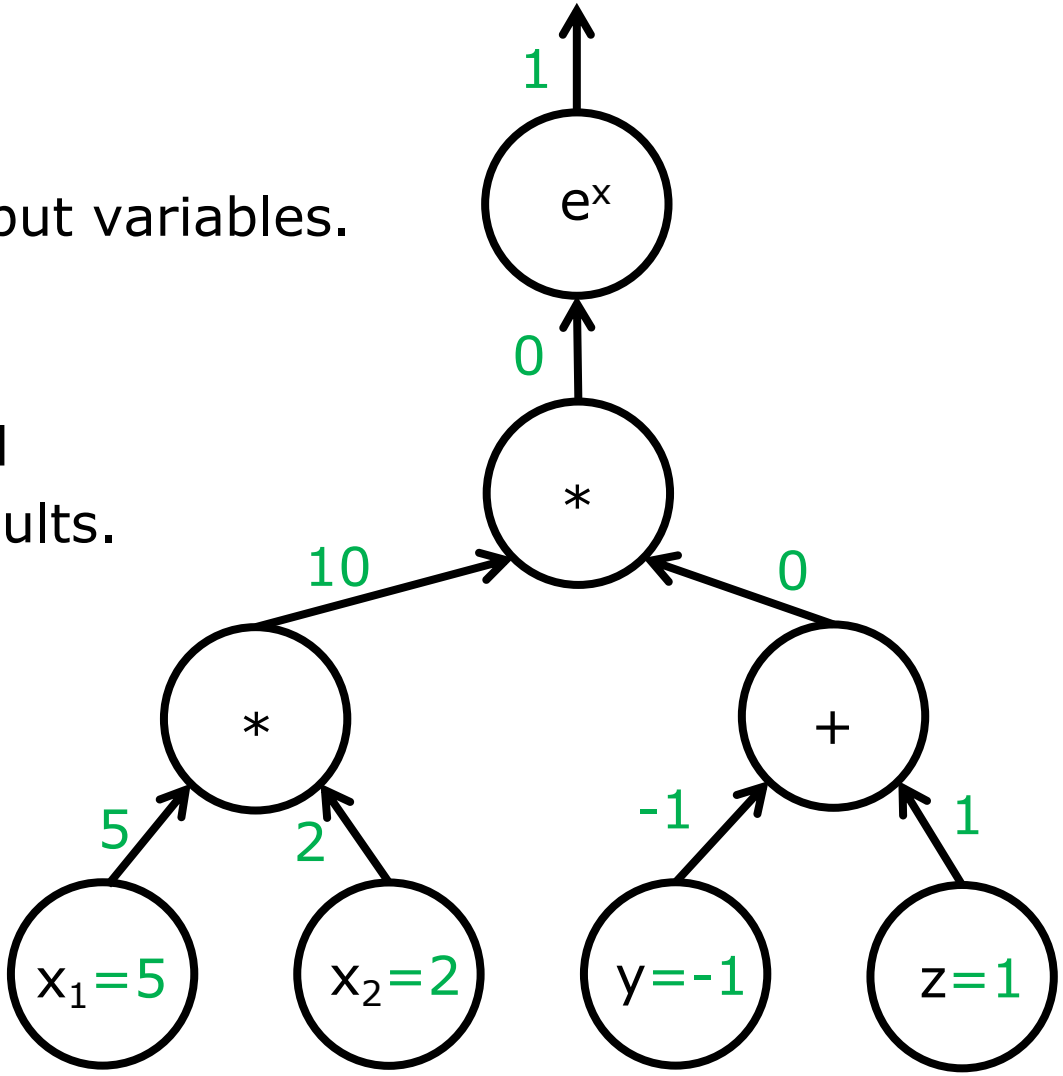
- Forward Pass:
  - Evaluate the graph and store intermediate results.



Forward Pass

- Computing Gradients
  - Input: Values for all input variables.

- Forward Pass:
  - Evaluate the graph and store intermediate results.

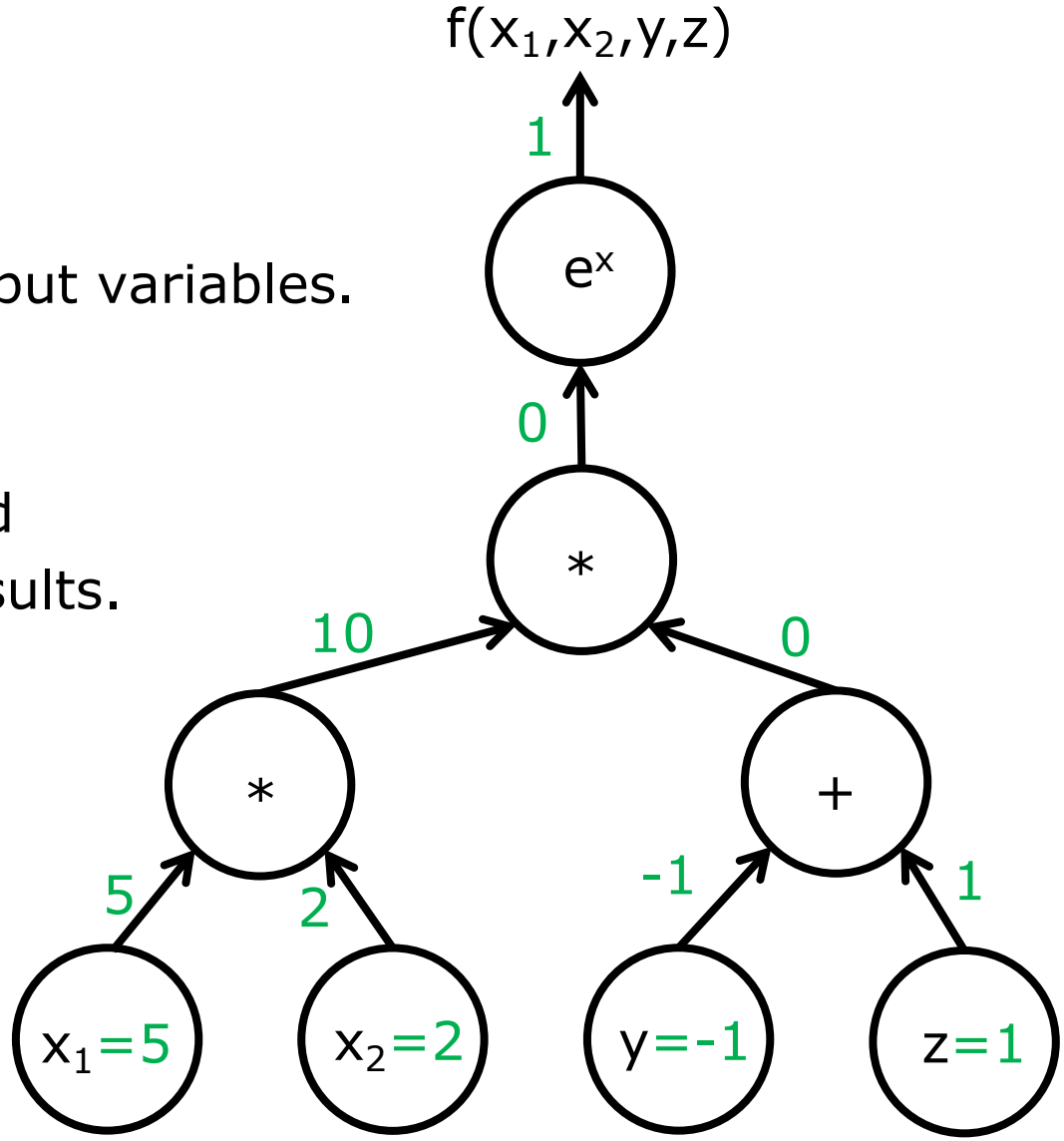


Backwards Pass

- Computing Gradients
  - Input: Values for all input variables.

- Forward Pass:
  - Evaluate the graph and store intermediate results.

How can we compute $\frac{\partial f}{\partial y}(x_1, x_2, y, z)$?

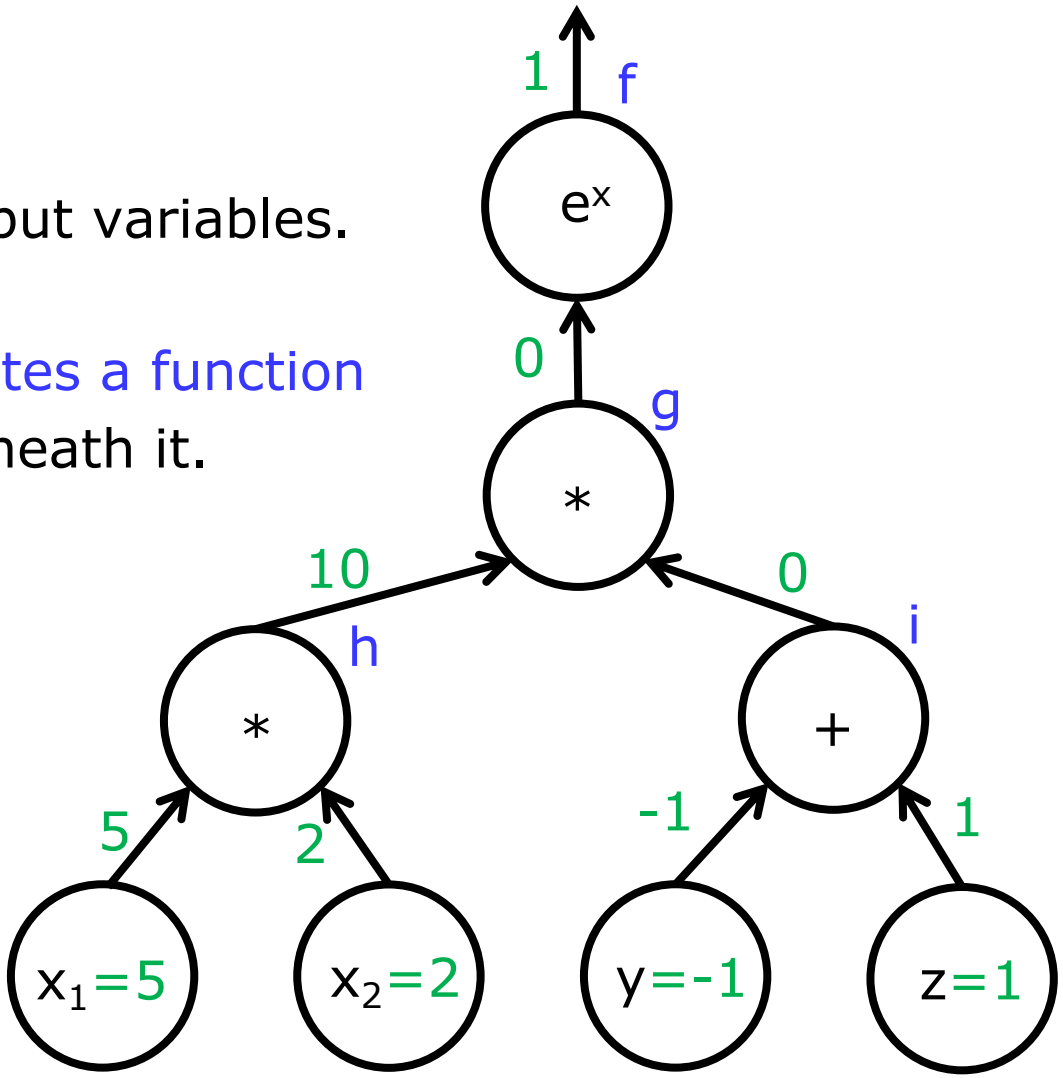


Backwards Pass

- Computing Gradients
  - Input: Values for all input variables.
- Each non-leaf node computes a function
  - Computation graph beneath it.

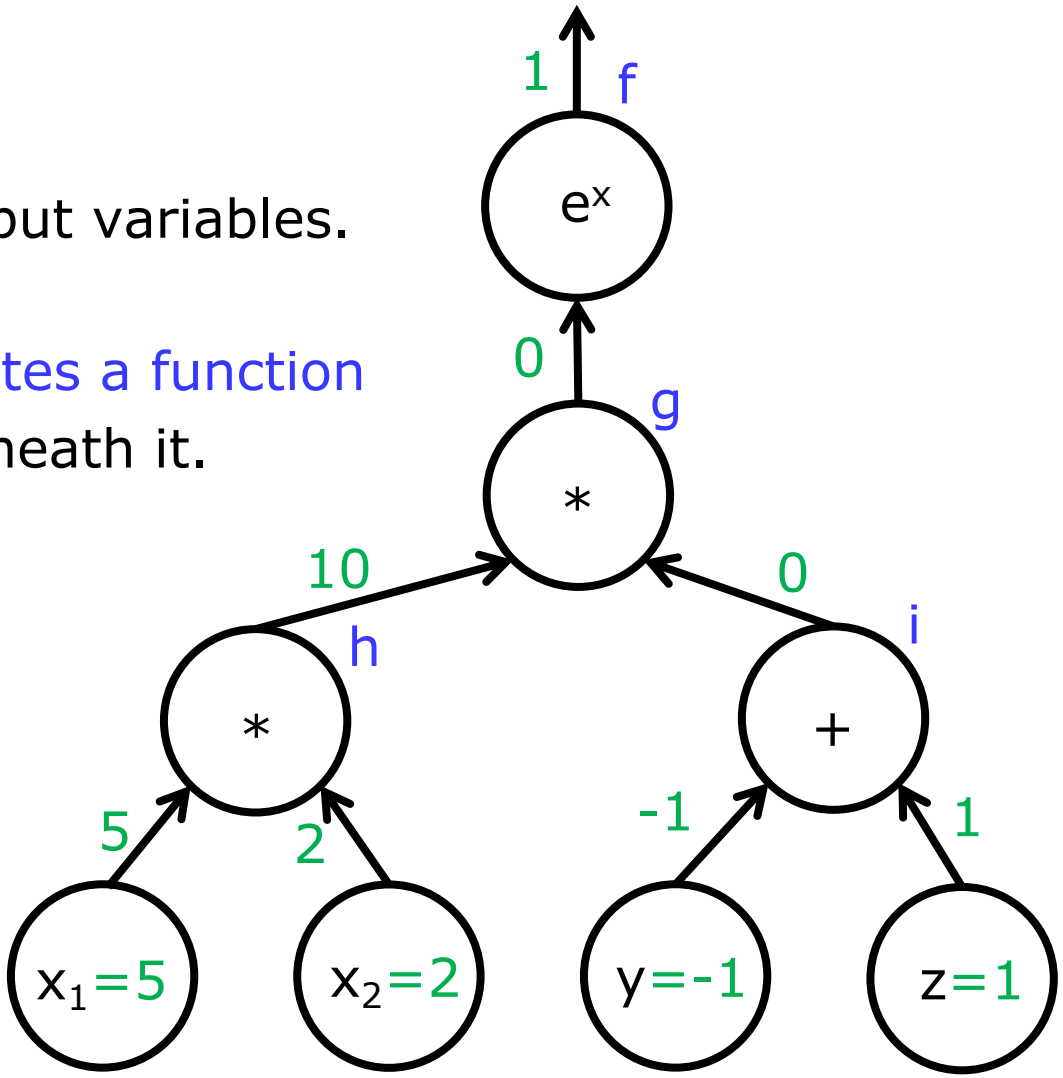
- $f=e^g$
- $g=h*i$
- $h=x_1*x_2$
- $i=y+z$

- $\frac{\partial f}{\partial y}$=?



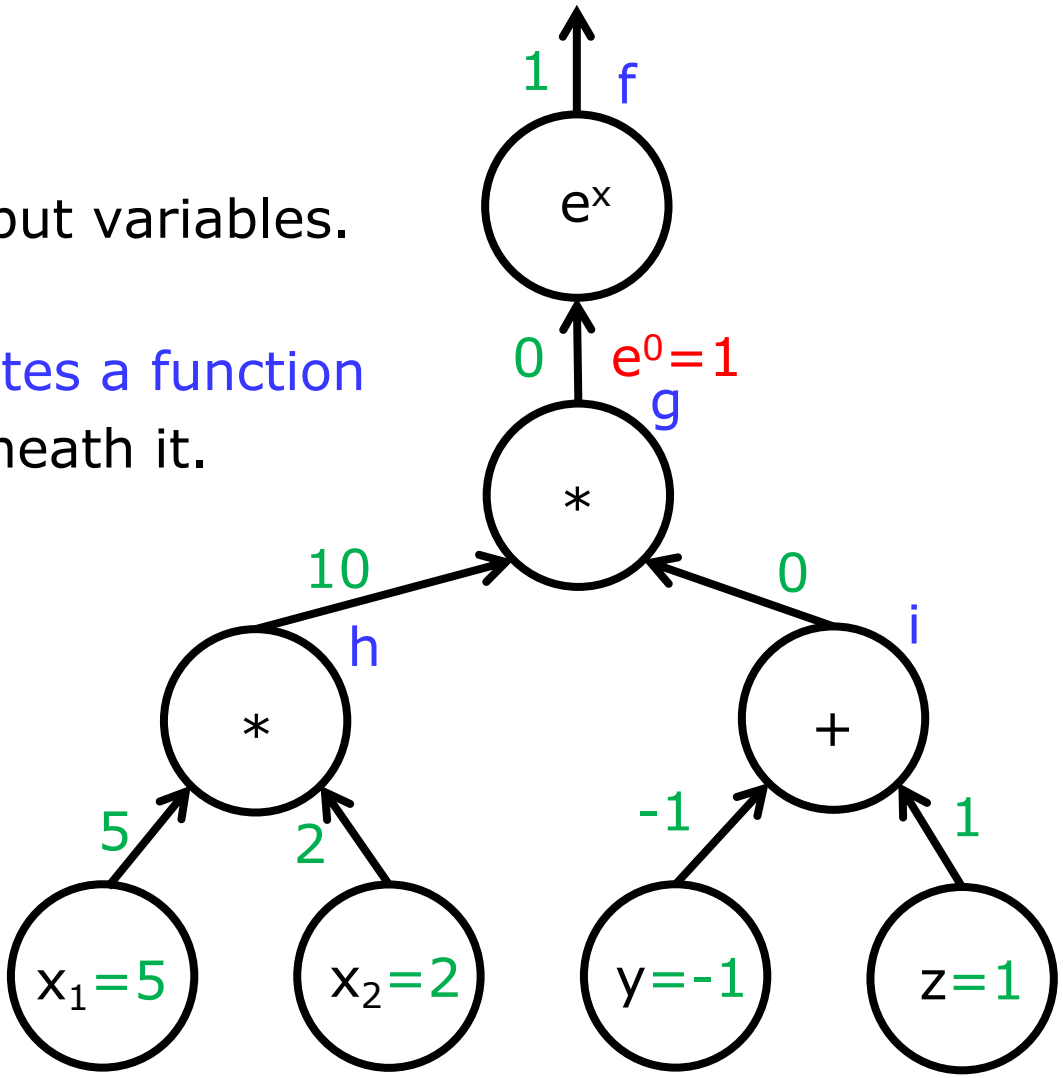
Backwards Pass

- Computing Gradients
  - Input: Values for all input variables.
- Each non-leaf node computes a function
  - Computation graph beneath it.
- $\frac{\partial f}{\partial y}$=?
- Start simpler.
- What about:
- $\frac{\partial f}{\partial g}$=?



Backwards Pass

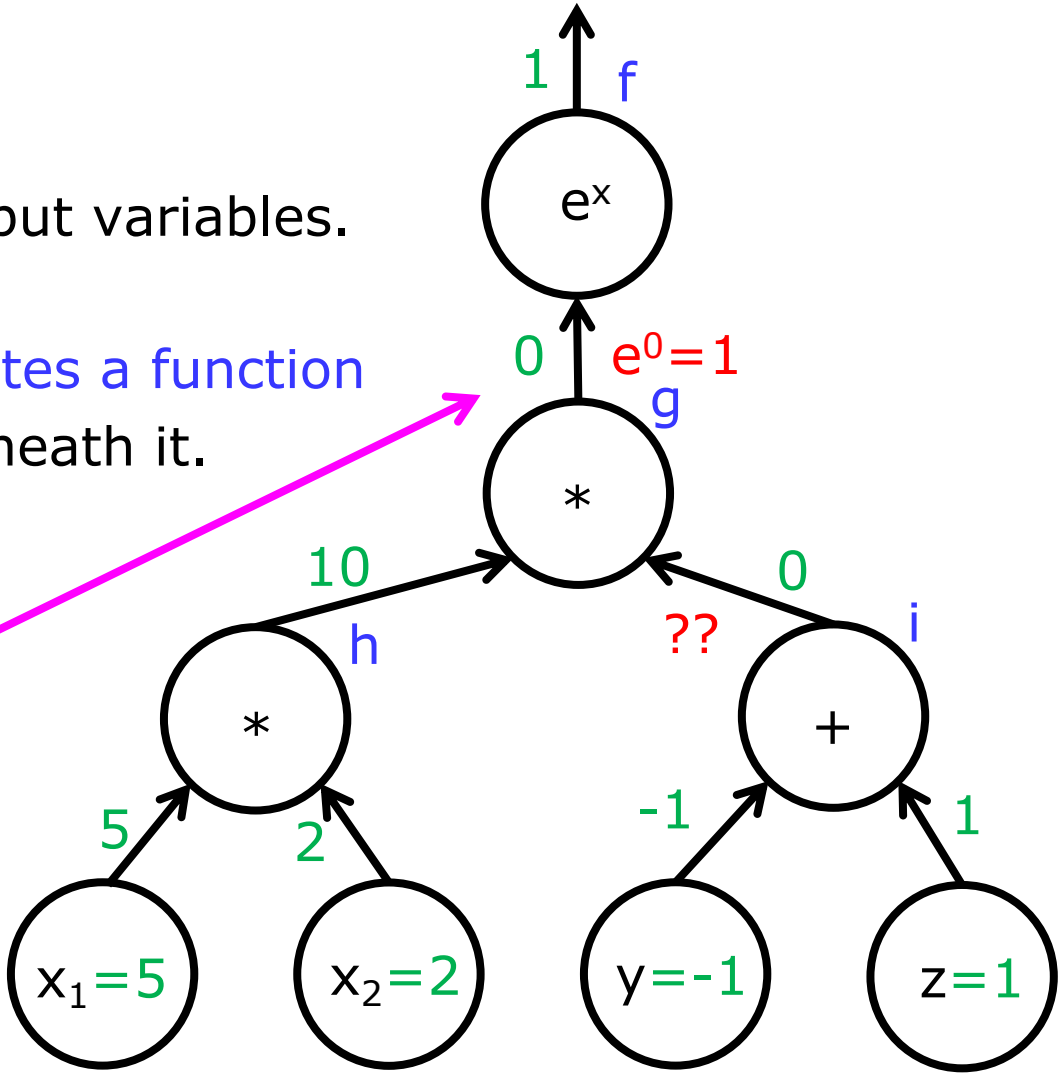
- Computing Gradients
  - Input: Values for all input variables.
- Each non-leaf node computes a function
  - Computation graph beneath it.
- $\frac{\partial f}{\partial g}$=?
- $f=e^g$
- $\frac{\partial f}{\partial g}=e^g$
- Know g from forward pass.



Chain Rule

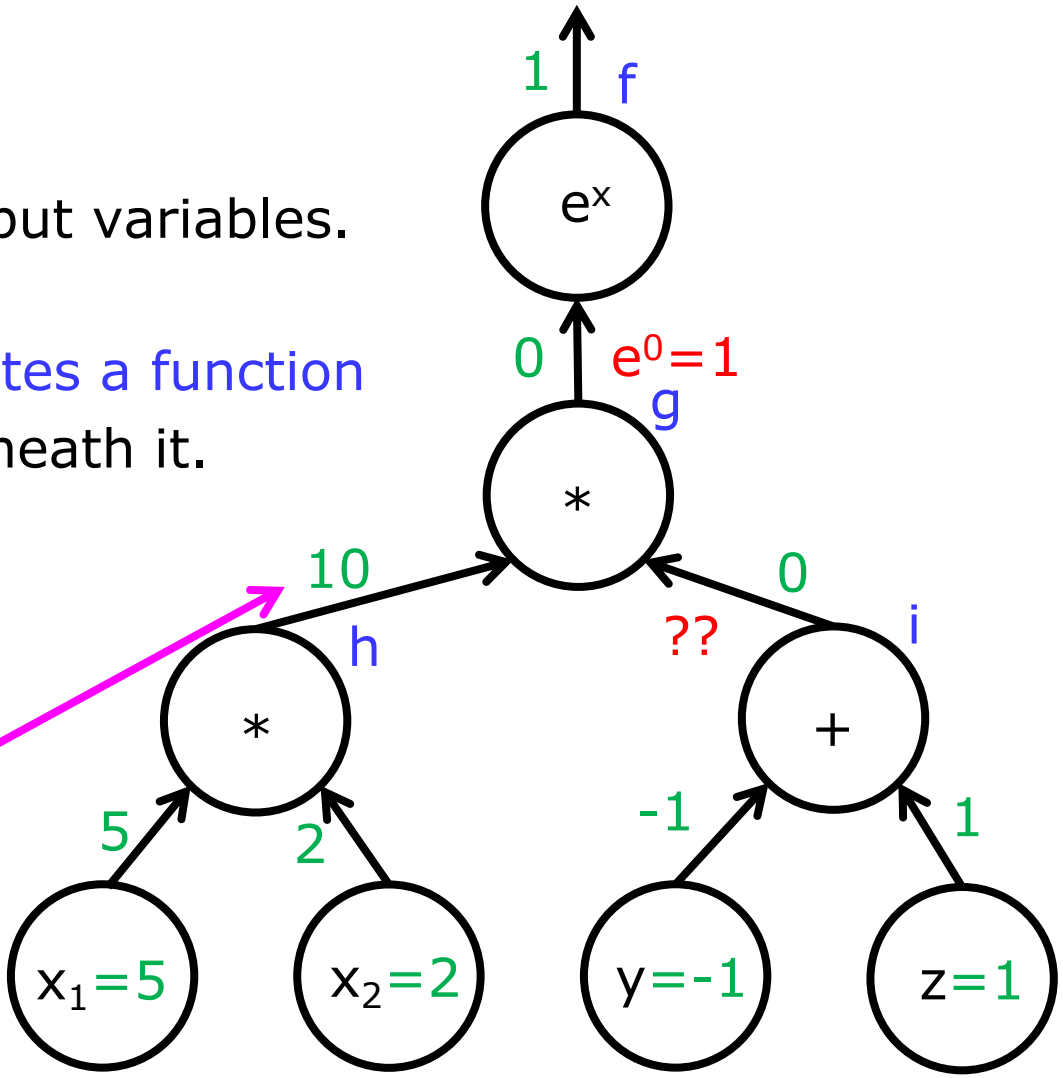
- Computing Gradients
  - Input: Values for all input variables.
- Each non-leaf node computes a function
  - Computation graph beneath it.
- What about
- $\frac{\partial f}{\partial i}$=?
- Chain Rule!
- $\frac{\partial f}{\partial i}$=
- $\frac{\partial f}{\partial g}\frac{\partial g}{\partial i}$

Just computed!



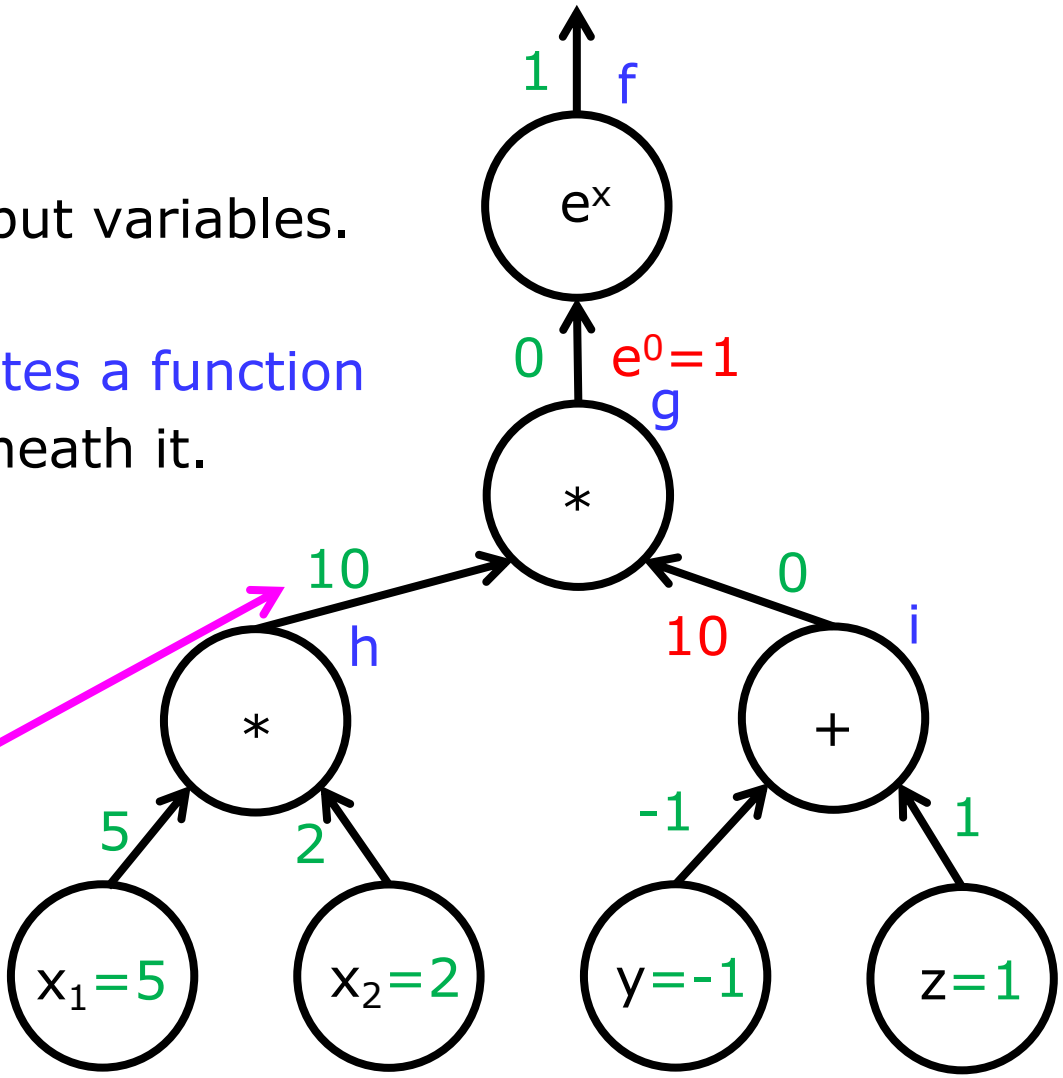
Chain Rule

- Computing Gradients
  - Input: Values for all input variables.
- Each non-leaf node computes a function
  - Computation graph beneath it.
- What about
- $\frac{\partial f}{\partial i} = 1 * \frac{\partial g}{\partial i}$
- g=h*i
- $\frac{\partial g}{\partial i}$=h
- $\frac{\partial f}{\partial i}$=h=10



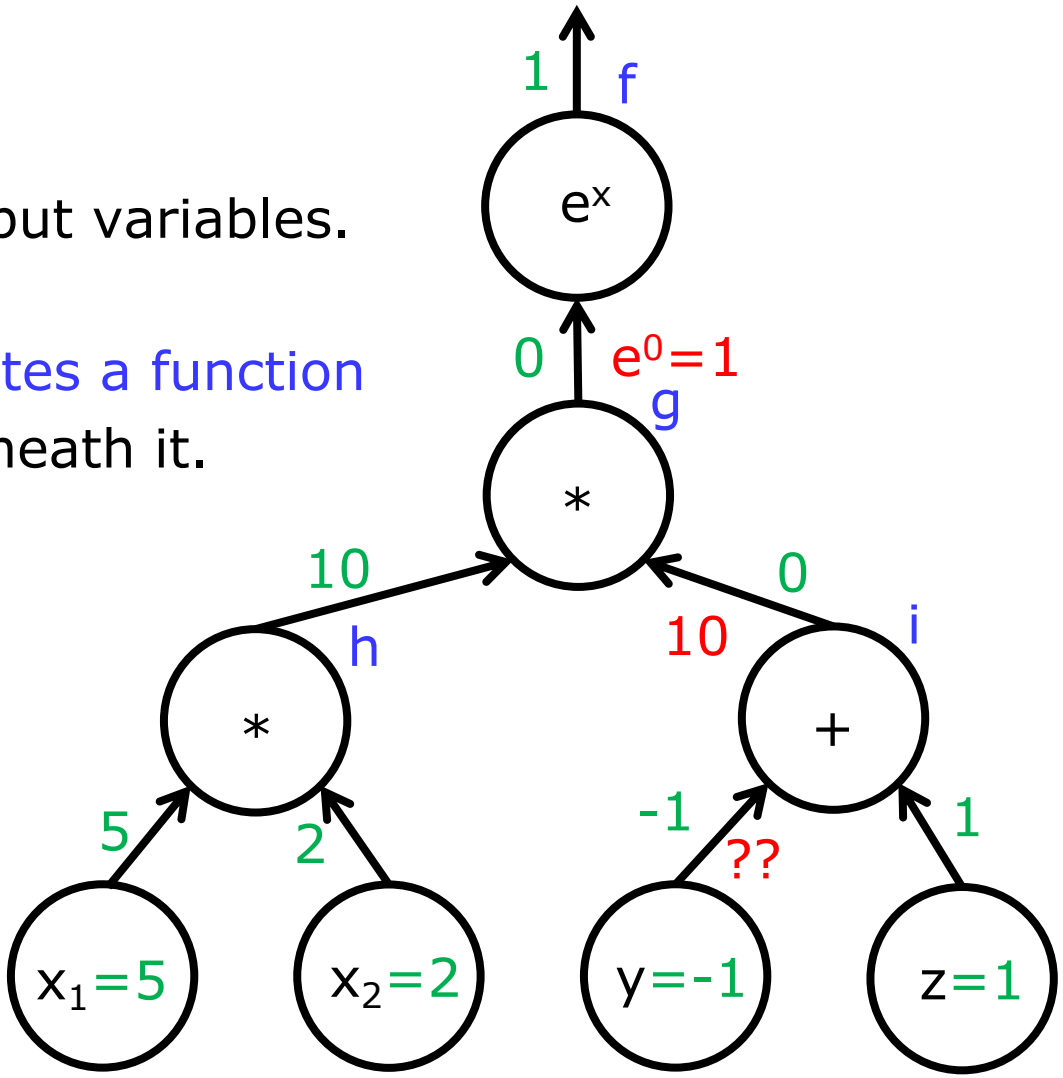
Chain Rule

- Computing Gradients
  - Input: Values for all input variables.
- Each non-leaf node computes a function
  - Computation graph beneath it.
- What about
- $\frac{\partial f}{\partial i} = 1 * \frac{\partial g}{\partial i}$
- g=h*i
- $\frac{\partial g}{\partial i}$=h
- $\frac{\partial f}{\partial i}$=h=10



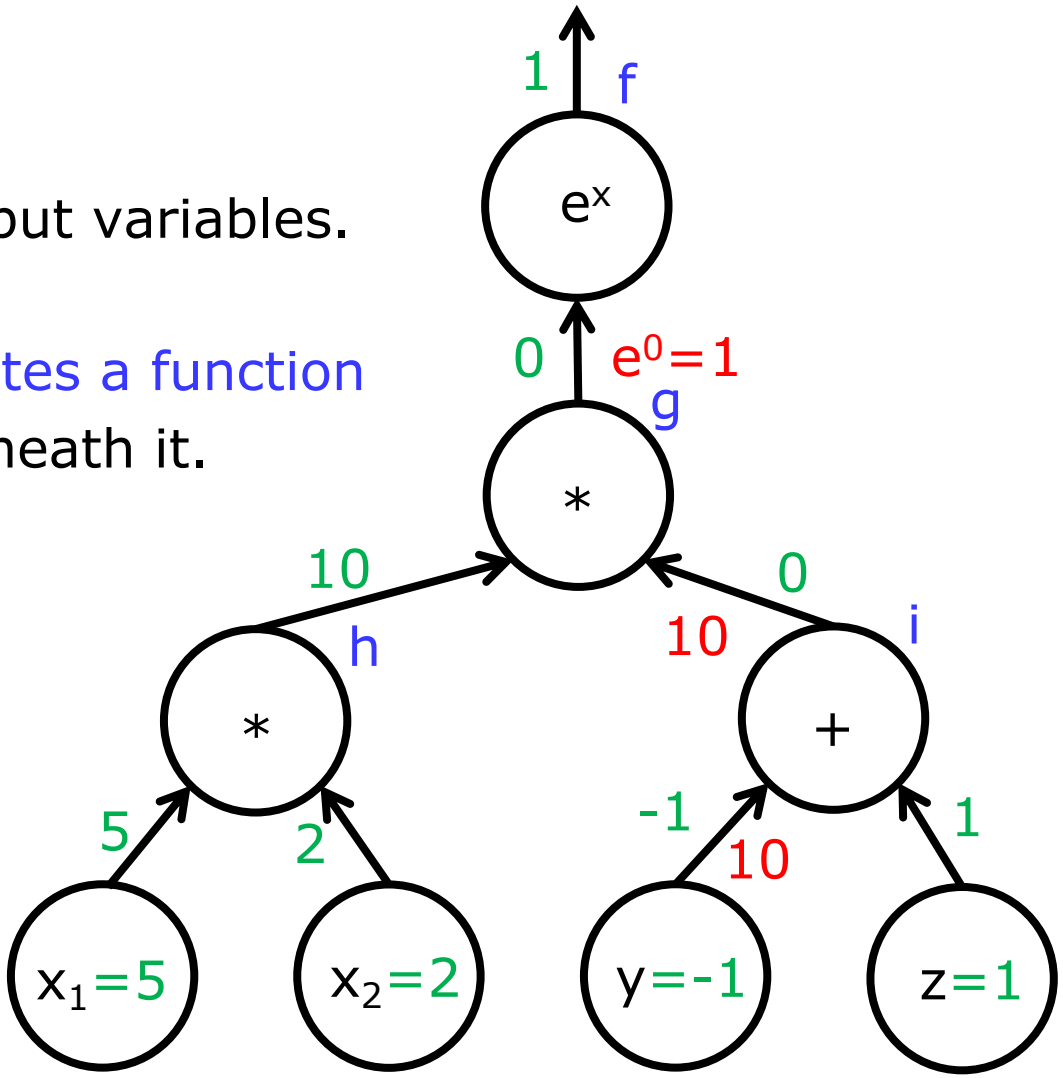
Chain Rule

- Computing Gradients
  - Input: Values for all input variables.
- Each non-leaf node computes a function
  - Computation graph beneath it.
- What about
- $\frac{\partial f}{\partial y}$=
- $\frac{\partial f}{\partial y}$= $\frac{\partial f}{\partial i}\frac{\partial i}{\partial y}$
- $\frac{\partial f}{\partial y}$= $10 * \frac{\partial i}{\partial y}$
- i=y+z, so $\frac{\partial i}{\partial y}$=1



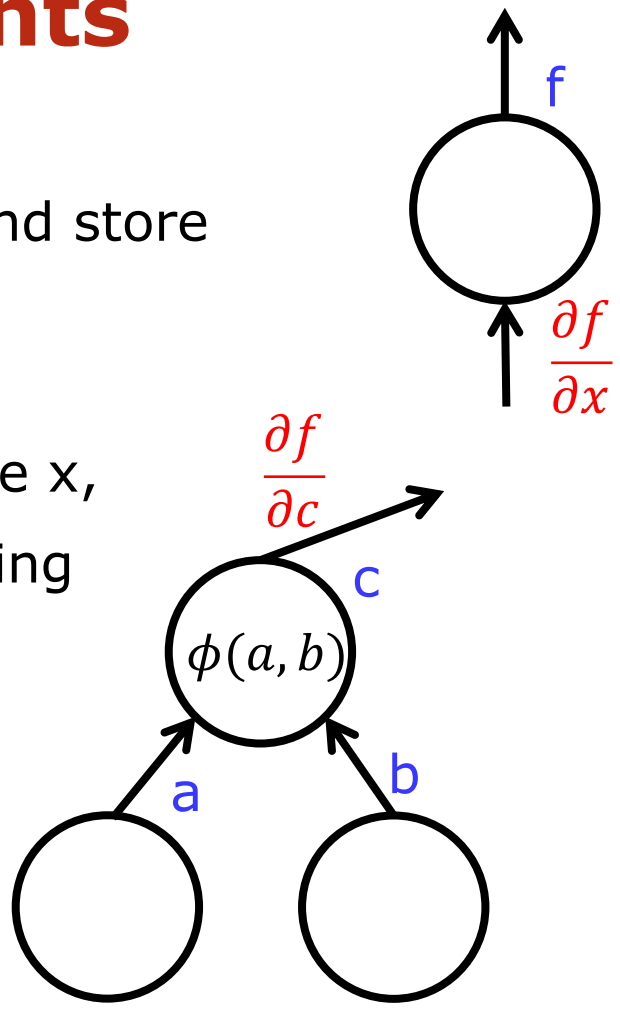
Chain Rule

- Computing Gradients
  - Input: Values for all input variables.
- Each non-leaf node computes a function
  - Computation graph beneath it.
- What about
- $\frac{\partial f}{\partial y}$=10



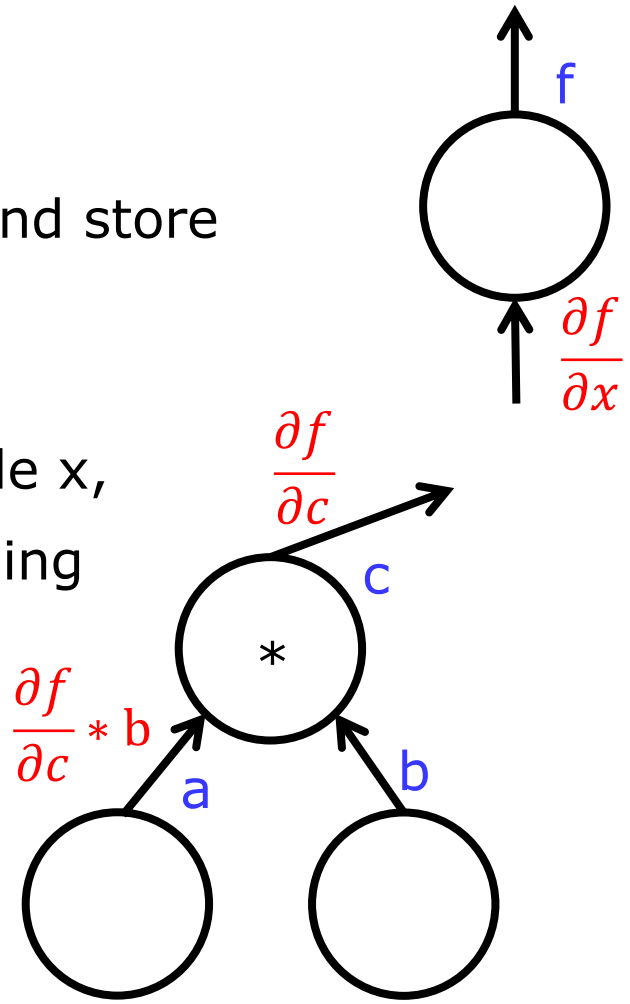
Computing Gradients

- General Algorithm:
  - Forward pass, compute and store values of all nodes.
- Backprop:
  - Compute $\frac{\partial f}{\partial x}$ for every node x, starting from top and going down.
  - To compute $\frac{\partial f}{\partial a}$:
    - $\frac{\partial f}{\partial a} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial a} = \frac{\partial f}{\partial c}\frac{\partial \phi(a,b)}{\partial a}$
  - Already computed $\frac{\partial f}{\partial c}$
  - $\phi(a,b)$ is simple and a,b known from forward pass.



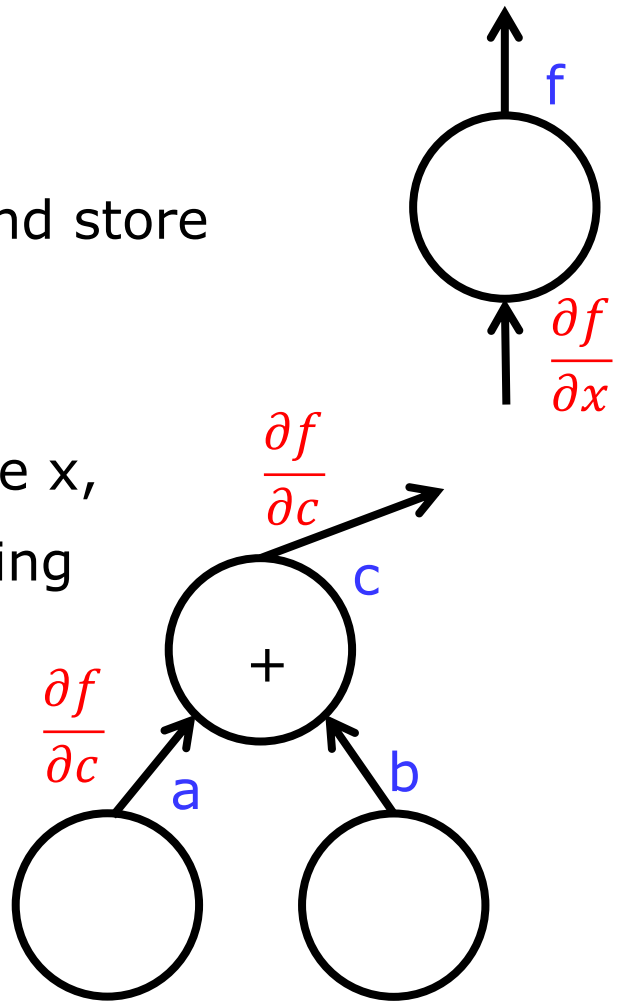
Backpropagation

- General Algorithm:
  - Forward pass, compute and store values of all nodes.
- Backprop:
  - Compute $\frac{\partial f}{\partial x}$ for every node x, starting from top and going down.
  - To compute $\frac{\partial f}{\partial a}$:
    - $\frac{\partial f}{\partial a} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial a} = \frac{\partial f}{\partial c}\frac{\partial \phi(a,b)}{\partial a}$
  - Already computed $\frac{\partial f}{\partial c}$
  - $\frac{\partial a*b}{\partial a}$=b



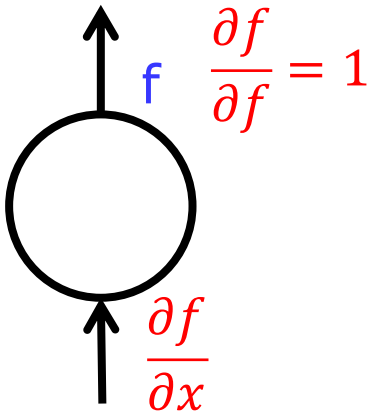
Backpropagation

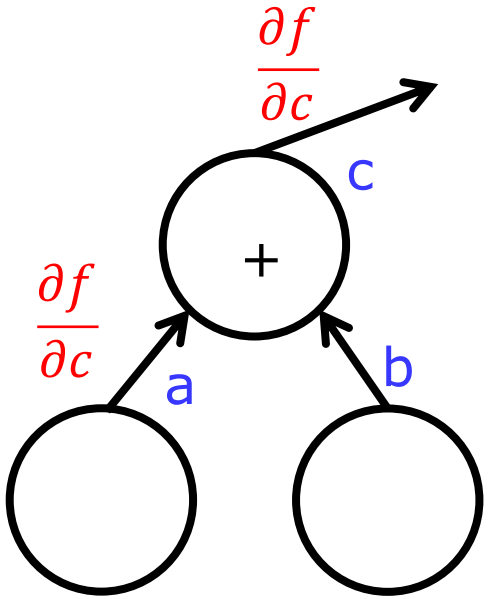
- General Algorithm:
  - Forward pass, compute and store values of all nodes.
- Backprop:
  - Compute $\frac{\partial f}{\partial x}$ for every node x, starting from top and going down.
  - To compute $\frac{\partial f}{\partial a}$:
    - $\frac{\partial f}{\partial a} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial a} = \frac{\partial f}{\partial c}\frac{\partial \phi(a,b)}{\partial a}$
  - Already computed $\frac{\partial f}{\partial c}$
  - $\frac{\partial a+b}{\partial a}=1$



Backpropagation

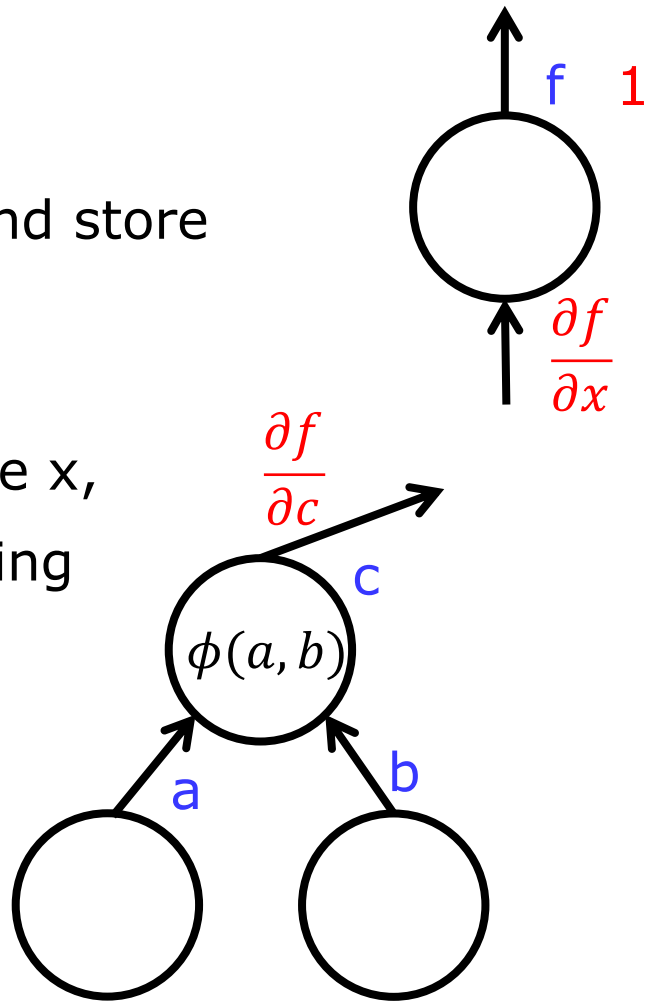
- General Algorithm:
  - Forward pass, compute and store values of all nodes.
- Backprop:
  - Rule to get started:
    - $\frac{\partial f}{\partial f} = 1$





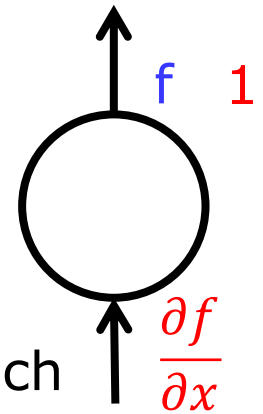
Backpropagation

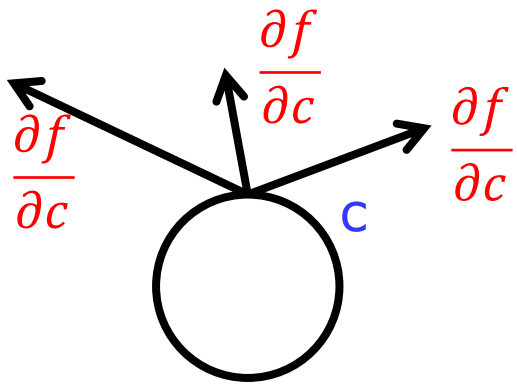
- General Algorithm:
  - Forward pass, compute and store values of all nodes.
- Backprop:
  - Compute $\frac{\partial f}{\partial x}$ for every node x, starting from top and going down.
  - To compute $\frac{\partial f}{\partial a}$:
    - $\frac{\partial f}{\partial a} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial a} = \frac{\partial f}{\partial c}\frac{\partial \phi(a,b)}{\partial a}$
  - Already computed $\frac{\partial f}{\partial c}$
  - $\phi(a,b)$ is simple and a,b known from forward pass.



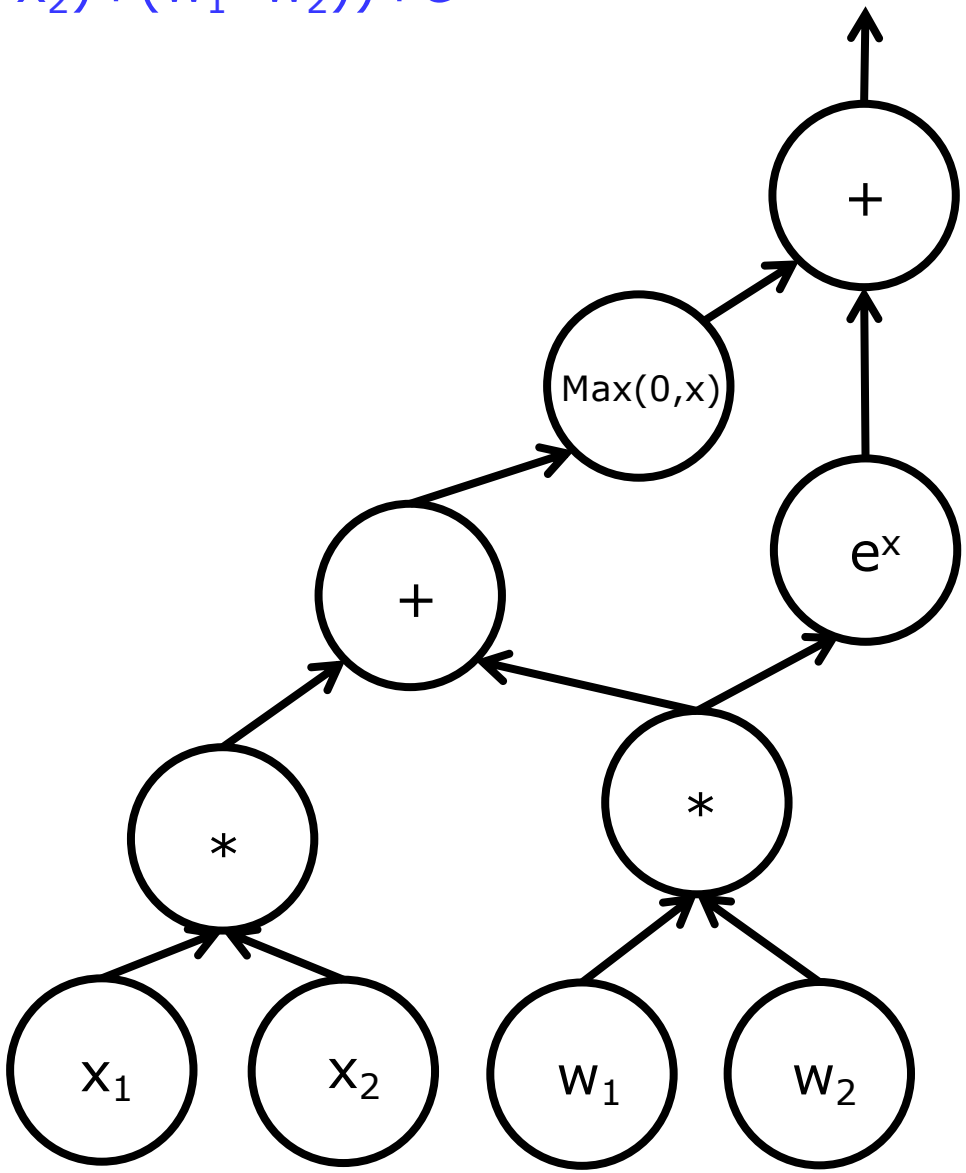
Backpropagation

- Special Rule:
  - If a node sends output to multiple destinations:
    - Backprop computes a gradient on each edge.
    - These must be summed to obtain actual $\frac{\partial f}{\partial c}$ to be used further down the computation graph.

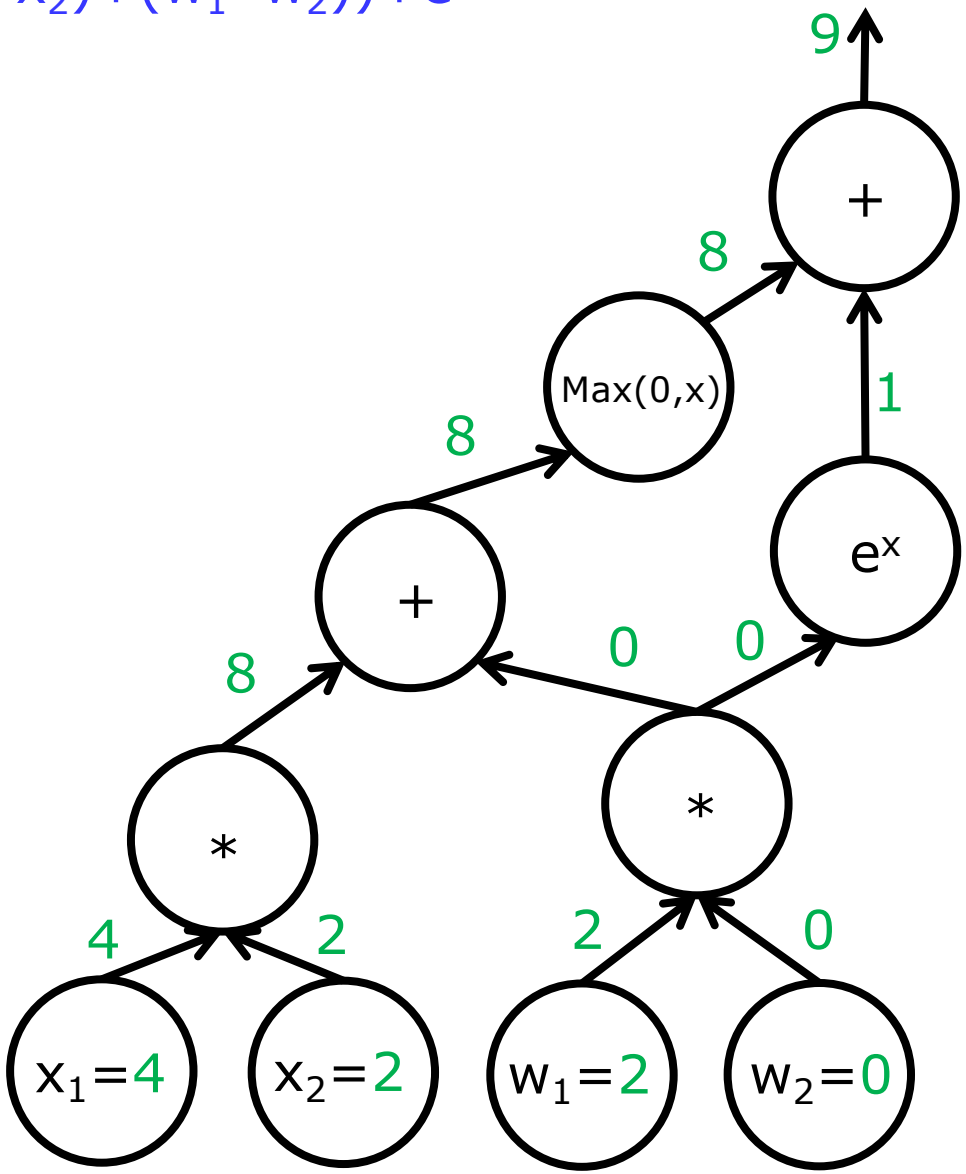




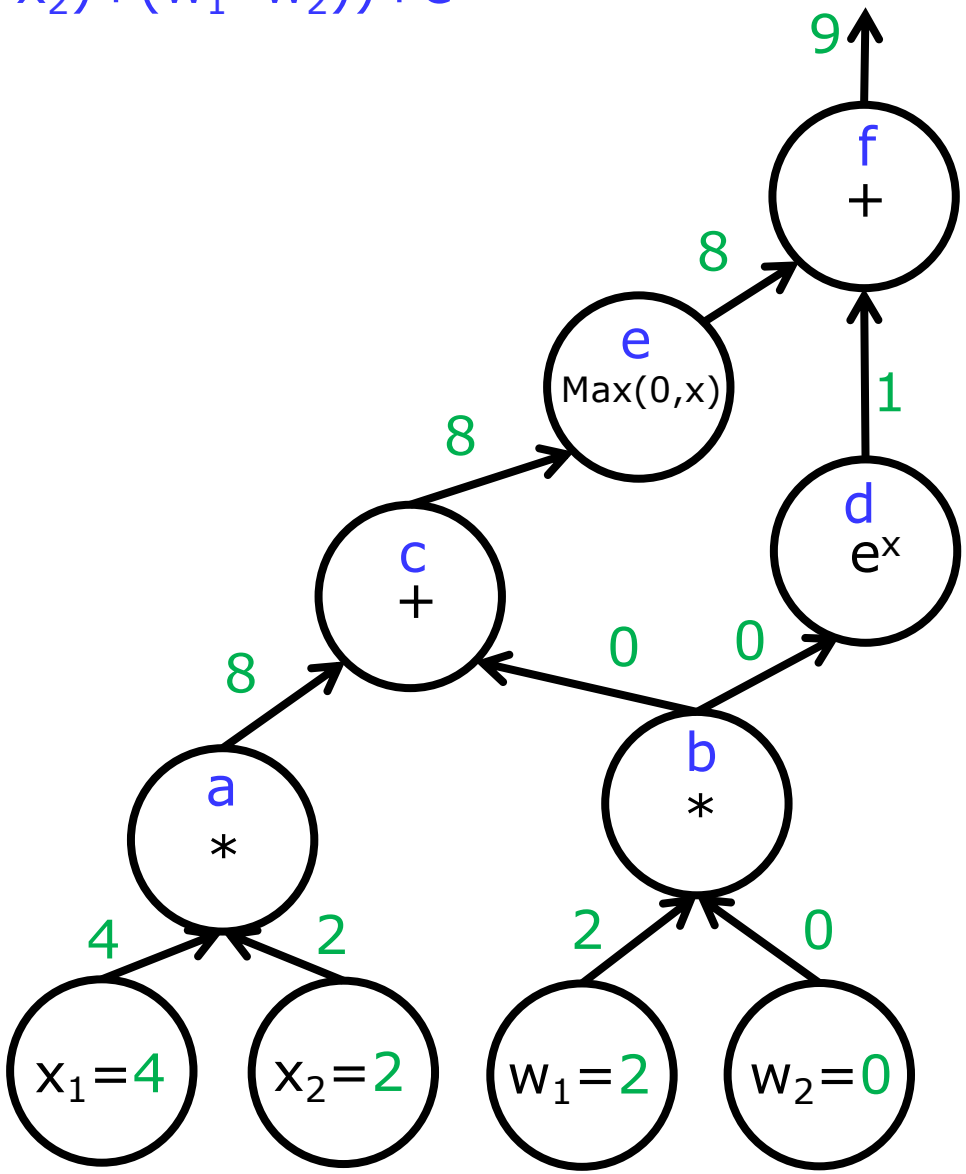
$Max(0,(x_1*x_2)+(w_1*w_2))+e^{w1*w2}$



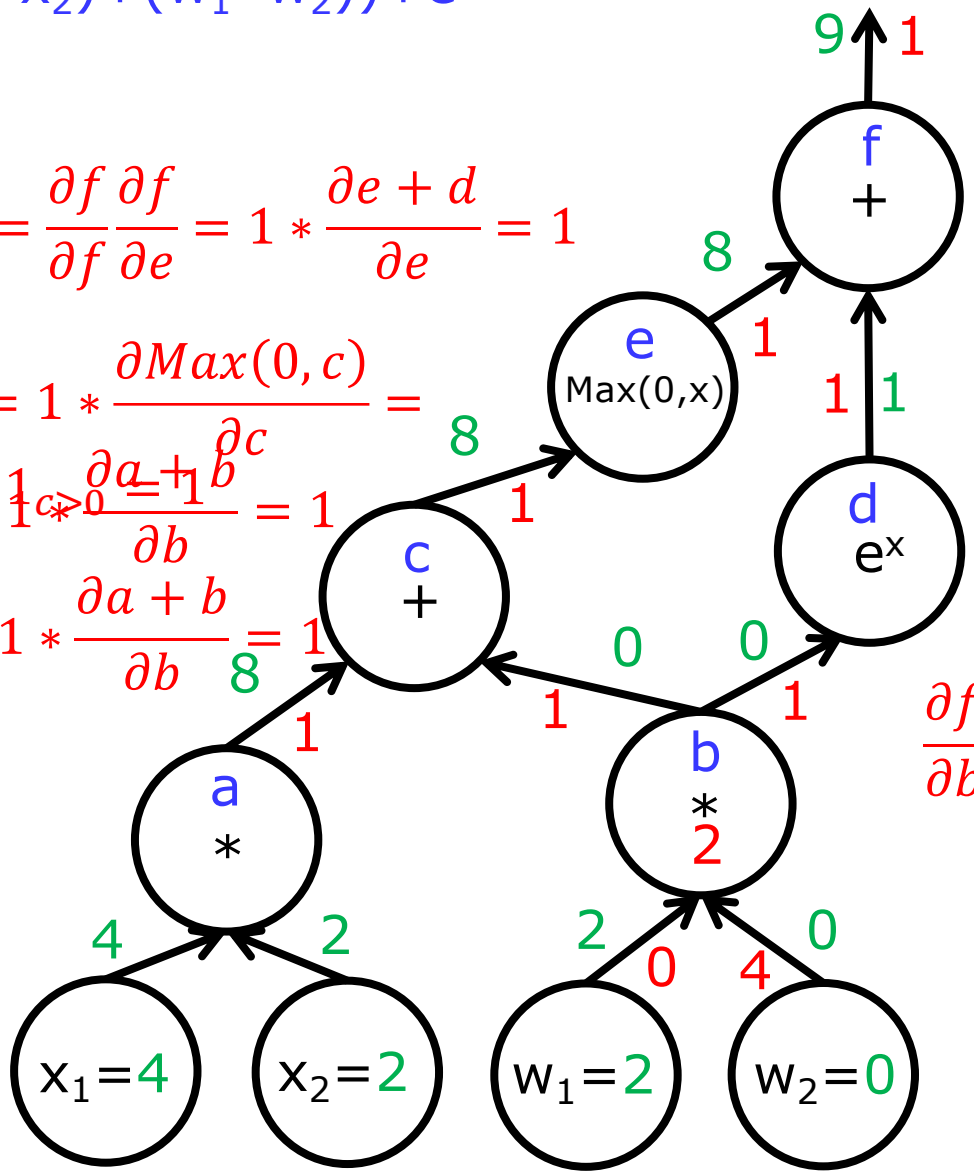
$Max(0,(x_1*x_2)+(w_1*w_2))+e^{w1*w2}$



$Max(0,(x_1*x_2)+(w_1*w_2))+e^{w1*w2}$



$Max(0,(x_1*x_2)+(w_1*w_2))+e^{w1*w2}$



$$\frac{\partial f}{\partial f} = 1$$

$$\frac{\partial f}{\partial e} = \frac{\partial f}{\partial f}\frac{\partial f}{\partial e} = 1 * \frac{\partial e + d}{\partial e} = 1$$

$$\frac{\partial f}{\partial d} = \frac{\partial f}{\partial f}\frac{\partial f}{\partial d} = 1 * \frac{\partial e + d}{\partial d} = 1$$

$$\frac{\partial f}{\partial c} = \frac{\partial f}{\partial e}\frac{\partial e}{\partial c} = 1 * \frac{\partial Max(0,c)}{\partial c} = 1 * \begin{cases} 1 & c > 0 \\ 0 & c \leq 0 \end{cases} = 1$$

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial b} = 1 * \frac{\partial a + b}{\partial b} = 1$$

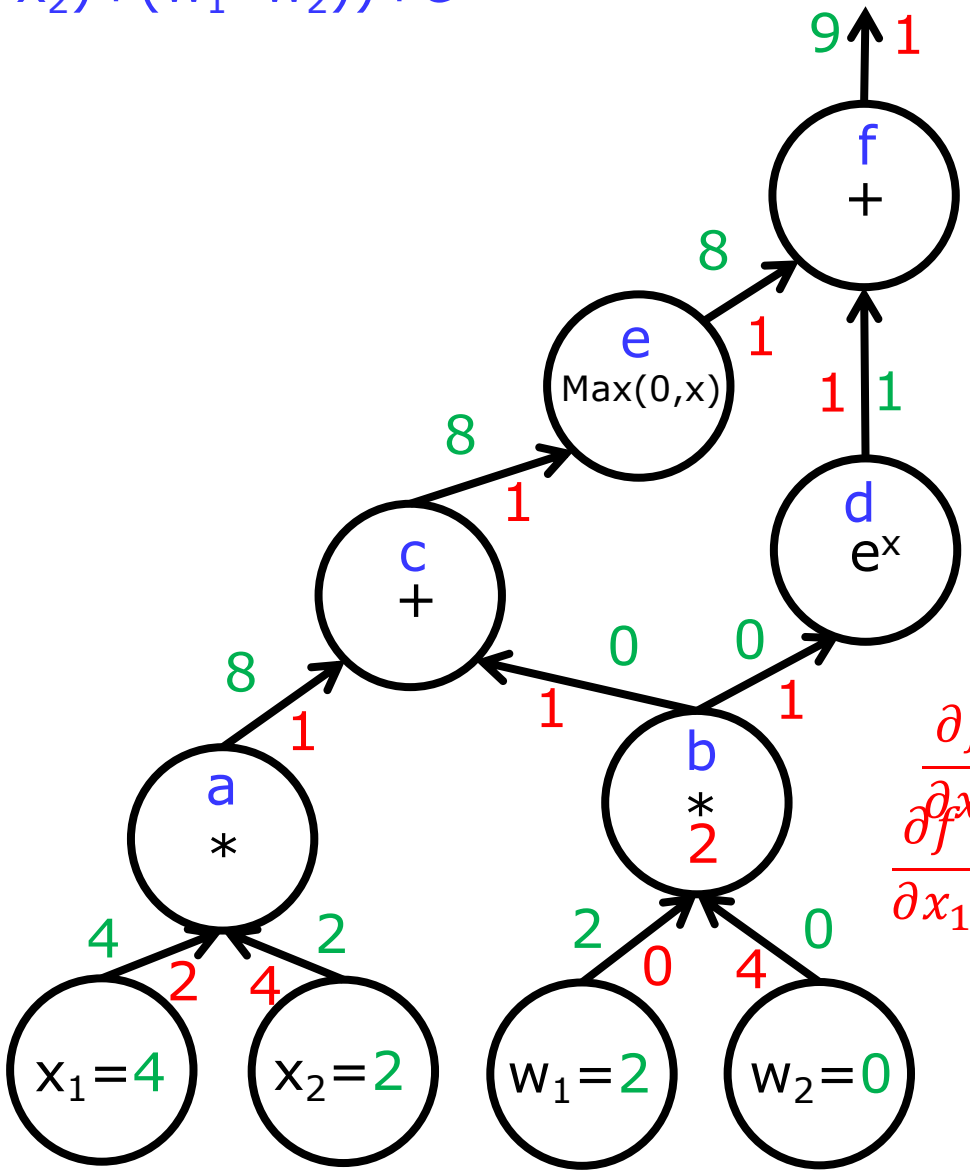
$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial a} = 1 * \frac{\partial a + b}{\partial b} = 1$$

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial d}\frac{\partial d}{\partial b} = 1 * \frac{\partial e^b}{\partial b} = 1 * e^0 = 1$$

$$\frac{\partial f}{\partial w_1} = \frac{\partial f}{\partial b}\frac{\partial b}{\partial w_1} = 2 * \frac{\partial w_1 w_2}{\partial w_1} = 2 * w_2 = 0$$

$$\frac{\partial f}{\partial w_2} = \frac{\partial f}{\partial b}\frac{\partial b}{\partial w_2} = 2 * \frac{\partial w_1 w_2}{\partial w_2} = 2 * w_1 = 4$$

$Max(0,(x_1*x_2)+(w_1*w_2))+e^{w1*w2}$

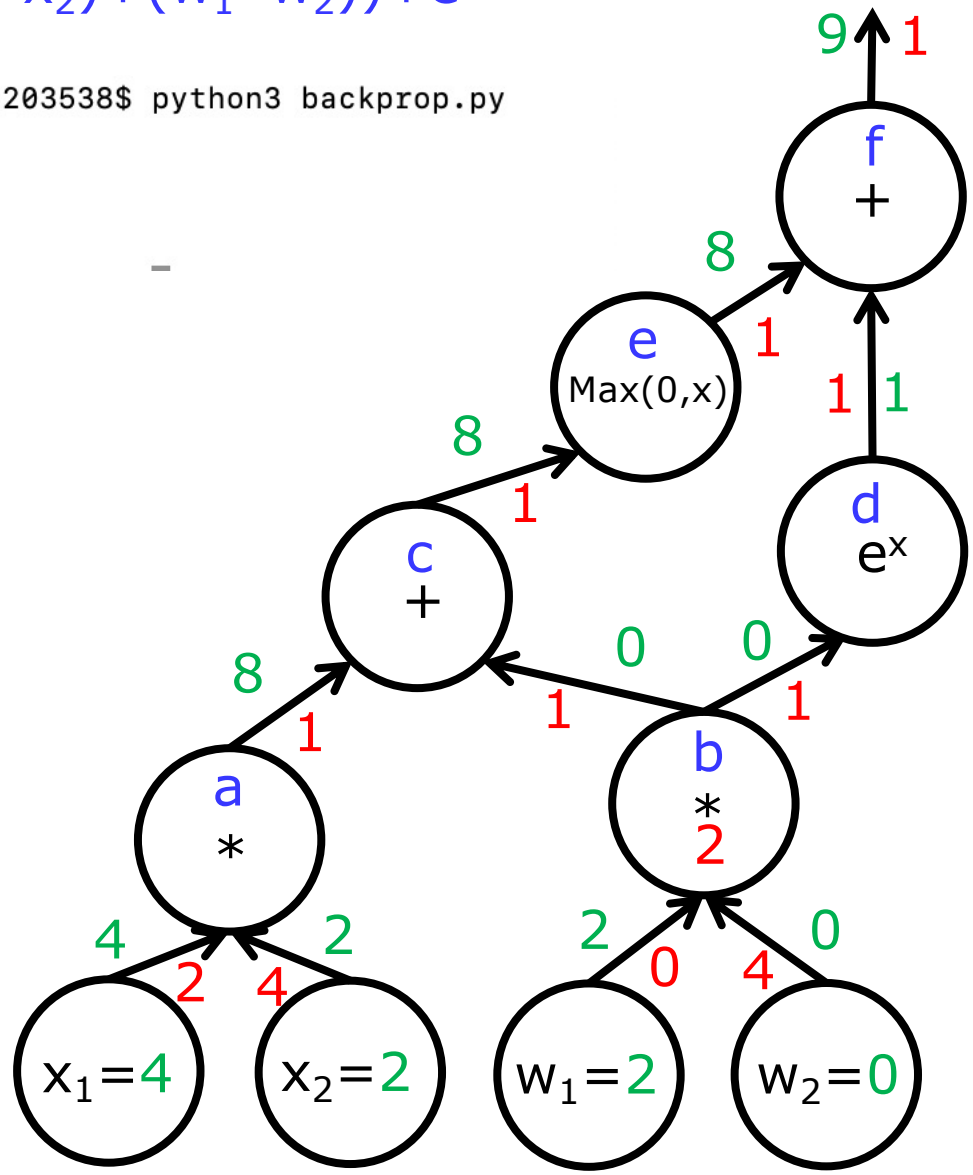


$$\frac{\partial f}{\partial x_2} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial x_2} = 1 * \frac{\partial x_1 x_2}{\partial x_2} = x_1 = 4$$

$$\frac{\partial f}{\partial x_1} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial x_1} = 1 * \frac{\partial x_1 x_2}{\partial x_1} = x_2 = 2$$

$Max(0,(x_1*x_2)+(w_1*w_2))+e^{w1*w2}$

```
d39655:Week5 au203538$ python3 backprop.py
df_dw1: 0.0
df_dw2: 4.0
df_dx1: 2
df_dx2: 4
```




```python
import numpy as np

#Forward Pass
x1=4
x2=2
w1=2
w2=0
a=x1*x2
b=w1*w2
c=a+b
e=max(0,c)
d=np.exp(b)
f=e+d

#Backprop
df_df=1
df_de=1
df_dd=1
dd_db=np.exp(b)
df_db=df_dd*dd_db
de_dc=1 if c>0 else 0
df_dc=df_de*de_dc
dc_da=1
dc_db=1
df_da=df_dc*dc_da
df_db+=df_dc*dc_db
db_dw1=w2
db_dw2=w1
df_dw1=df_db*db_dw1
df_dw2=df_db*db_dw2
da_dx1=x2
da_dx2=x1
df_dx1=df_da*da_dx1
df_dx2=df_da*da_dx2
print('df_dw1:',df_dw1)
print('df_dw2:',df_dw2)
print('df_dx1:',df_dx1)
print('df_dx2:',df_dx2)
```

Stochastic Gradient Descent

- Any point-wise loss:
  - $E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n} L(w(x_i), y_i)$
- Mini-batch SGD:
  - Initialize w (e.g. random N(0,1) distributed er
  - While more time:
    - Permute $(x_1,y_1),\ldots,(x_n,y_n)$ randomly
    - For i=1,...,n/B:
      - Let $(x_1',y_1'),\ldots,(x_B',y_B')$ be next B data items.
      - $g = \frac{1}{B}\sum_{i=1}^{B} \nabla L(w(x_i'), y_i')$
      - $w = w - \eta g$
  - Return w.

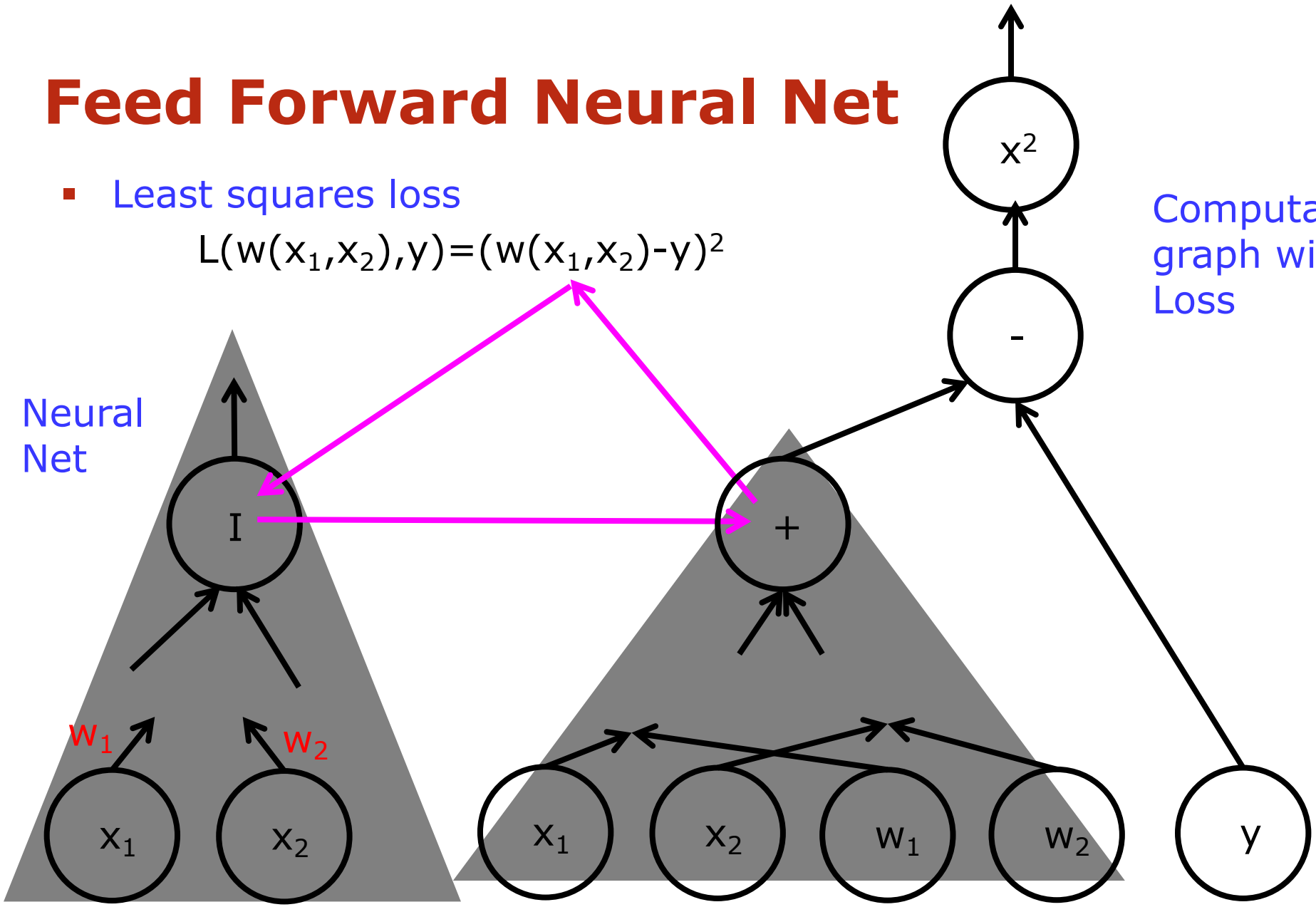
$w(x_i)$ output of neural net with weights and biases w

Computation graph must also include L

Feed Forward Neural Net

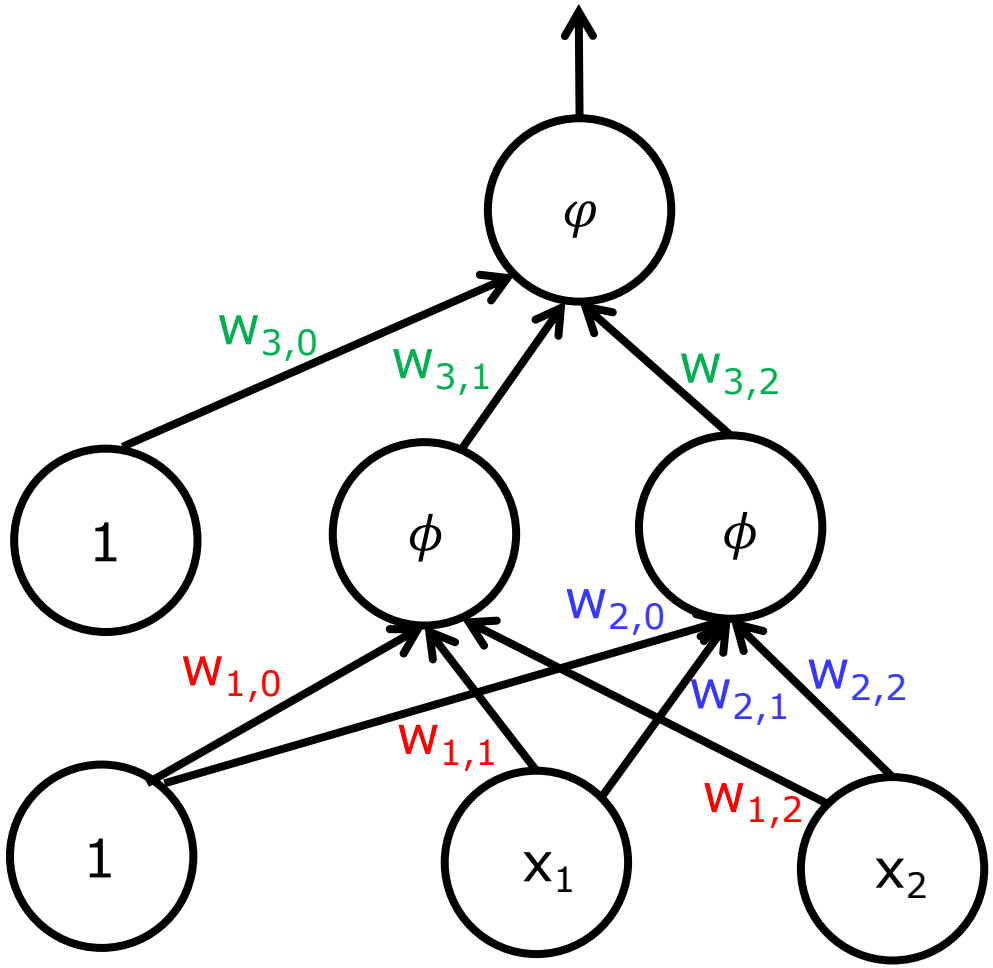
- Least squares loss

$$L(w(x_1,x_2),y)=(w(x_1,x_2)-y)^2$$



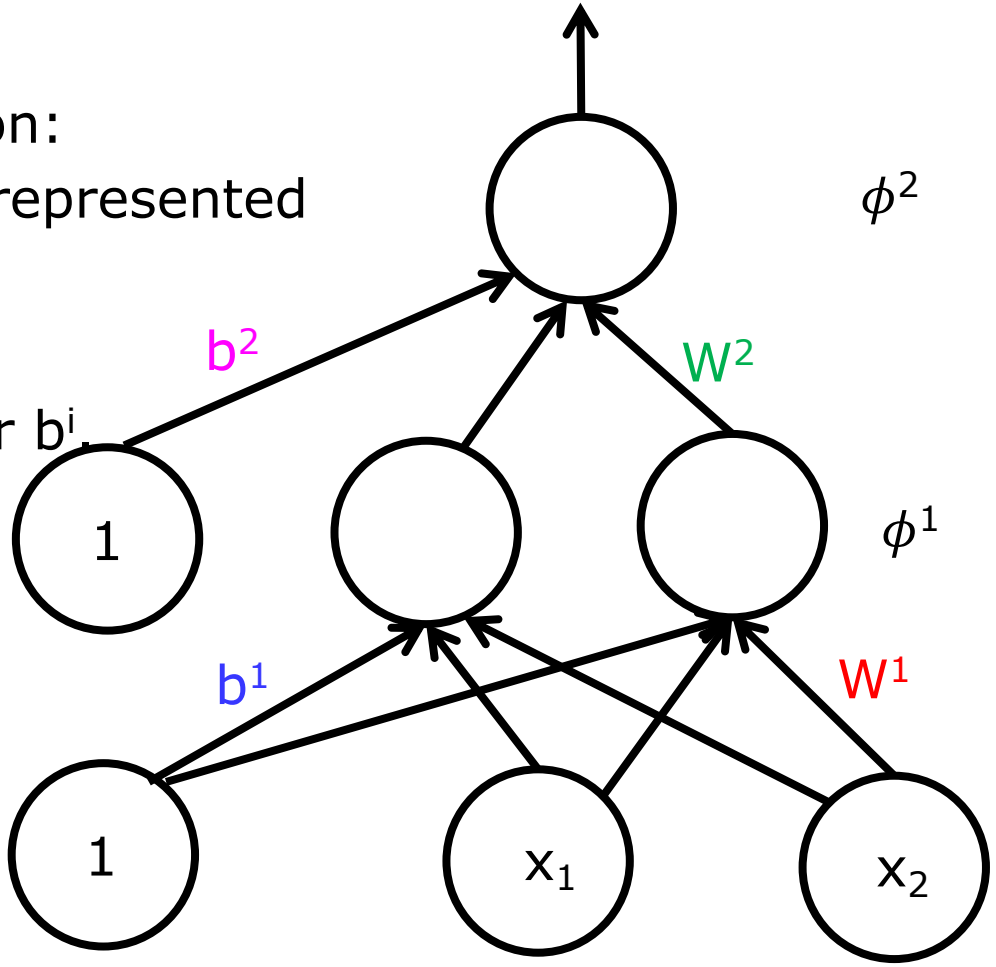
Neural Networks

Feed Forward Neural Net



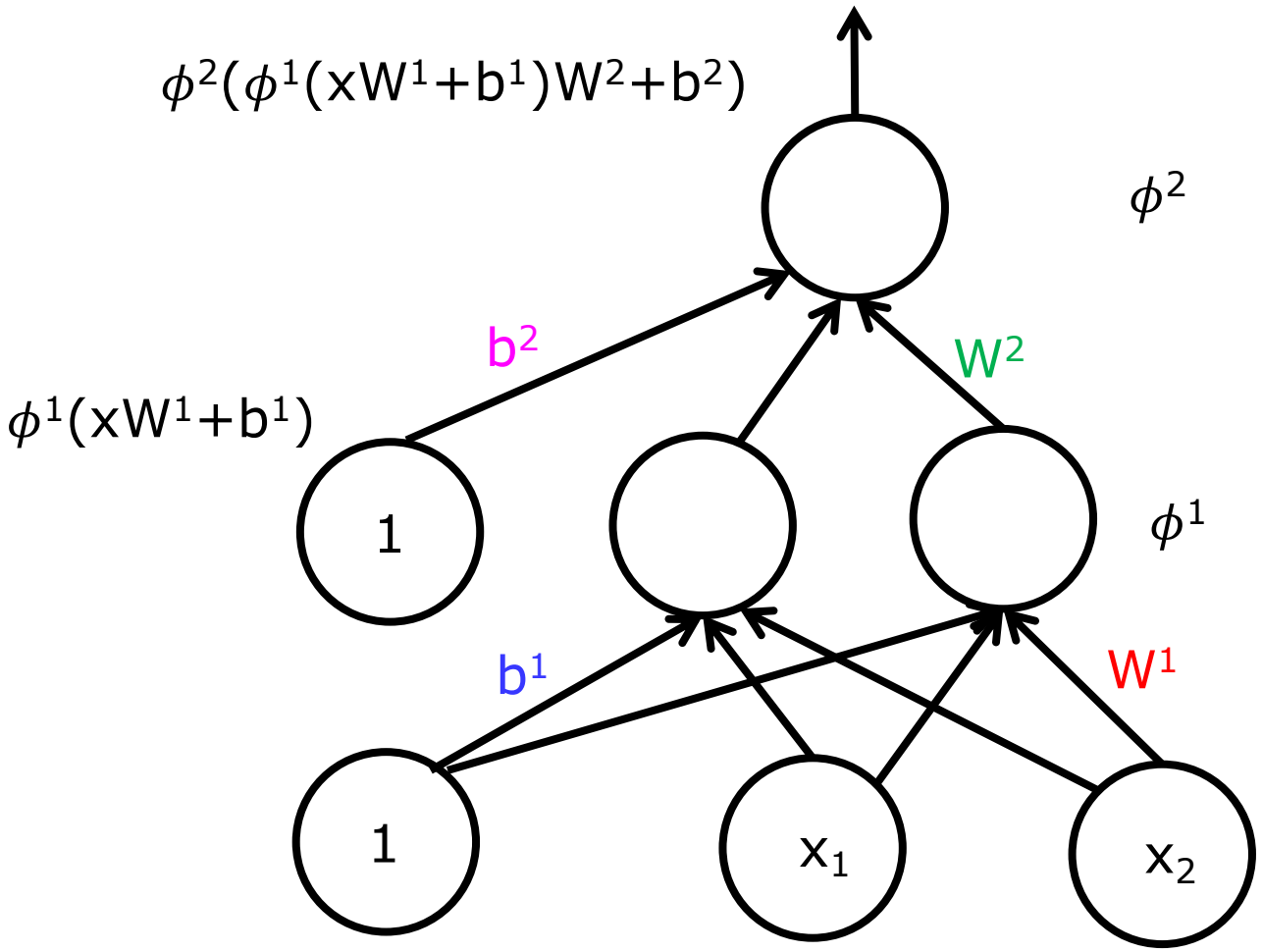
Vectorized Notation

- Fully Connected
  - Convenient representation:
    - All weights in layer i represented by a matrix W^i.
    - All biases in layer i represented by a vector b^i.
    - All neurons in layer i same activation function ϕ^i.



Vectorized Notation

- Fully Connected

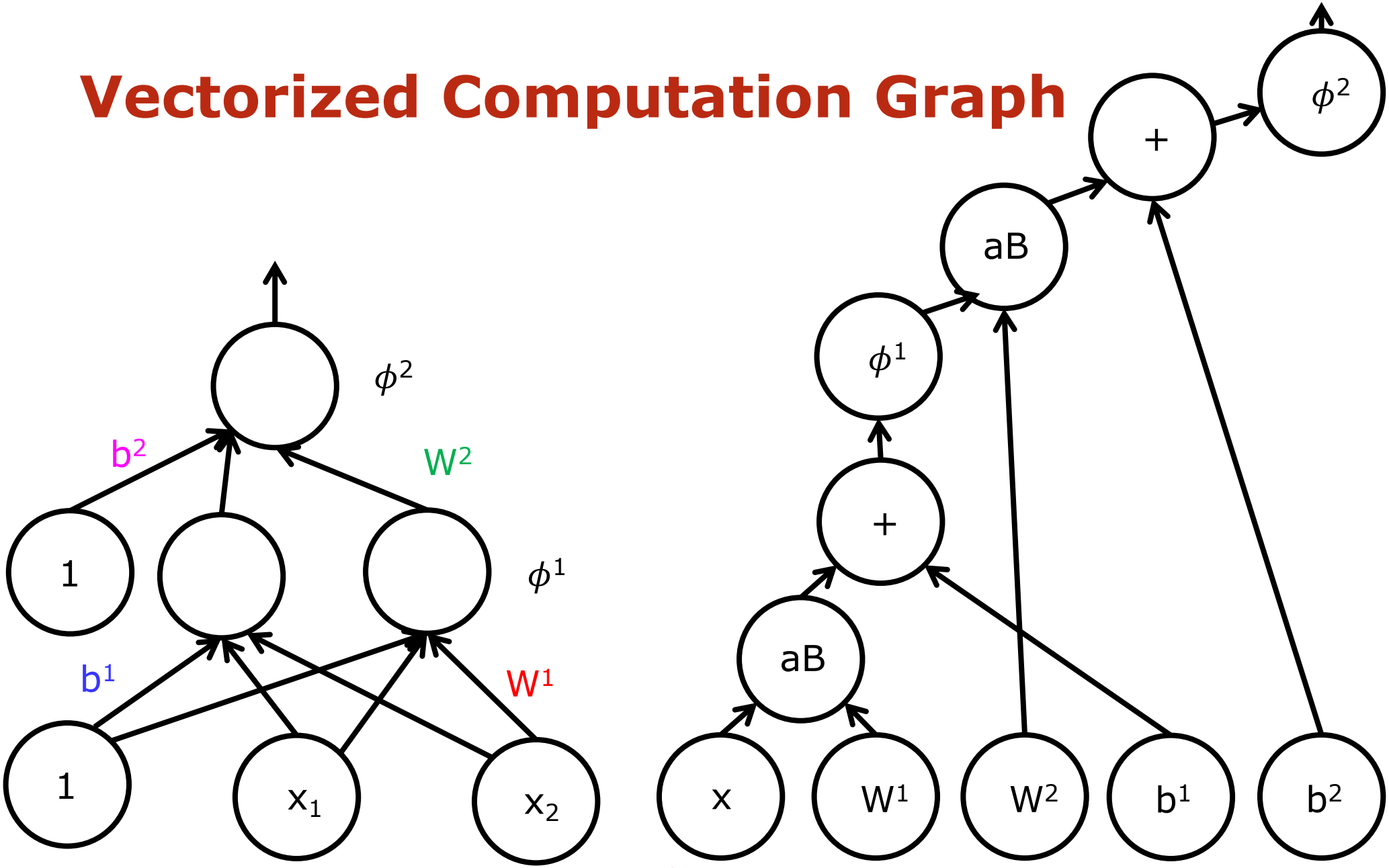


Vectorized Notation

- Fully Connected

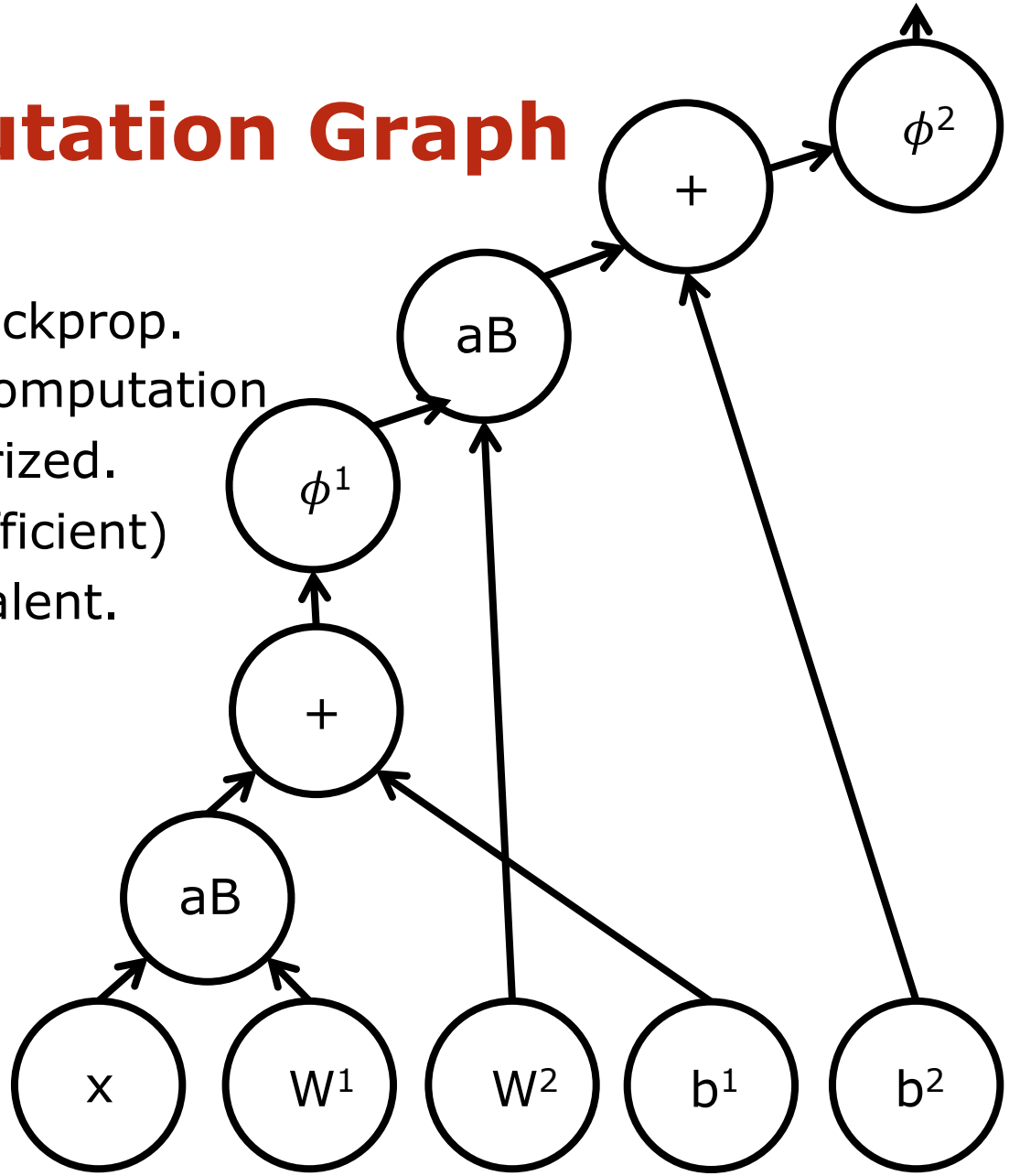
$$\phi^k(\phi^{k-1}(\ldots\phi^2(\phi^1(xW^1+b^1)W^2+b^2)\ldots\ )W^k+b^k)$$

Vectorized Computation Graph



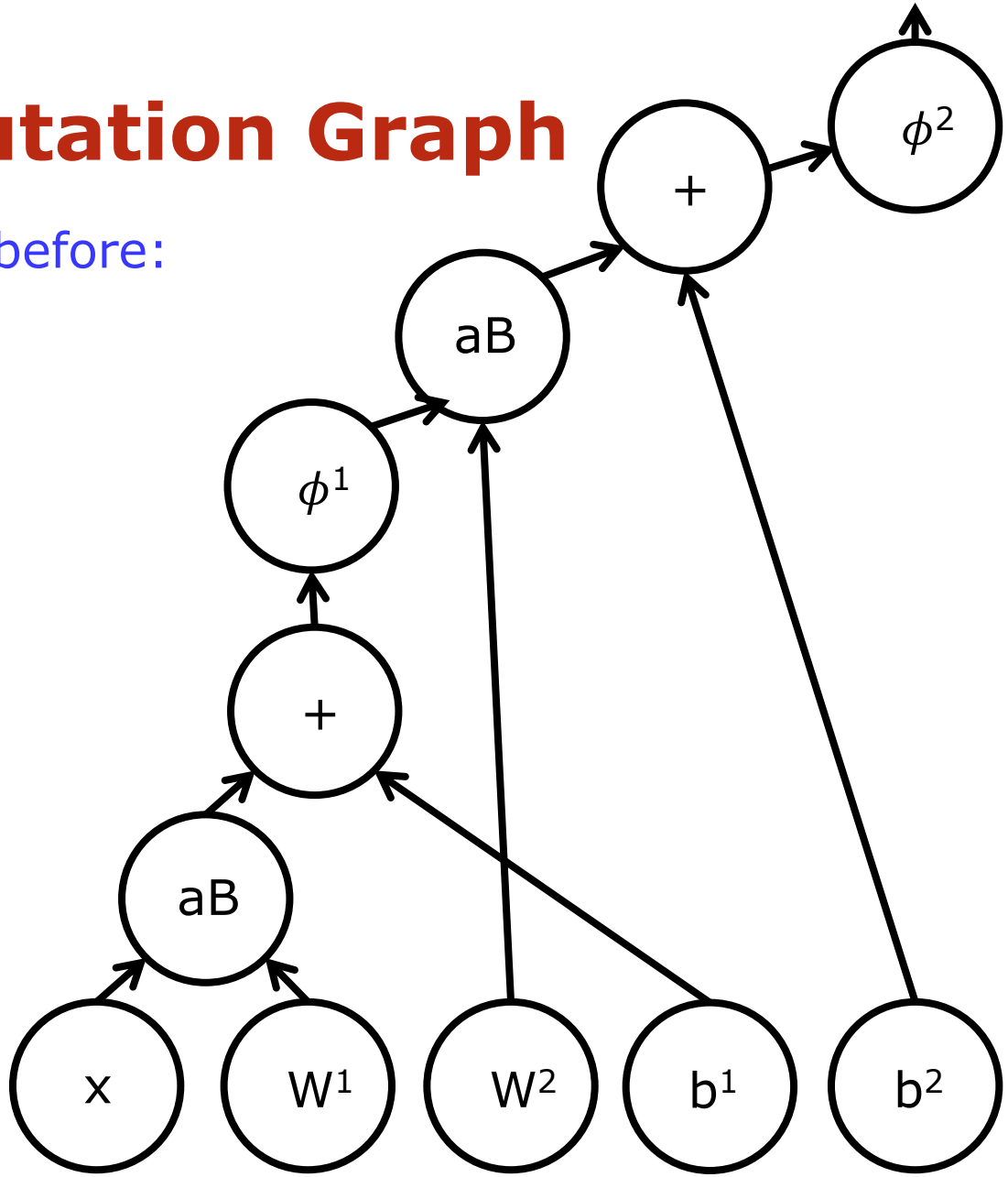
Vectorized Computation Graph

- Compute Gradients.
  - Want to use forward+backprop.
  - Could convert to large computation graph that is not vectorized.
  - More convenient (and efficient) if find vectorized equivalent.



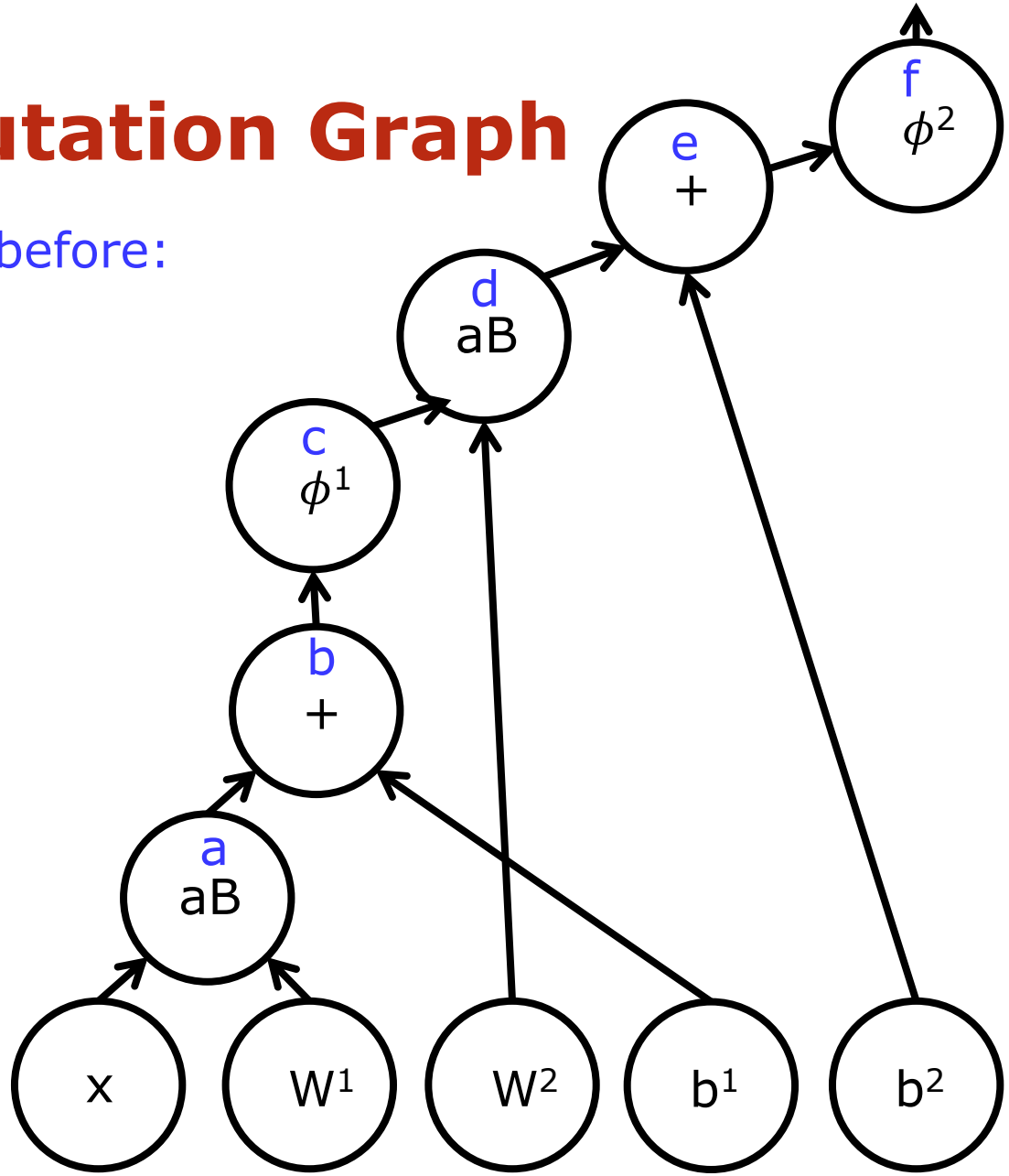
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.



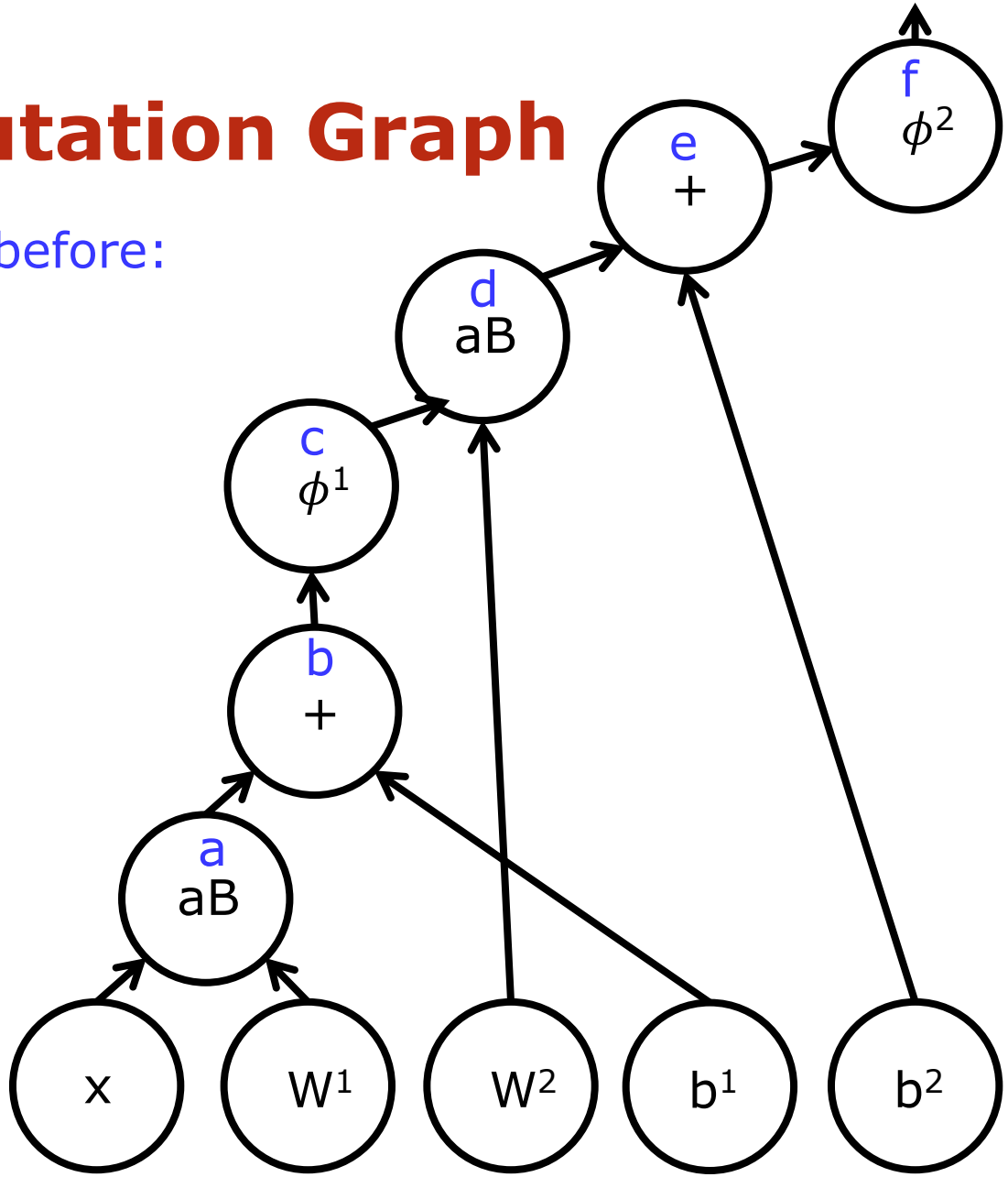
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.



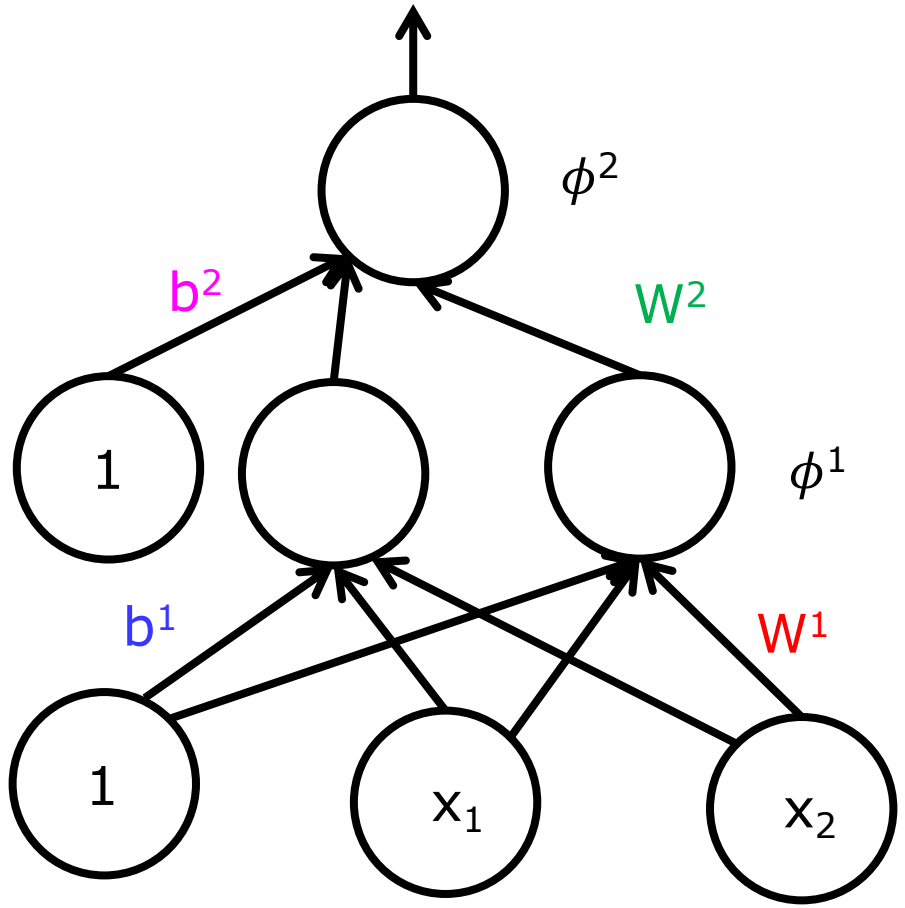
Vectorized Computation Graph

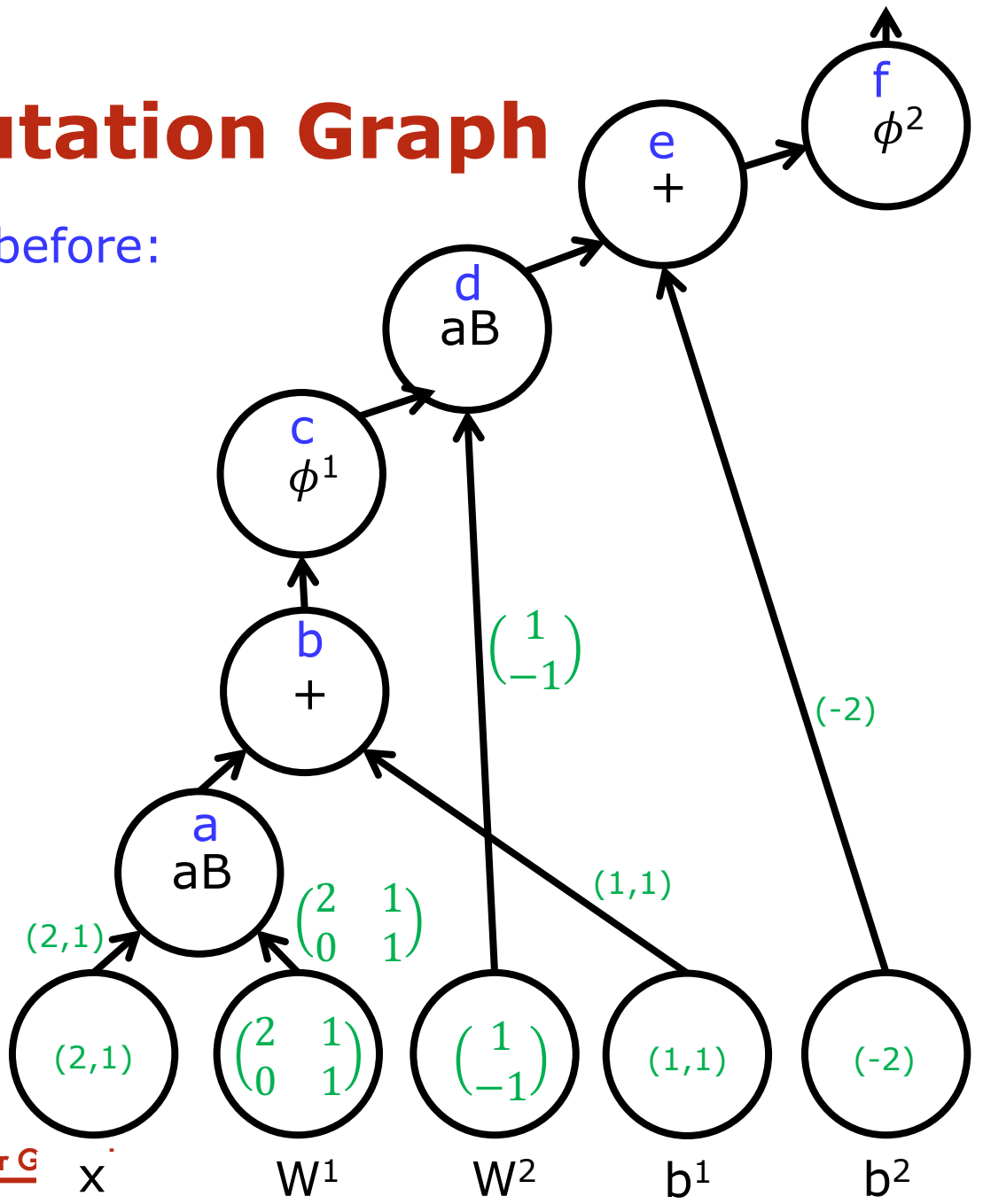
- Let's try same approach as before:
  - Name nodes.
  - Forward pass.



Vectorized Computation Graph

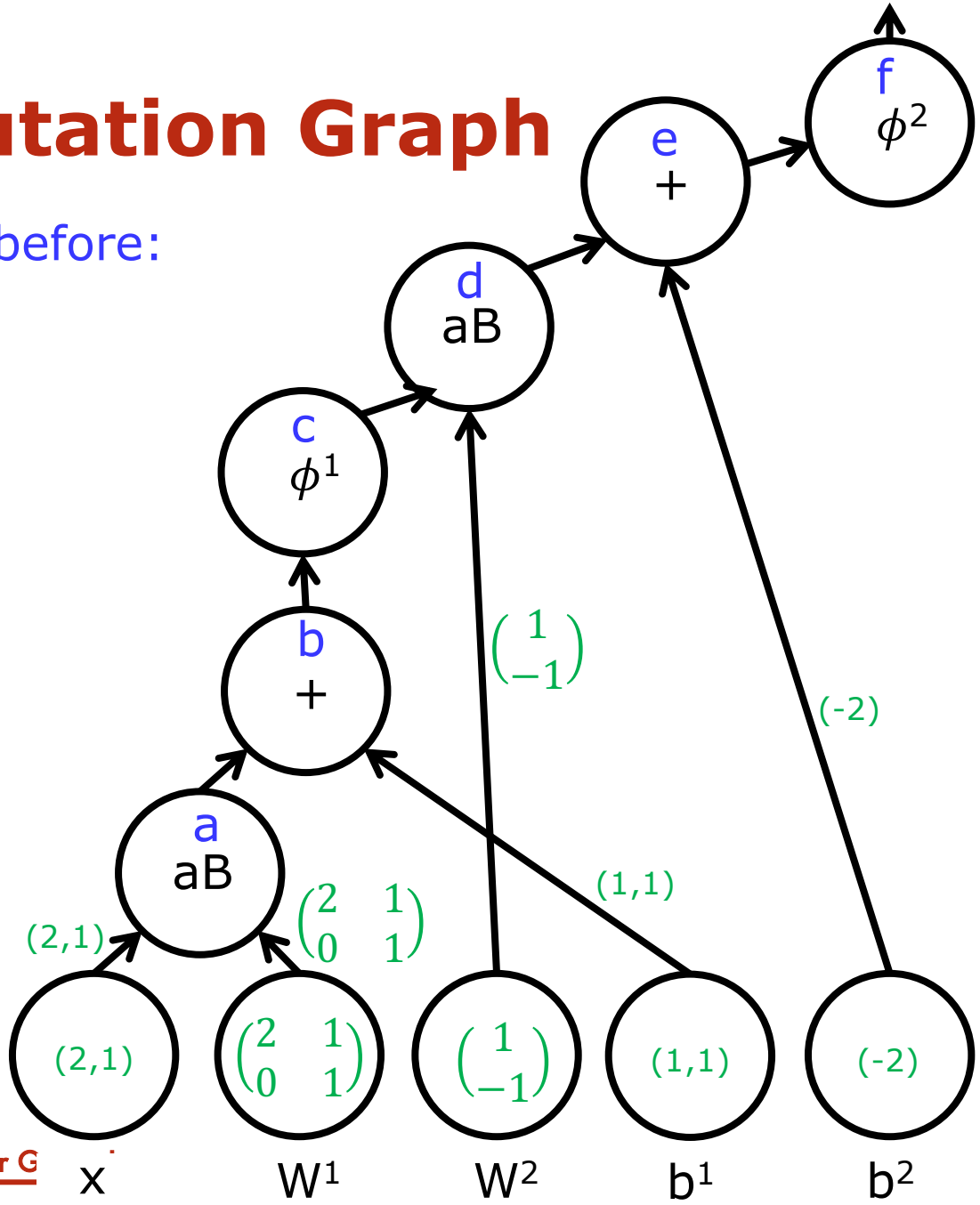
- Let's try same approach as before:





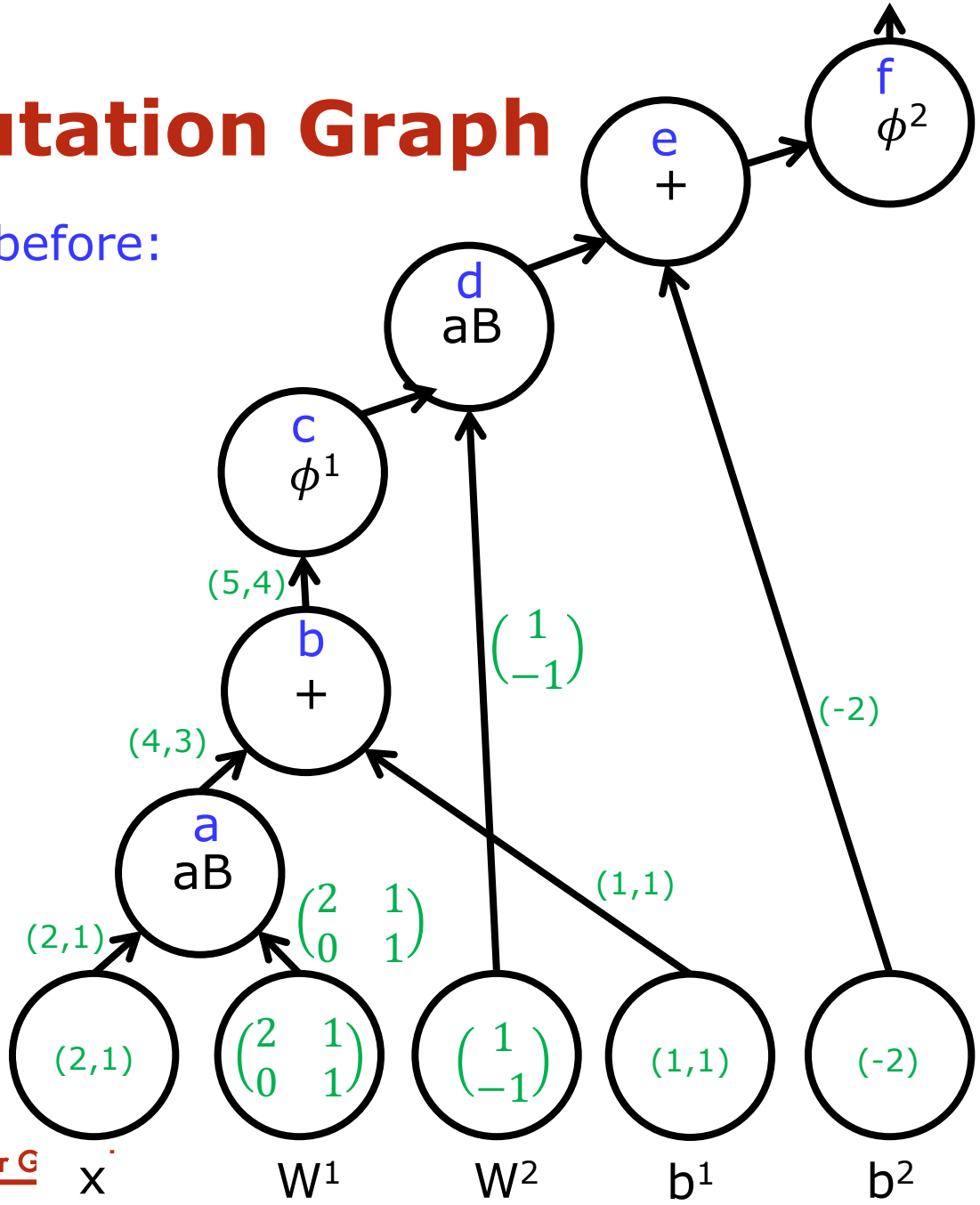
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.



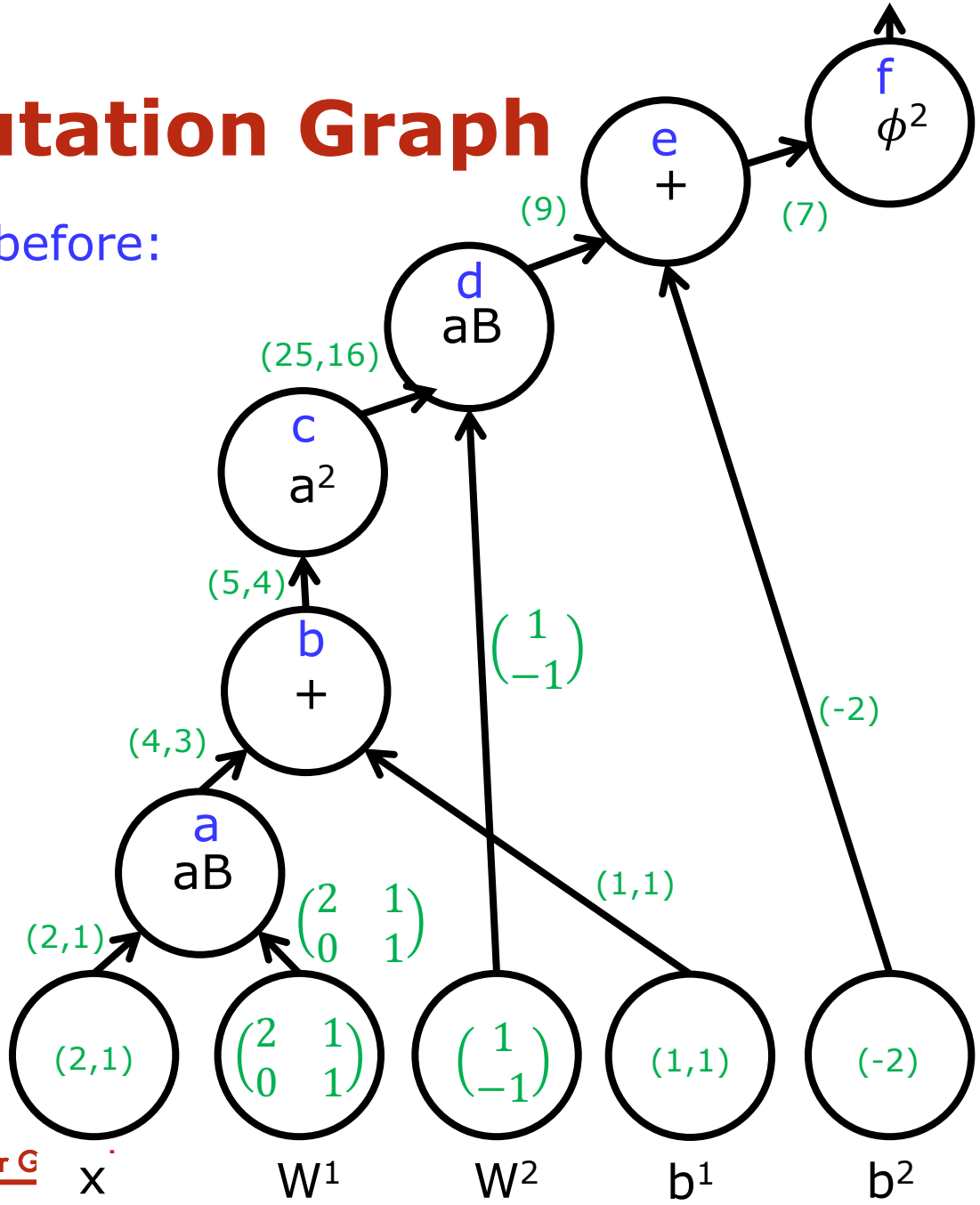
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.



Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.



Vectorized Computation Graph

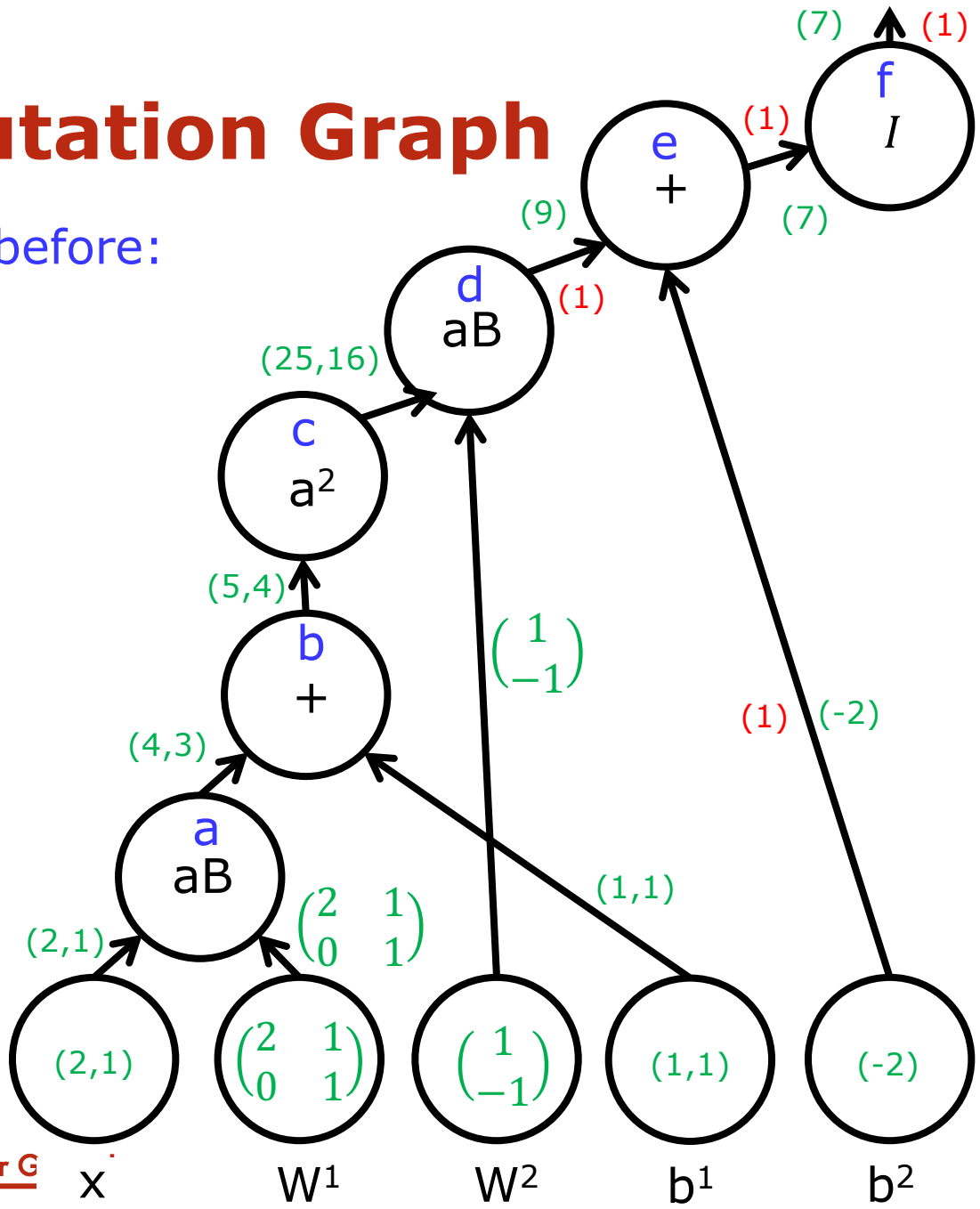
- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial f} = 1$$

$$\frac{\partial f}{\partial e} = \frac{\partial e}{\partial e} = 1$$

$$\frac{\partial f}{\partial d} = \frac{\partial f}{\partial e}\frac{\partial e}{\partial d} = 1 * \frac{\partial d+b^2}{\partial d} = 1$$

$$\frac{\partial f}{\partial b^2} = \frac{\partial f}{\partial e}\frac{\partial e}{\partial b^2} = 1 * \frac{\partial d+b^2}{\partial b^2} = 1$$



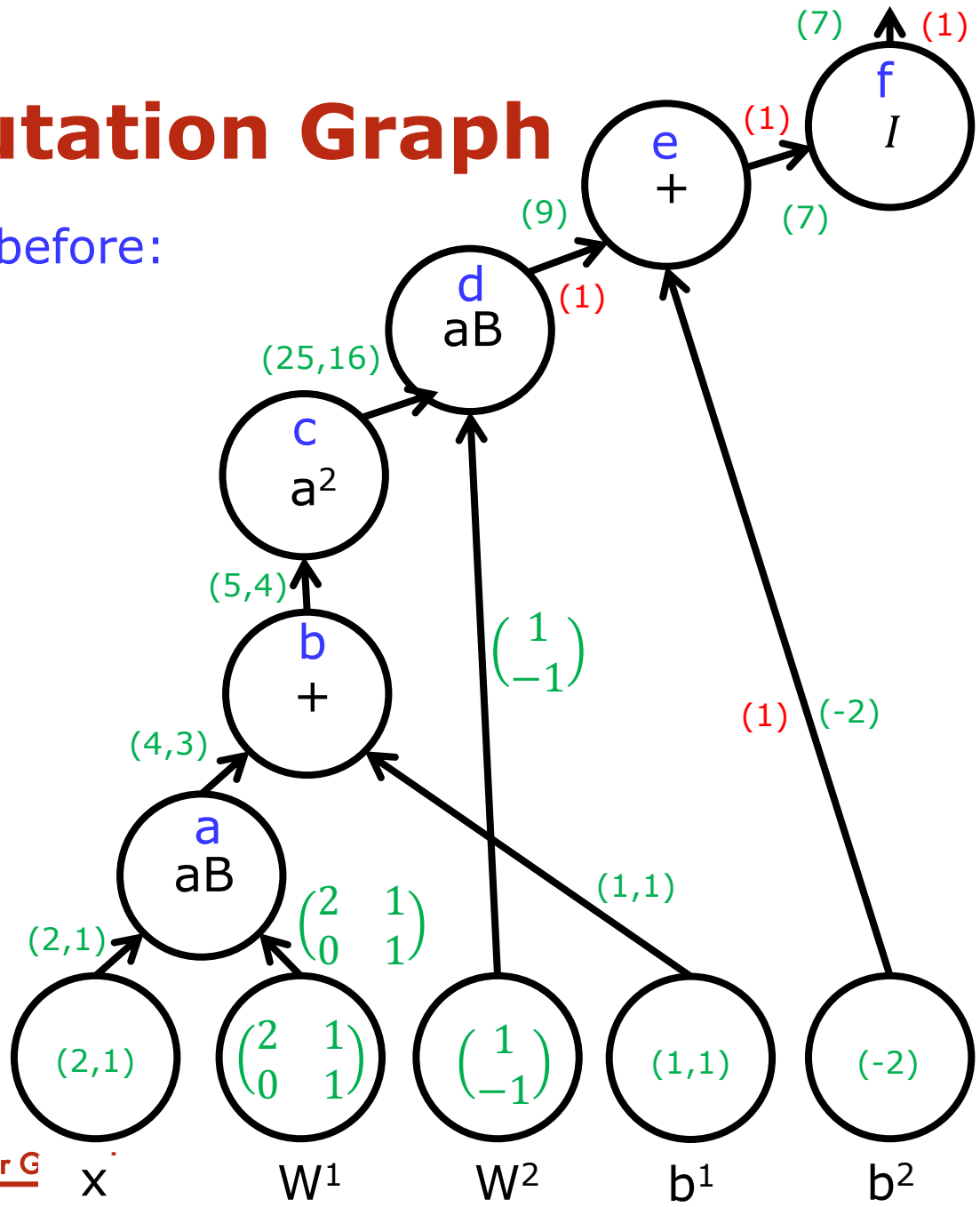
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial c} = \frac{\partial f}{\partial d}\frac{\partial d}{\partial c} = (1)\ \frac{\partial d}{\partial c}$$

c is not a scalar!

Expression still correct, only $\frac{\partial d}{\partial c}$ is the matrix of derivatives $\nabla_c d(c)$



Matrix of Derivatives

- Functions of multiple variables to multiple outputs:
  - If $l = (l_1, \ldots, l_k)$ is a function of variables $x_1, \ldots x_d$, i.e. from R^d to R^k.
  - Assume each l_i differentiable.

- Matrix of derivatives (a.k.a. Jacobian) is k x d:
  - $\nabla_x(l(x)) = \begin{pmatrix} \frac{\partial}{\partial x_1} l_1(x) & \cdots & \frac{\partial}{\partial x_d} l_1(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} l_k(x) & \cdots & \frac{\partial}{\partial x_d} l_k(x) \end{pmatrix}$

Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

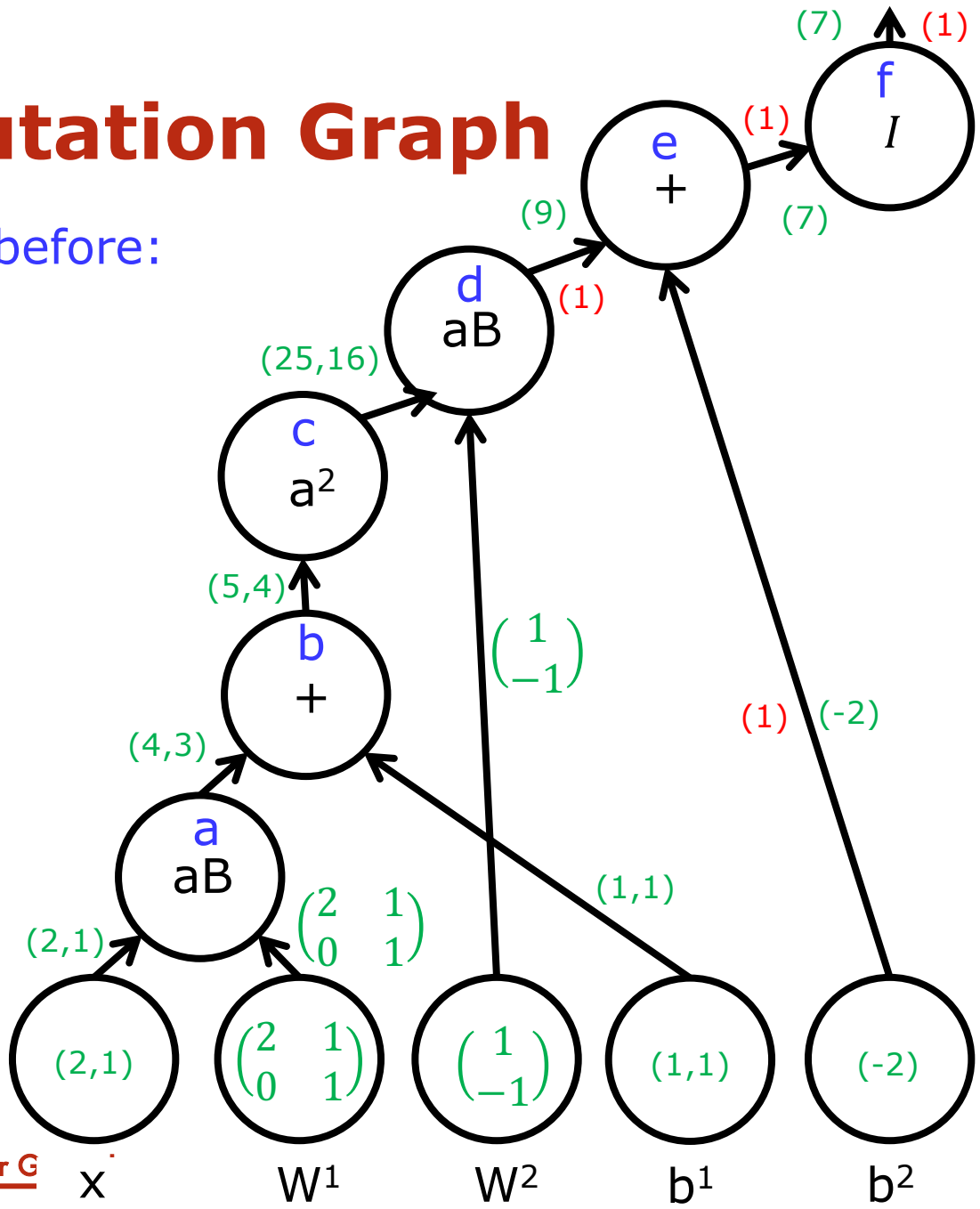
$$\frac{\partial f}{\partial c} = \frac{\partial f}{\partial d}\frac{\partial d}{\partial c} = (1)\frac{\partial d}{\partial c}$$

$$\frac{\partial d}{\partial c} = \nabla_c d(c) = \left(\frac{\partial d}{\partial c_1}, \frac{\partial d}{\partial c_2}\right)$$

$$= \left(\frac{\partial cW^2}{\partial c_1}, \frac{\partial cW^2}{\partial c_2}\right) =$$

$$\left(\frac{\partial c_1 W_1^2 + c_2 W_2^2}{\partial c_1}, \frac{\partial c_1 W_1^2 + c_2 W_2^2}{\partial c_2}\right) =$$

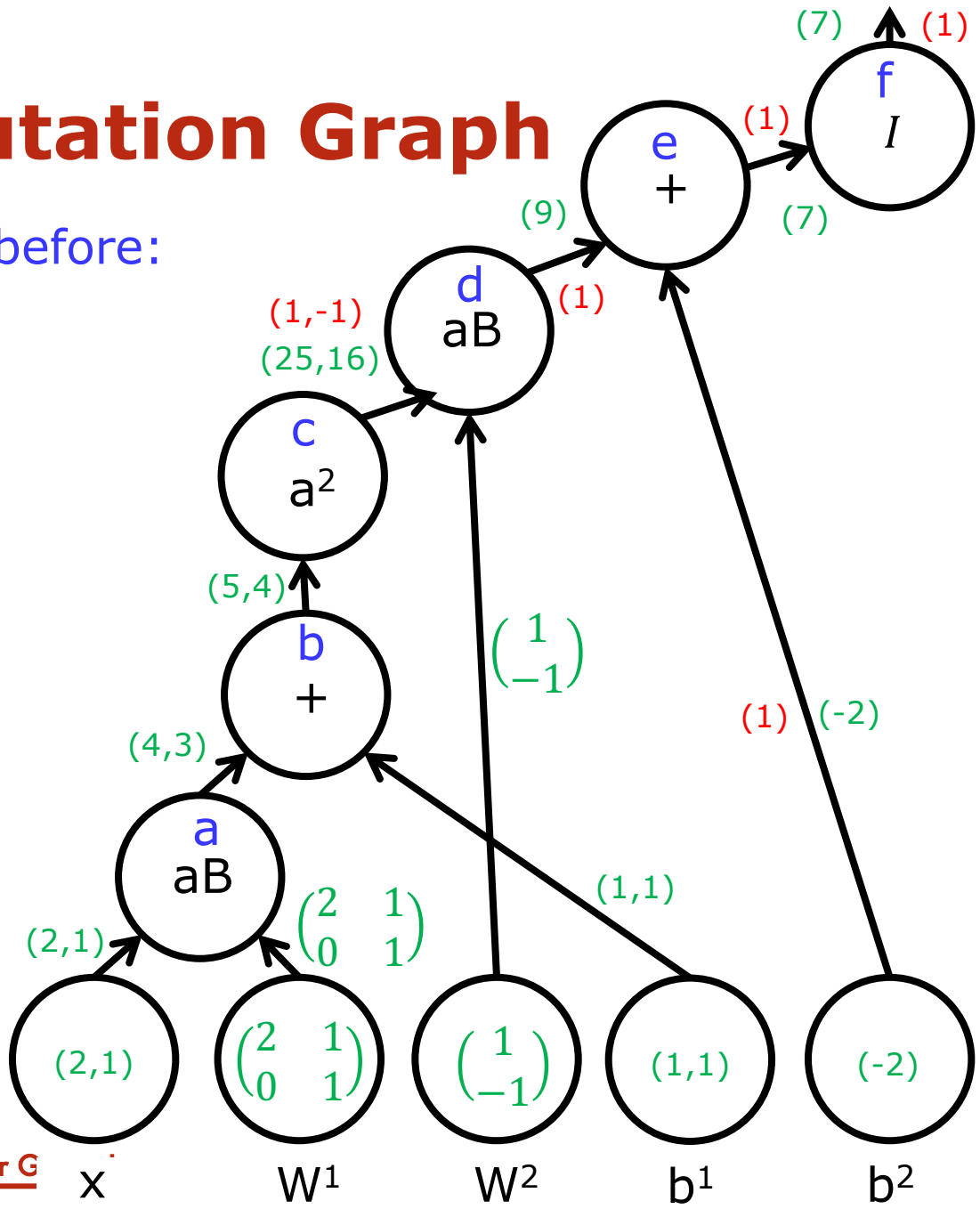
$$(W_1^2, W_2^2) = (1, -1)$$



Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial c} = \frac{\partial f}{\partial d}\frac{\partial d}{\partial c} = (1)(1,-1) = (1,-1)$$

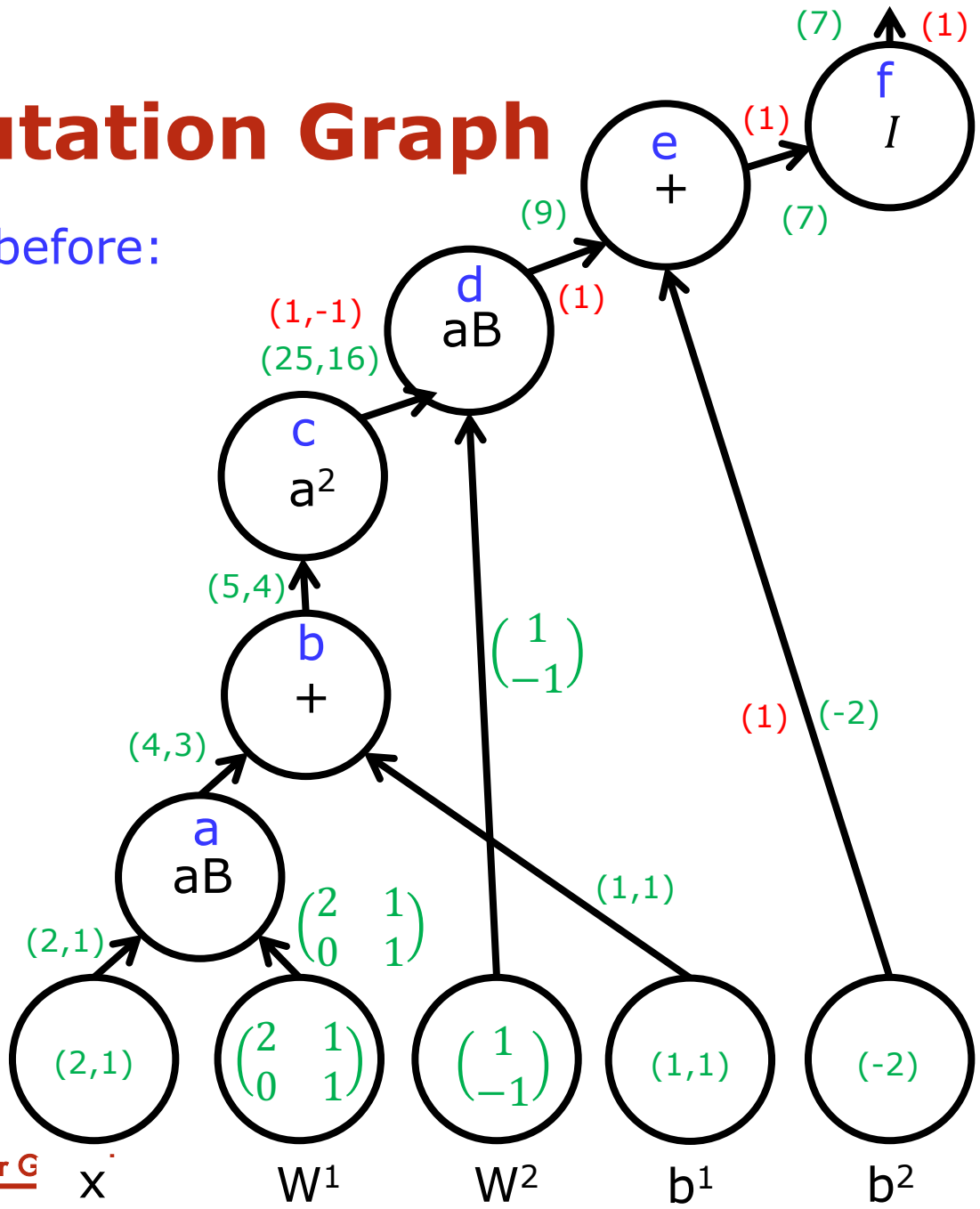


Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial b} = (1,-1)\frac{\partial c}{\partial b}$$

$$\frac{\partial c}{\partial b} = \nabla_b c(b) = \begin{pmatrix} \frac{\partial c_1}{\partial b_1} & \frac{\partial c_1}{\partial b_2} \\ \frac{\partial c_2}{\partial b_1} & \frac{\partial c_2}{\partial b_2} \end{pmatrix}$$



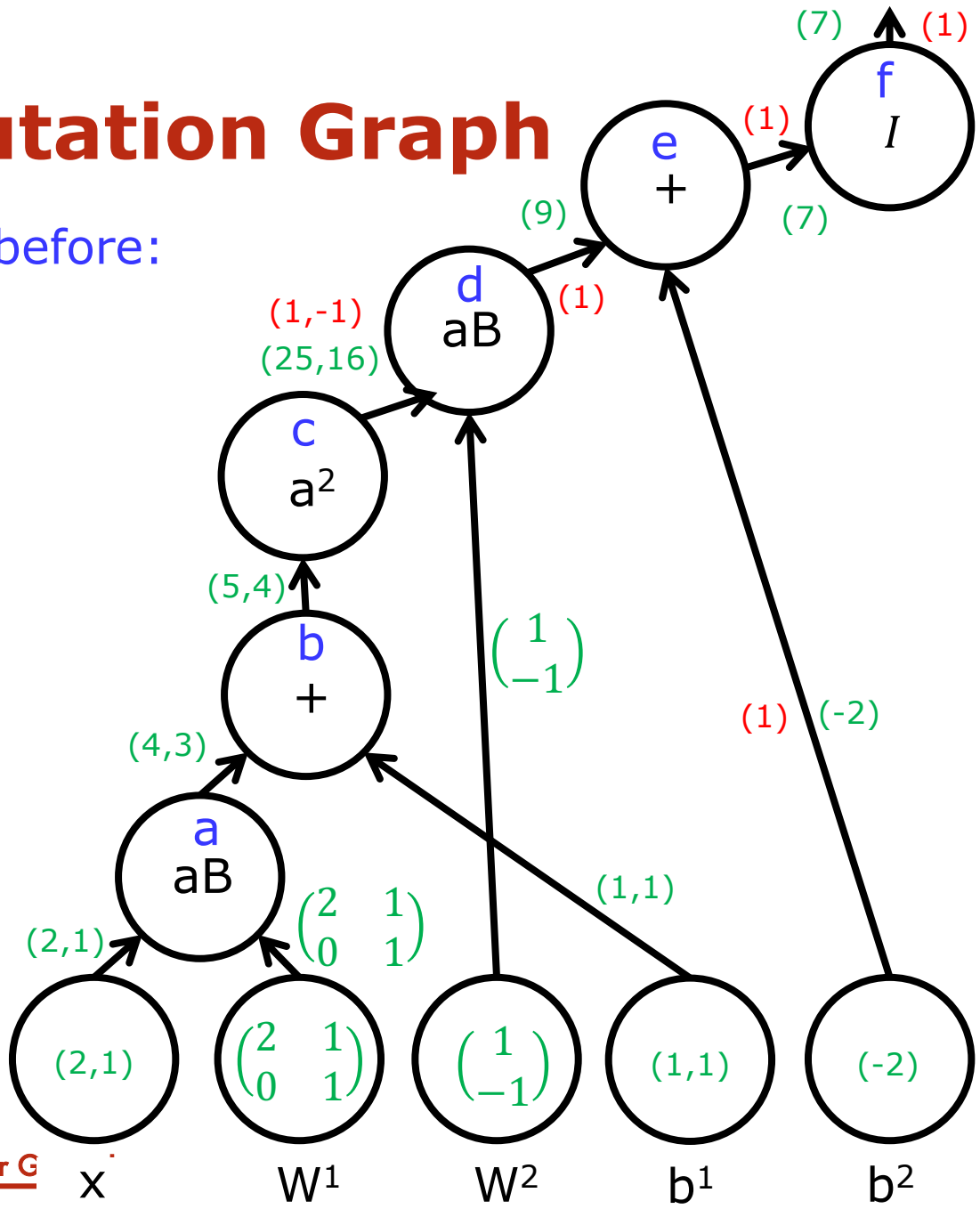
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial b} = (1, -1)\frac{\partial c}{\partial b}$$

$$\frac{\partial c_i}{\partial b_j} = \frac{\partial b_i^2}{\partial b_j}$$

0 if $i \neq j$ else $2b_i$



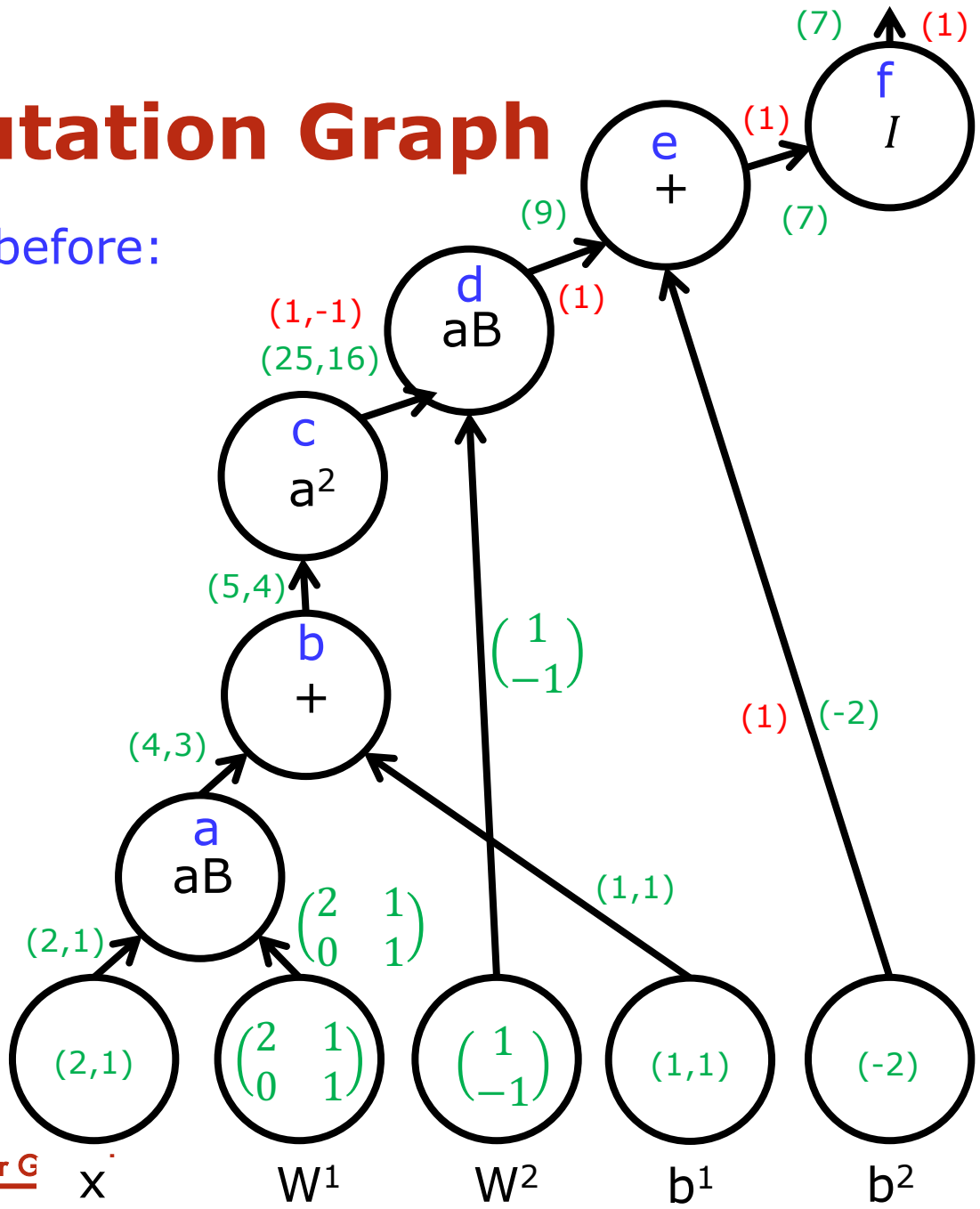
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial b} = (1,-1)\frac{\partial c}{\partial b}$$

$$\frac{\partial c}{\partial b} = \nabla_b c(b) = \begin{pmatrix} 2b_1 & 0 \\ 0 & 2b_2 \end{pmatrix}$$

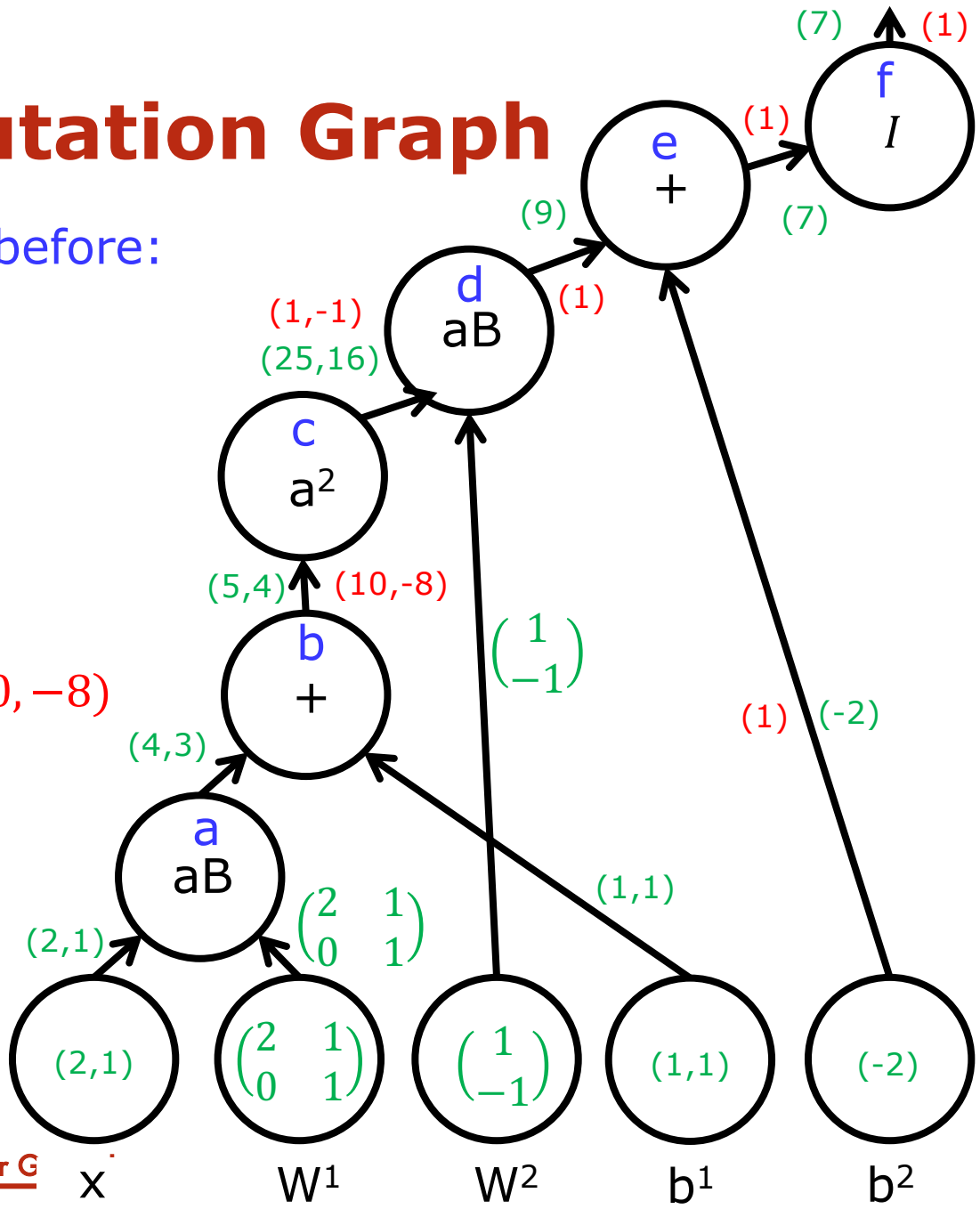
$$\frac{\partial c}{\partial b} = \begin{pmatrix} 10 & 0 \\ 0 & 8 \end{pmatrix}$$



Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial b} = (1,-1)\begin{pmatrix}10 & 0\\ 0 & 8\end{pmatrix} = (10,-8)$$

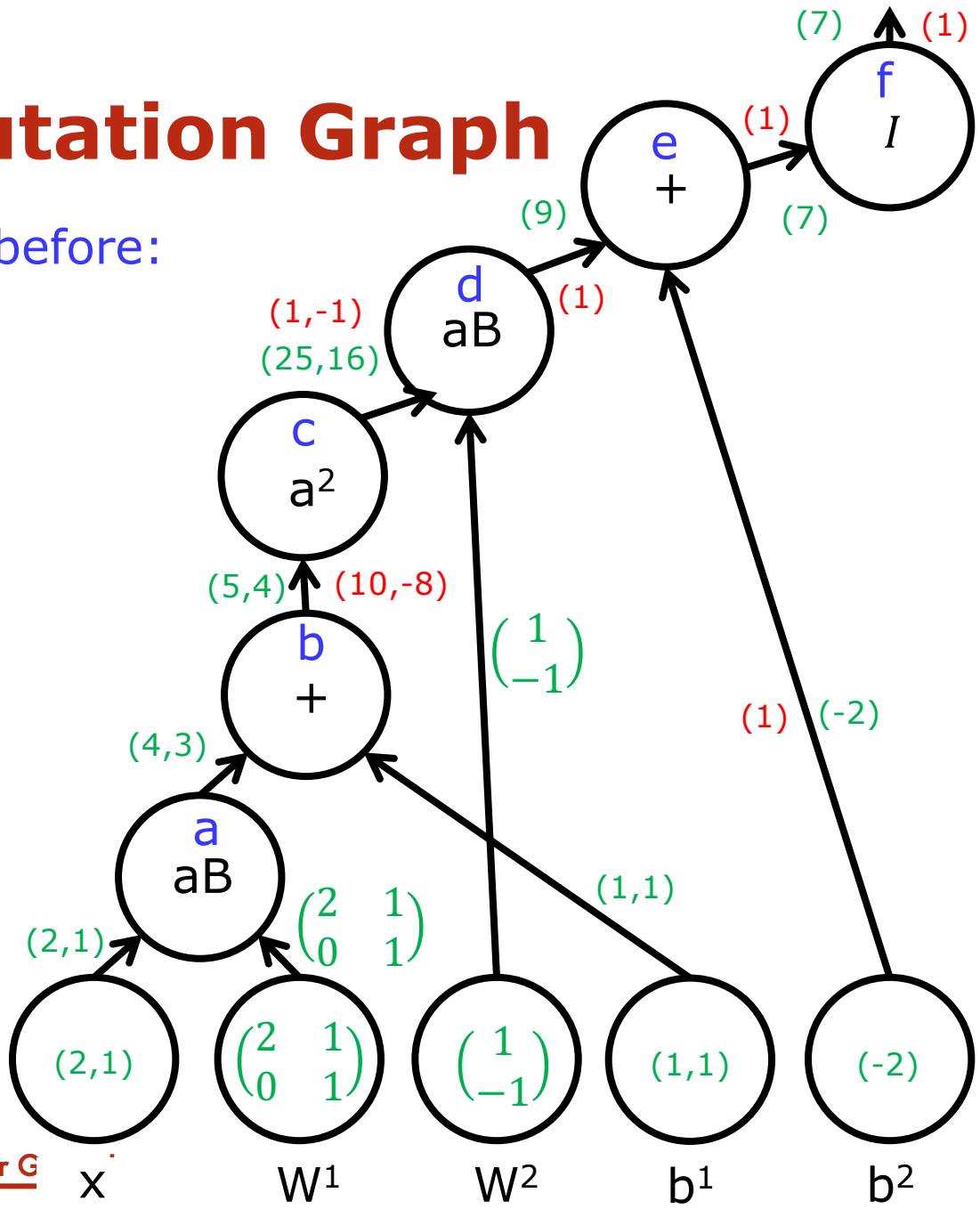


Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial b^1} = \frac{\partial f}{\partial b}\frac{\partial b}{\partial b^1} = (10, -8)\nabla_{b^1} b(b^1)$$

$$\nabla_{b^1} b(b^1) = \begin{pmatrix} \frac{\partial a_1 + b_1^1}{\partial b_1^1} & \frac{\partial a_1 + b_1^1}{\partial b_2^1} \\ \frac{\partial a_2 + b_2^1}{\partial b_1^1} & \frac{\partial a_2 + b_2^1}{\partial b_2^1} \end{pmatrix}$$



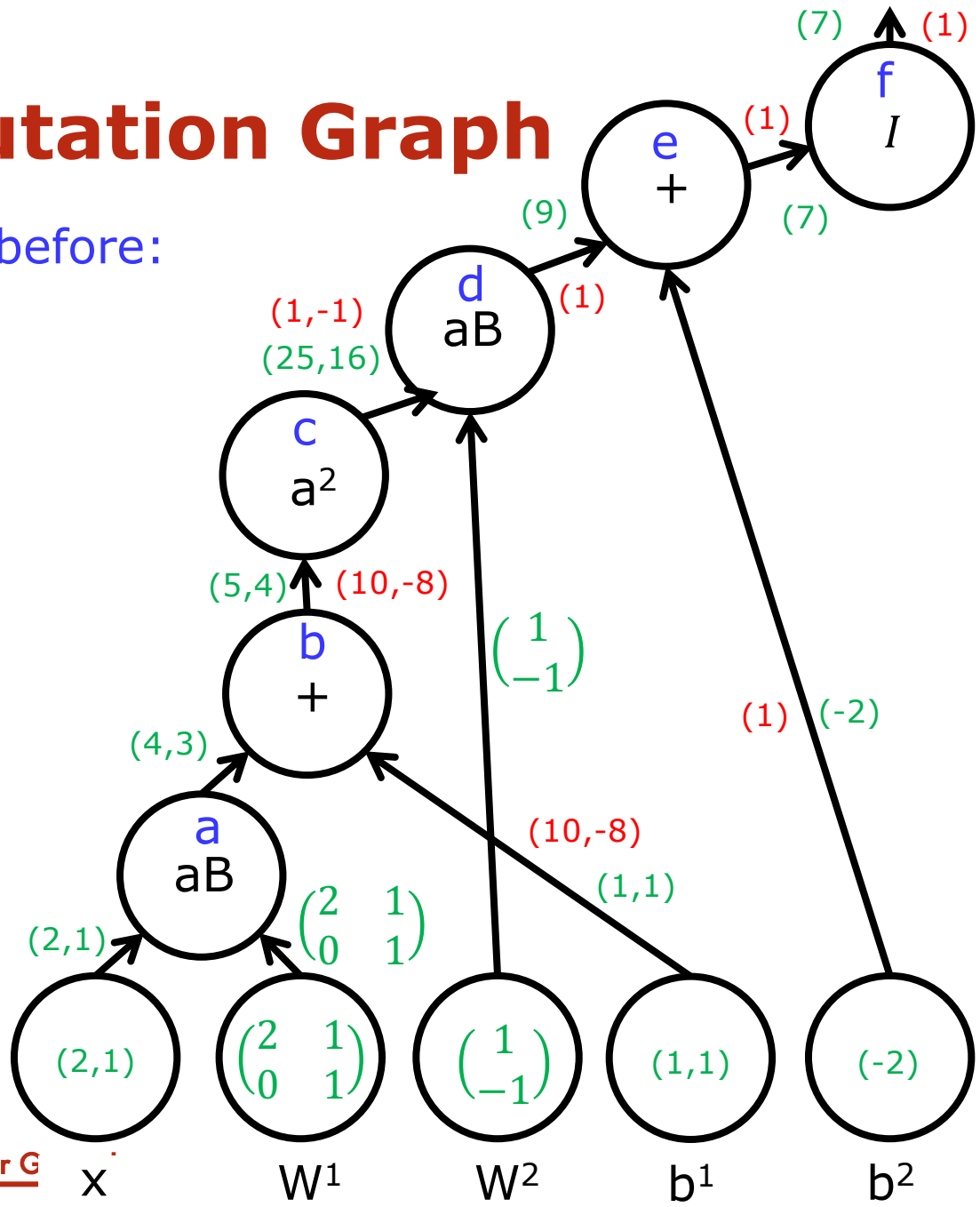
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial b^1} = \frac{\partial f}{\partial b}\frac{\partial b}{\partial b^1} = (10, -8)\nabla_{b^1} b(b^1)$$

$$\nabla_{b^1} b(b^1) = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$\frac{\partial f}{\partial b^1} = (10, -8)\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = (10, -8)$$



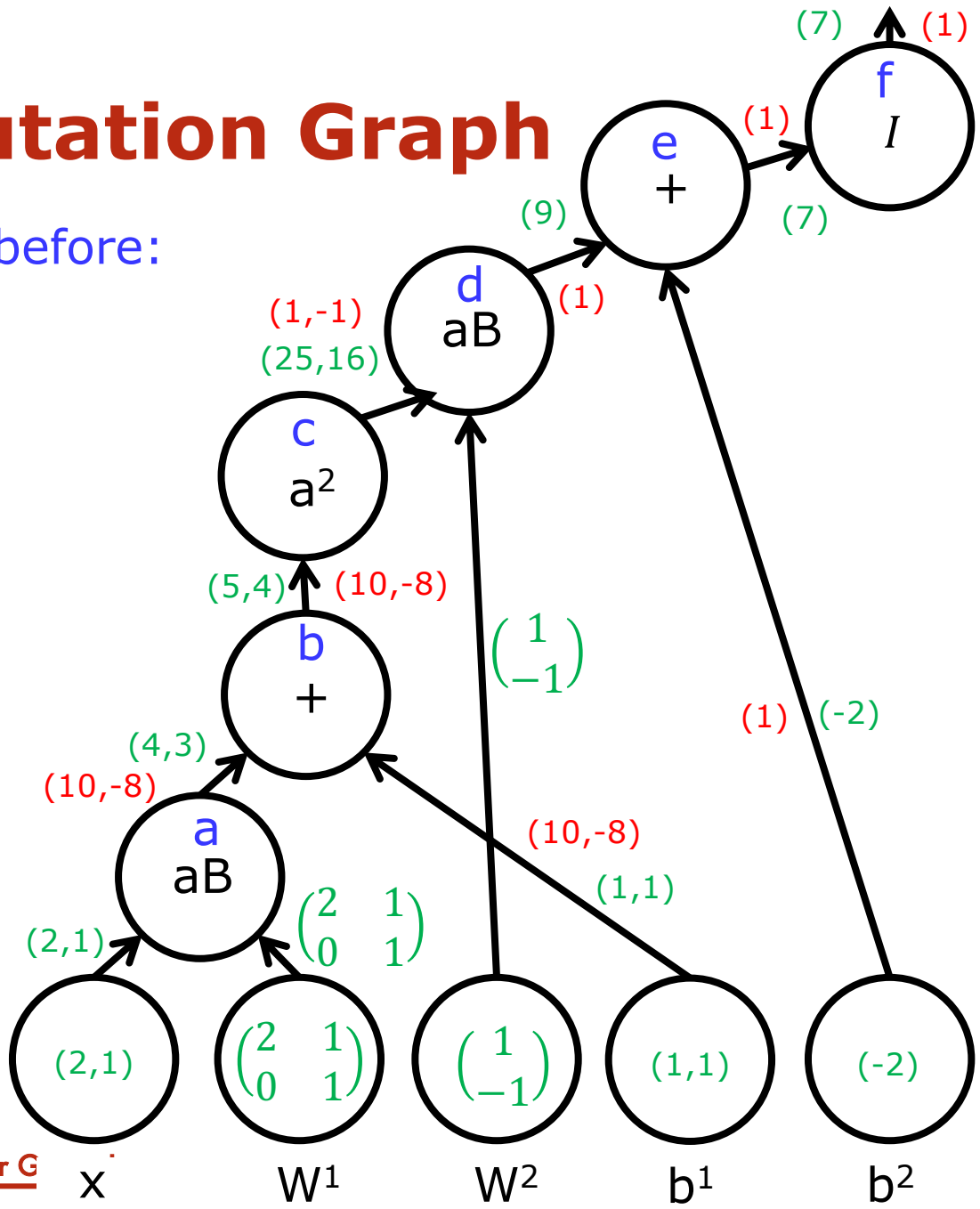
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial b}\frac{\partial b}{\partial a} = (10, -8)\nabla_a b(a)$$

$$\nabla_a b(a) = \begin{pmatrix} \frac{\partial a_1 + b_1^1}{\partial a_1} & \frac{\partial a_1 + b_1^1}{\partial a_2} \\ \frac{\partial a_2 + b_2^1}{\partial a_1} & \frac{\partial a_2 + b_2^1}{\partial a_2} \end{pmatrix}$$

$$\nabla_a b(a) = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

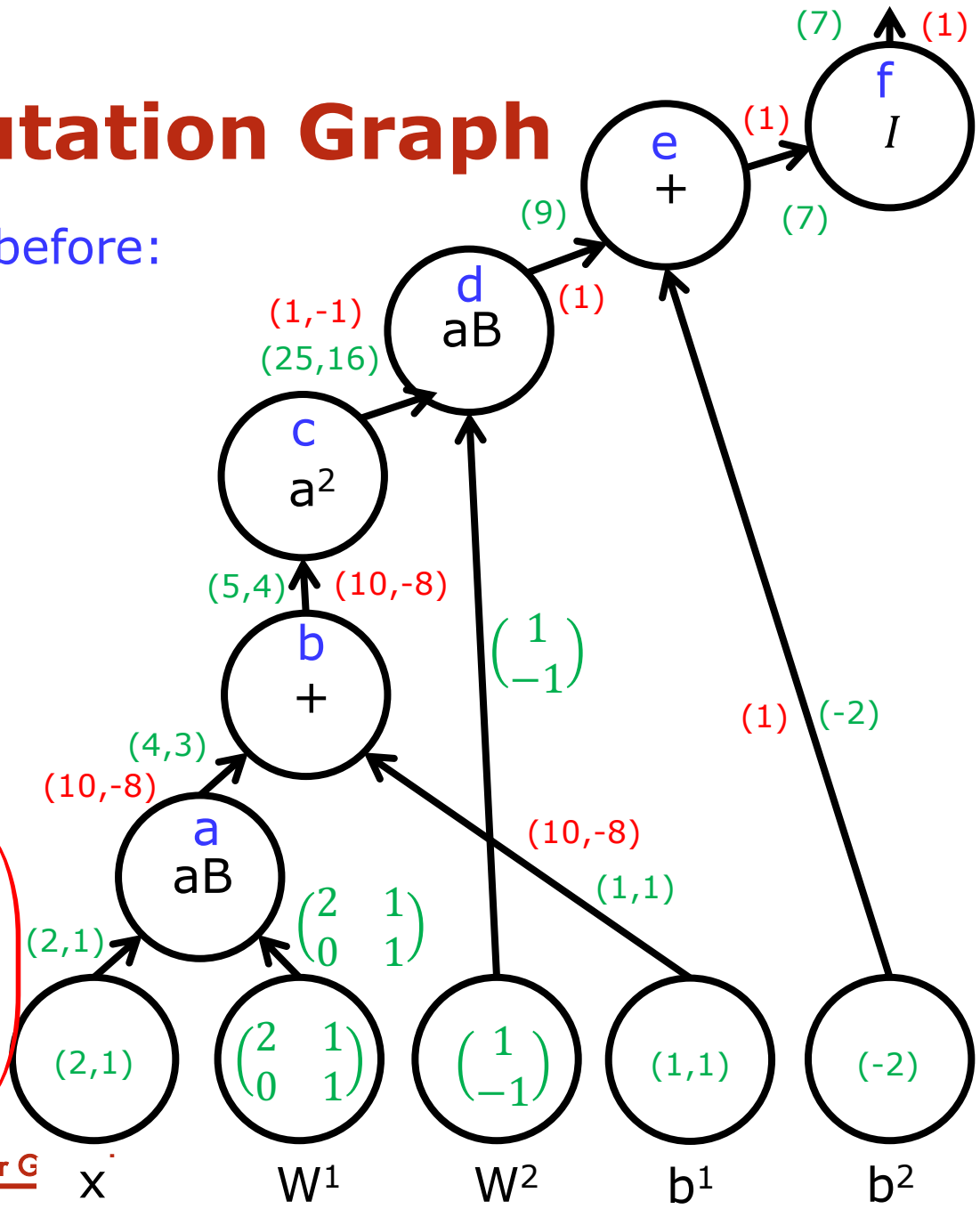


Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial x} = (10, -8)\nabla_x a(x)$$

$$\nabla_x a(x) = \begin{pmatrix} \frac{\partial W^1_{1,1}x_1 + W^1_{2,1}x_2}{\partial x_1} & \frac{\partial W^1_{1,1}x_1 + W^1_{2,1}x_2}{\partial x_2} \\ \frac{\partial W^1_{1,2}x_1 + W^1_{2,2}x_2}{\partial x_1} & \frac{\partial W^1_{1,2}x_1 + W^1_{2,2}x_2}{\partial x_2} \end{pmatrix}$$

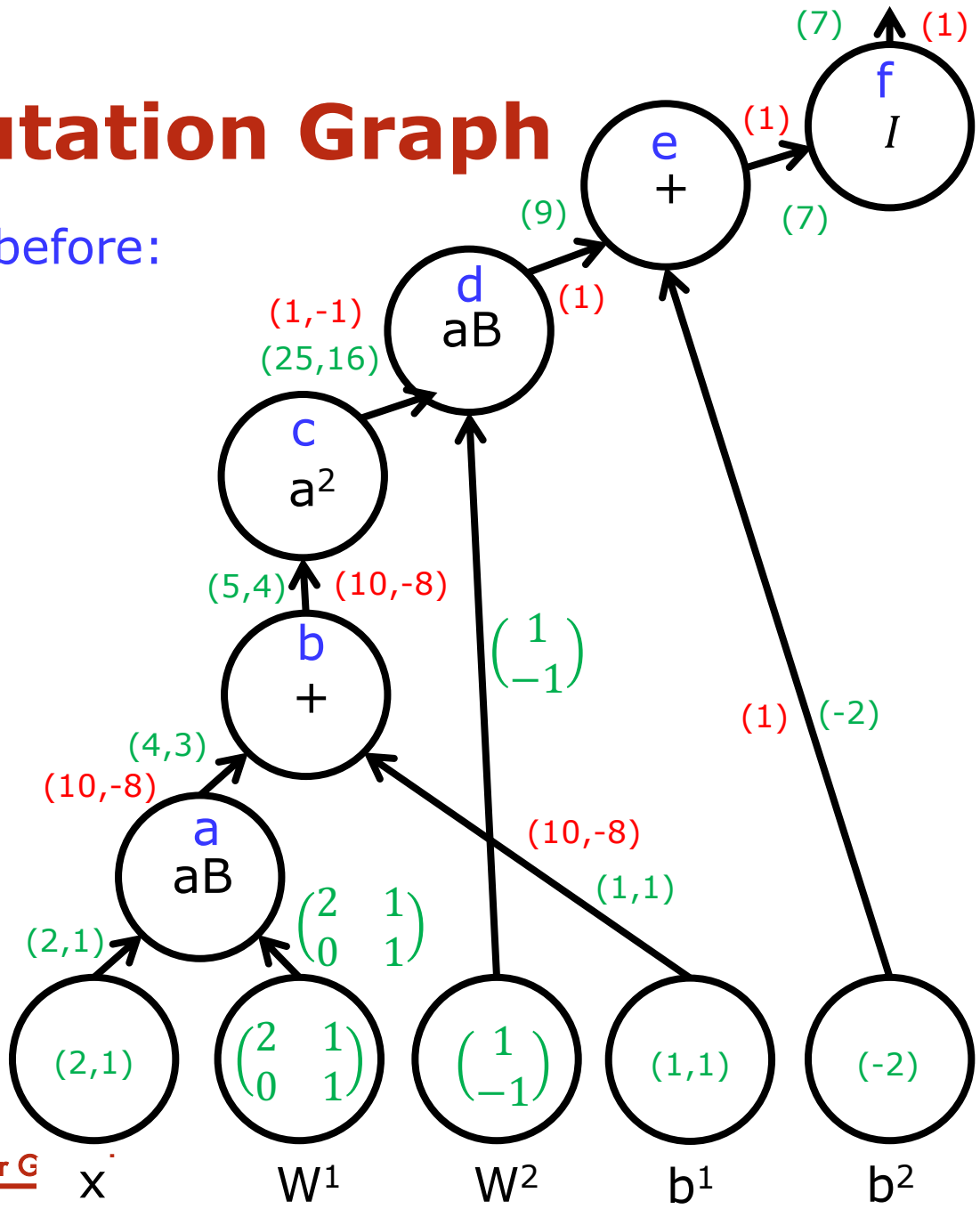


Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial x} = (10, -8)\nabla_x a(x)$$

$$\nabla_x a(x) = \begin{pmatrix} W^1_{1,1} & W^1_{2,1} \\ W^1_{1,2} & W^1_{2,2} \end{pmatrix}$$



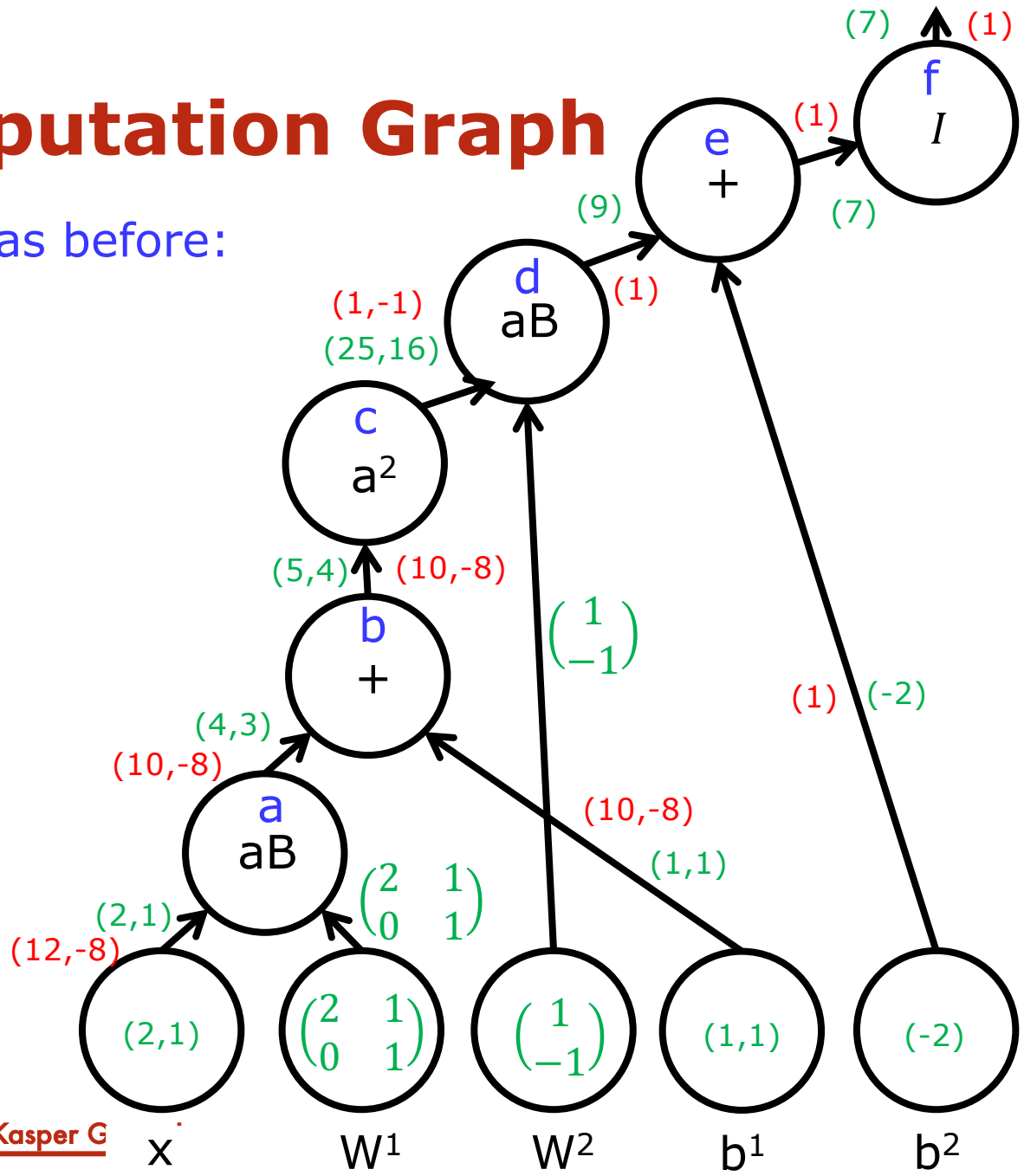
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial x} = (10, -8)\nabla_x a(x)$$

$$\nabla_x a(x) = (W^1)^T$$

$$\frac{\partial f}{\partial x} = (10, -8)\begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix} = (12, -8)$$

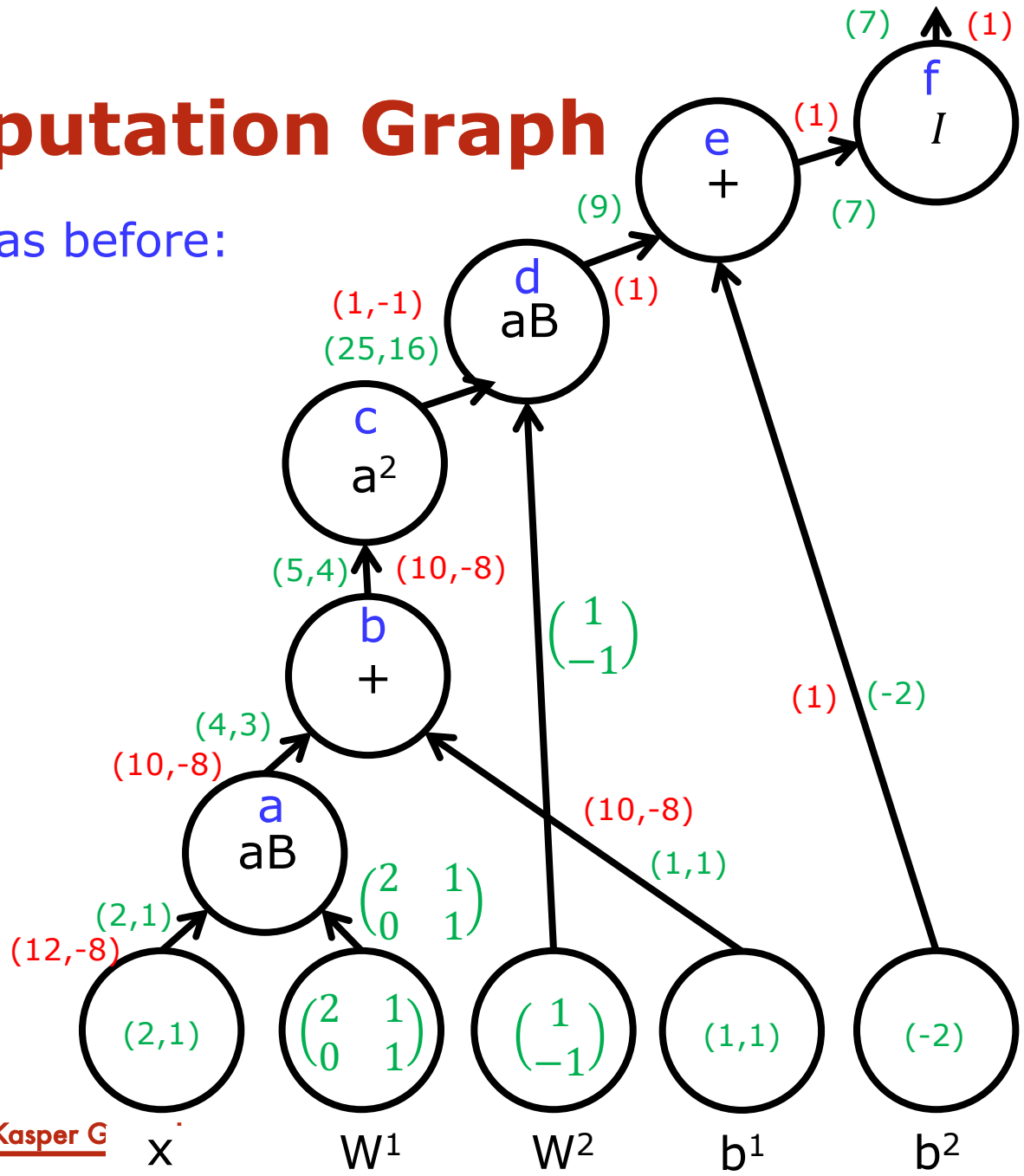


Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial W^1} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial W^1} = (10, -8)??$$

What is matrix of derivatives when W^1 is a matrix??



Vectorized Computation Graph

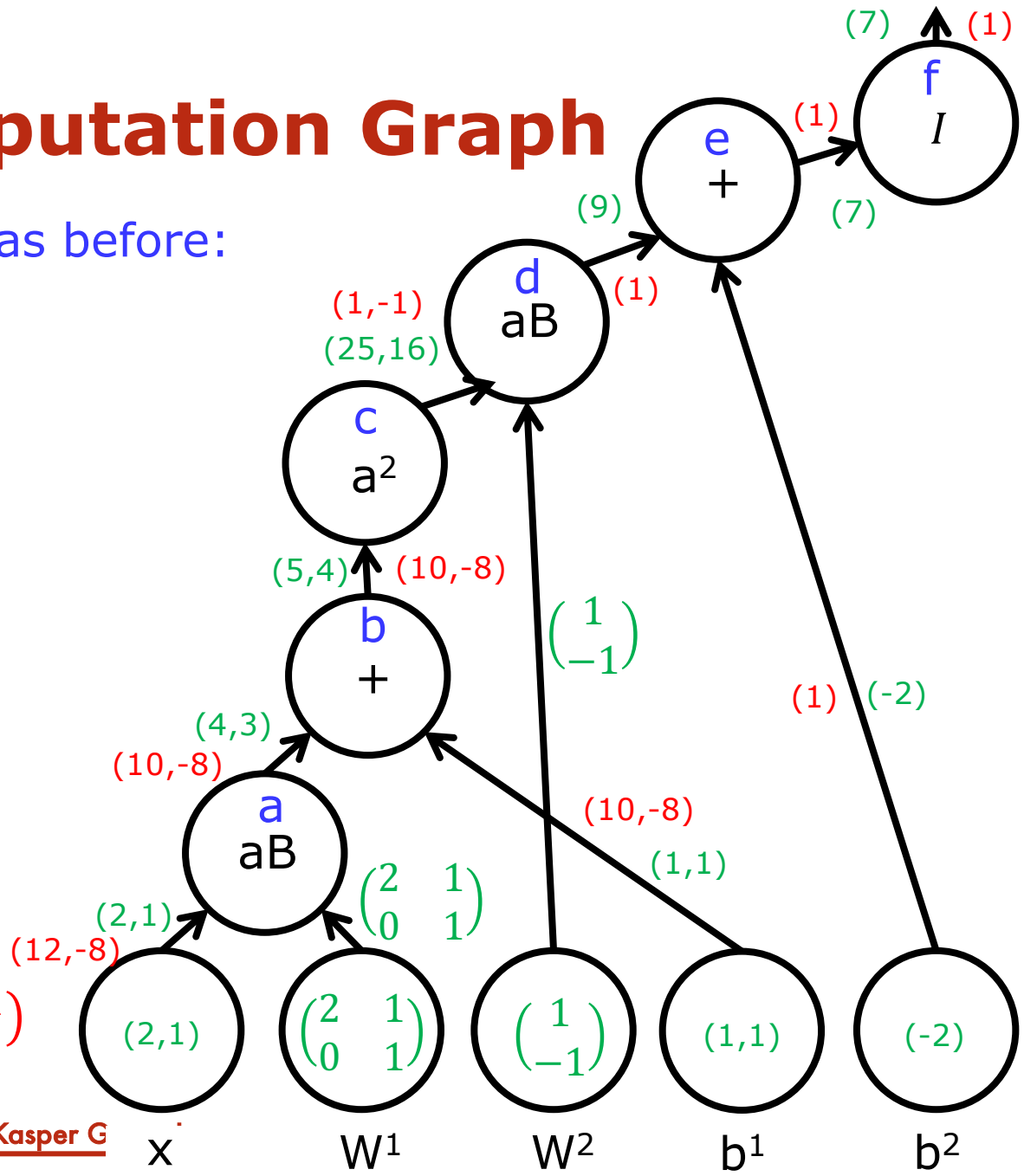
- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial W^1} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial W^1} = (10, -8)??$$

Flatten W^1:

$$\widehat{W}^1 = (W^1_{1,1}, W^1_{1,2}, W^1_{2,1}, W^1_{2,2})$$

$$\frac{\partial f}{\partial \widehat{W}^1} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial \widehat{W}^1} = (10, -8)\nabla_{\widehat{W}^1} a(\widehat{W}^1)$$

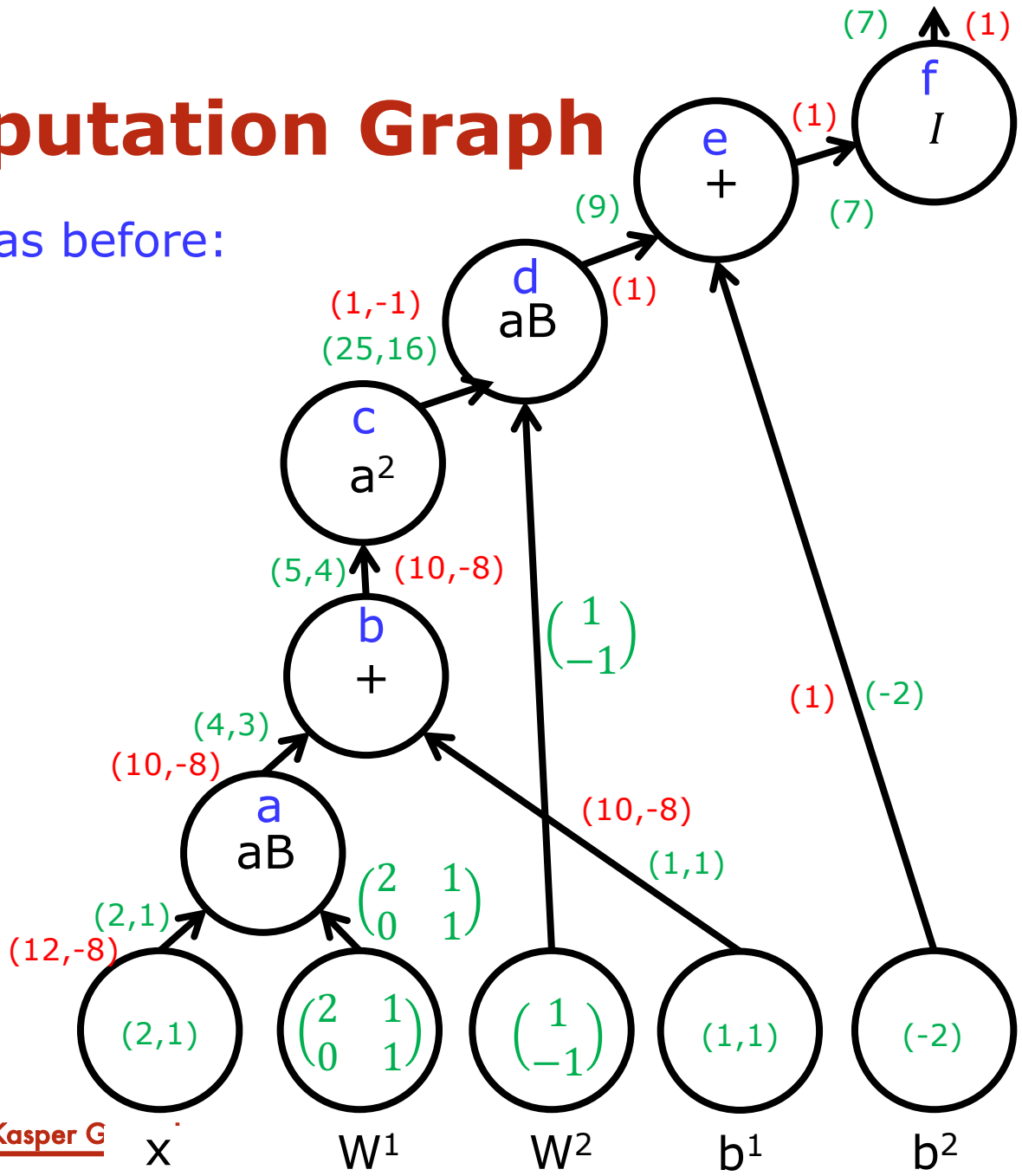


Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial W^1} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial W^1} = (10, -8)??$$

$$\nabla_{\widehat{W}^1} a(\widehat{W}^1) = \begin{pmatrix} \frac{\partial W^1_{1,1}x_1 + W^1_{2,1}x_2}{\partial W^1_{1,1}} & \cdots & \cdots \\ \cdots & \cdots & \cdots \end{pmatrix}$$



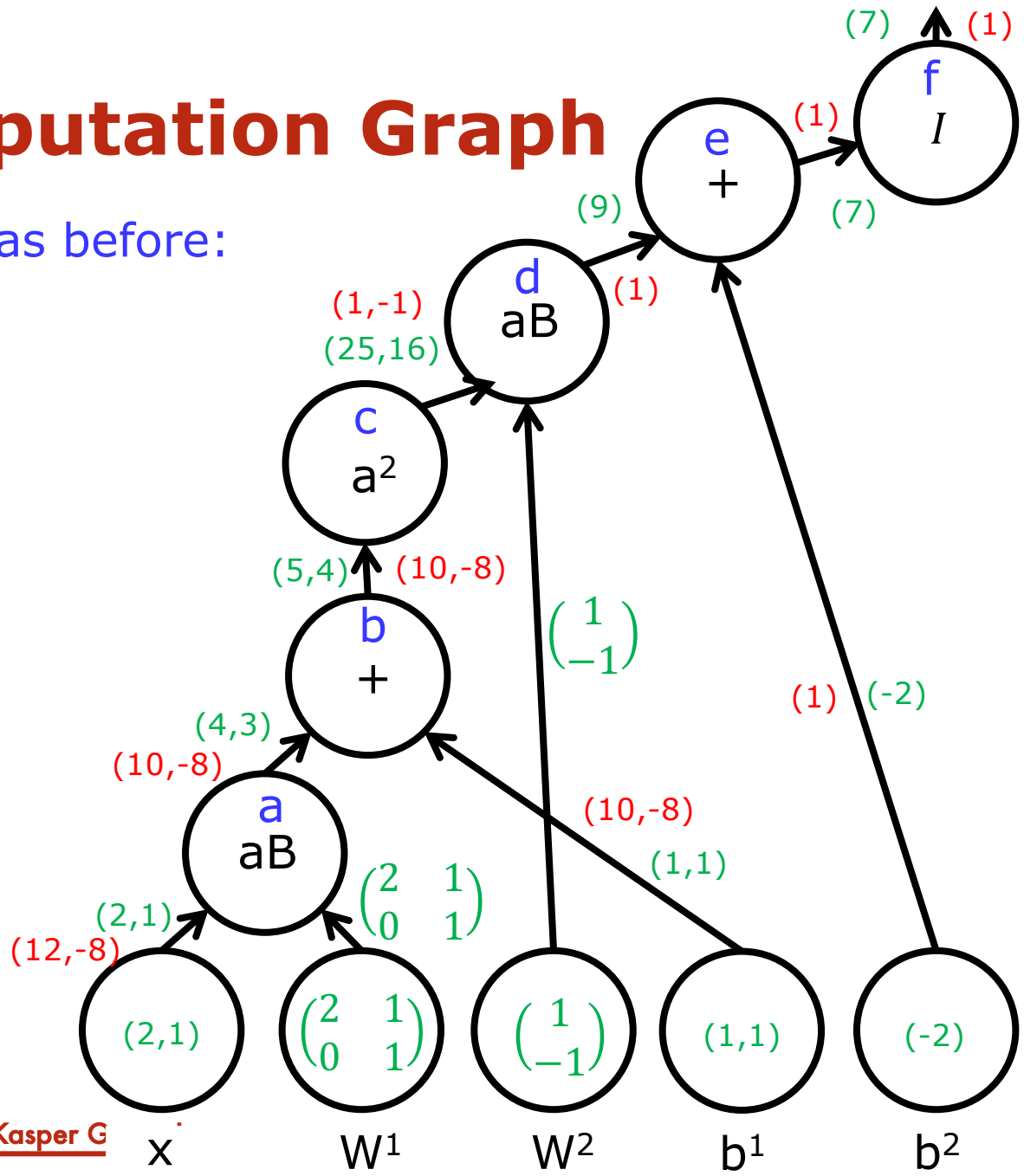
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial W^1} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial W^1} = (10, -8)??$$

$$\nabla_{\widehat{W}^1} a(\widehat{W}^1) = \begin{pmatrix} x_1 & 0 & x_2 & 0 \\ 0 & x_1 & 0 & x_2 \end{pmatrix}$$

$$\frac{\partial f}{\partial \widehat{W}^1} = (10, -8)\begin{pmatrix} x_1 & 0 & x_2 & 0 \\ 0 & x_1 & 0 & x_2 \end{pmatrix}$$



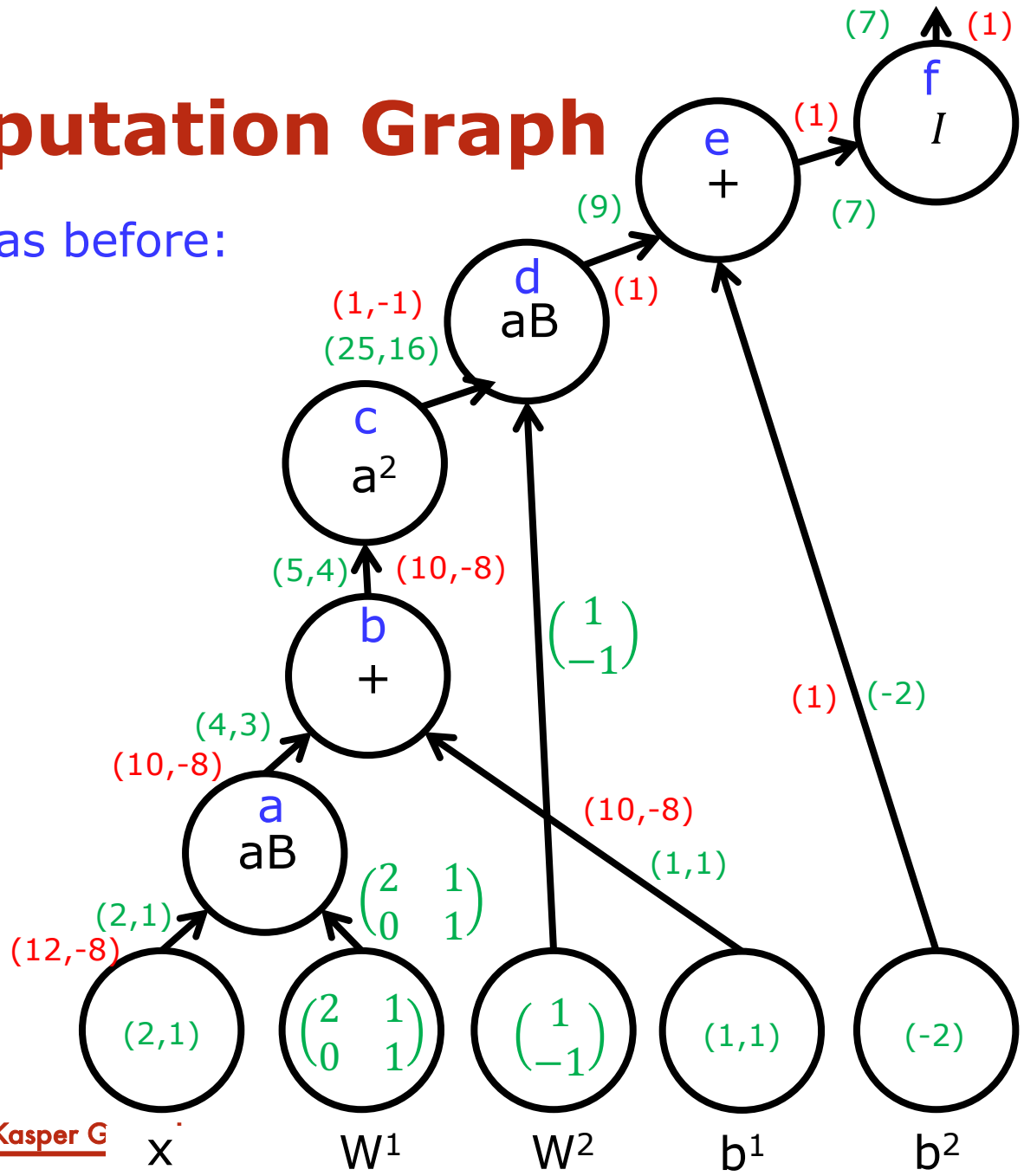
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial W^1} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial W^1} = (10, -8)??$$

$$\nabla_{\widehat{W}^1} a(\widehat{W}^1) = \begin{pmatrix} x_1 & 0 & x_2 & 0 \\ 0 & x_1 & 0 & x_2 \end{pmatrix}$$

$$\frac{\partial f}{\partial \widehat{W}^1} = (10, -8)\begin{pmatrix} 2 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 \end{pmatrix}$$



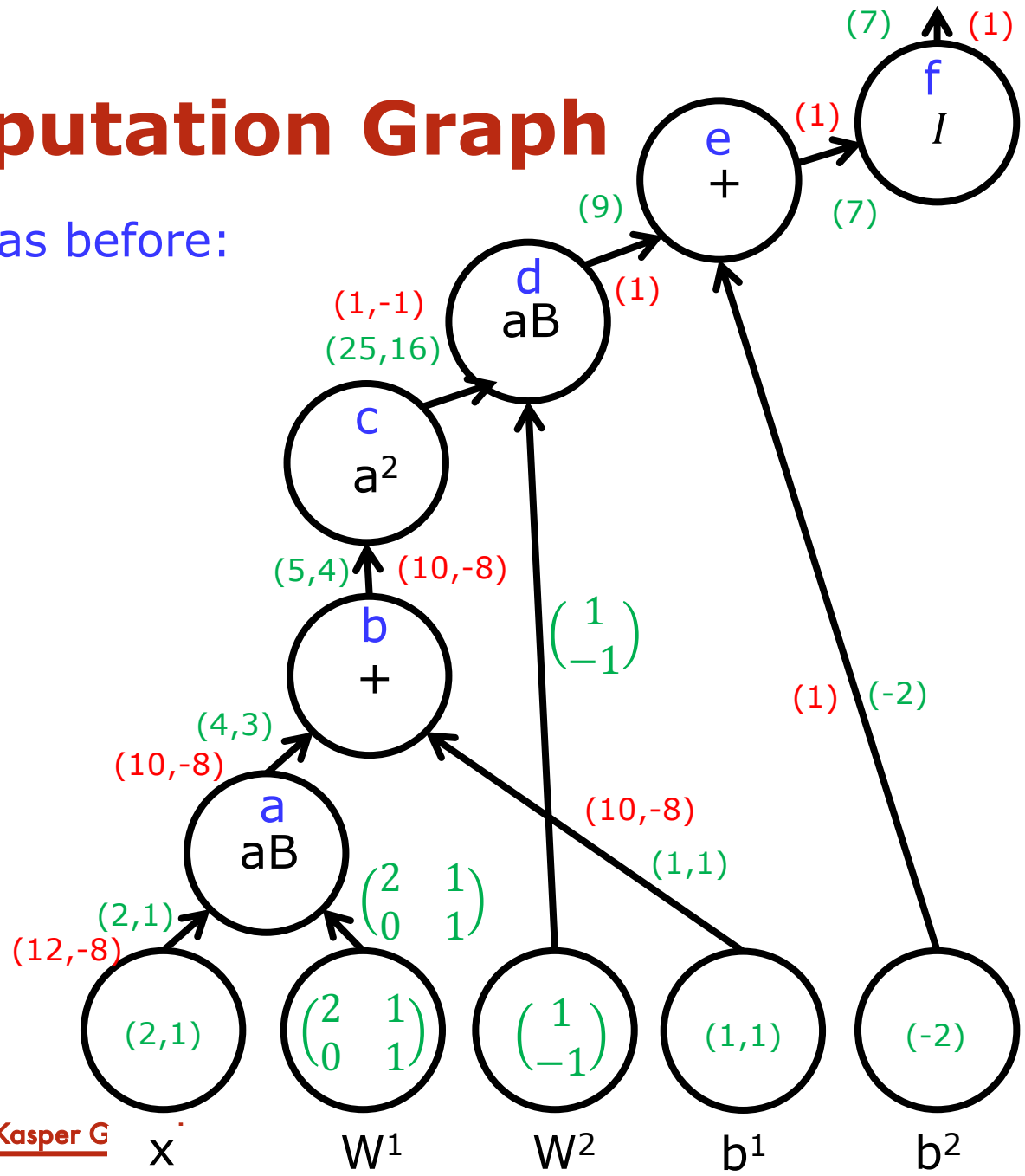
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial W^1} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial W^1} = (10, -8)??$$

$$\nabla_{\widehat{W}^1} a(\widehat{W}^1) = \begin{pmatrix} x_1 & 0 & x_2 & 0 \\ 0 & x_1 & 0 & x_2 \end{pmatrix}$$

$$\frac{\partial f}{\partial \widehat{W}^1} = (20, -16, 10, -8)$$



Vectorized Computation Graph

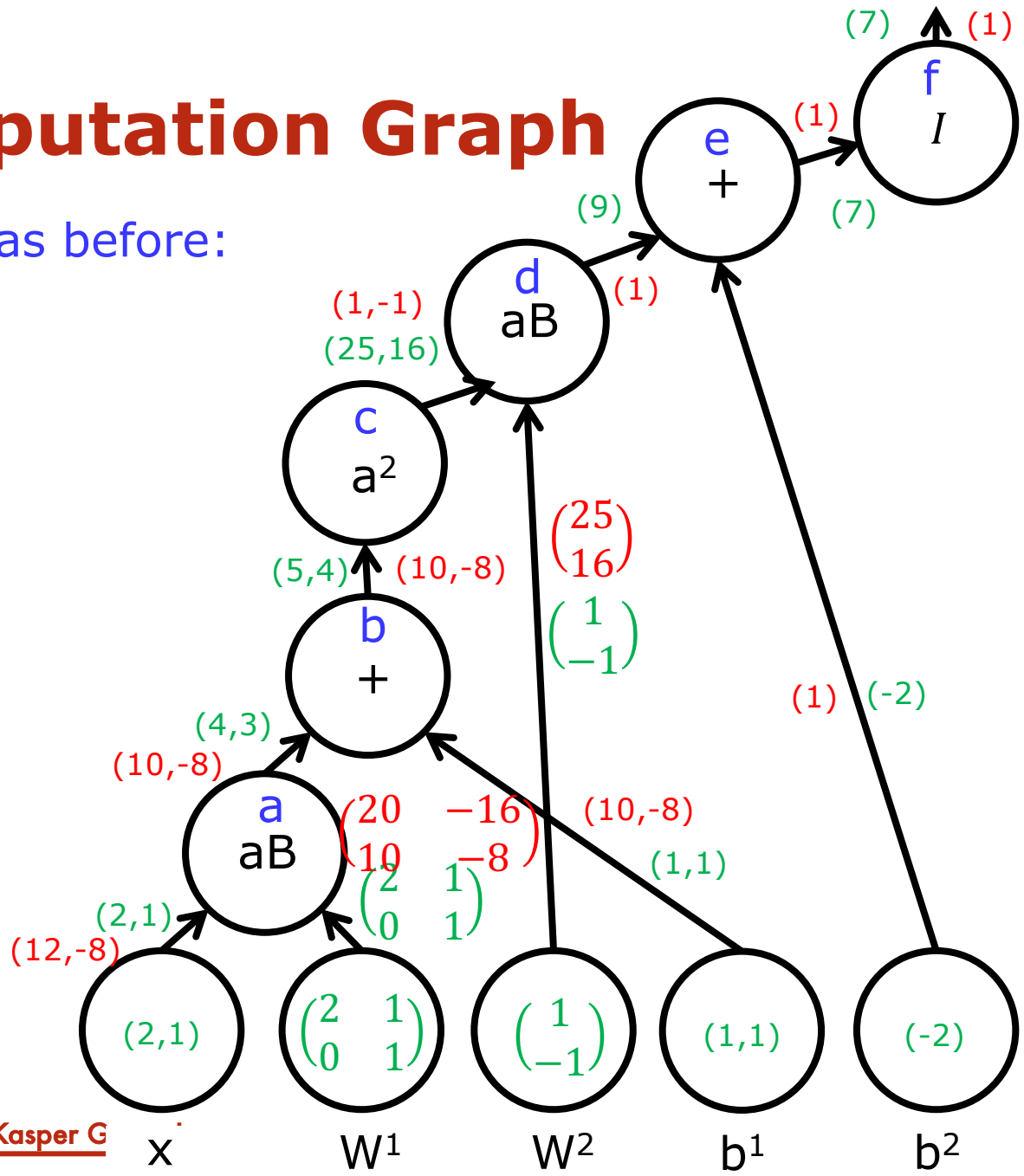
- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

$$\frac{\partial f}{\partial W^1} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial W^1} = (10, -8)??$$

$$\frac{\partial f}{\partial \widehat{W}^1} = (20, -16, 10, -8)$$

De-flatten to

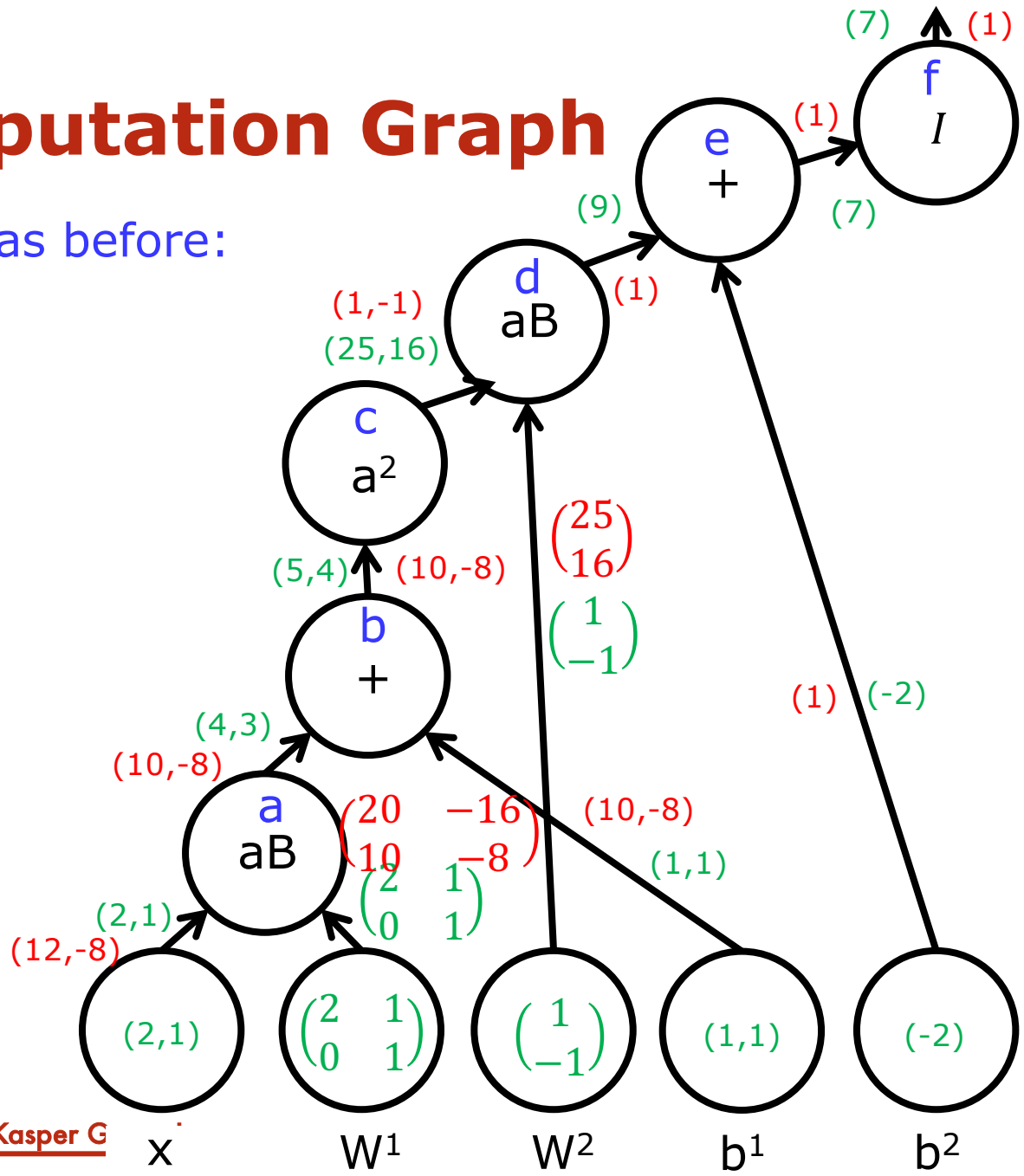
$$\frac{\partial f}{\partial W^1} = \begin{pmatrix} 20 & -16 \\ 10 & -8 \end{pmatrix}$$



Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

- Very tedious by hand.
  - But doable.



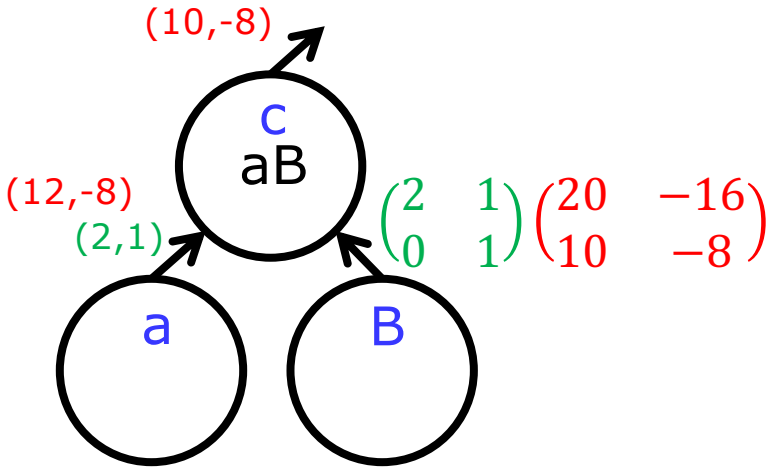
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

- Most tricky case aB.
  - Just memorize:

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial c} B^T$$

$$\frac{\partial f}{\partial B} = \begin{pmatrix} a_1 \frac{\partial f}{\partial c} \\ \vdots \\ a_k \frac{\partial f}{\partial c} \end{pmatrix}$$



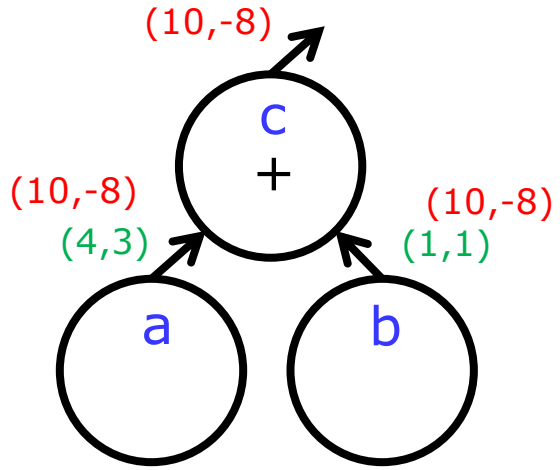
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

- Easy Case.
  - Just memorize:

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial c} I = \frac{\partial f}{\partial c}$$

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c} I = \frac{\partial f}{\partial c}$$



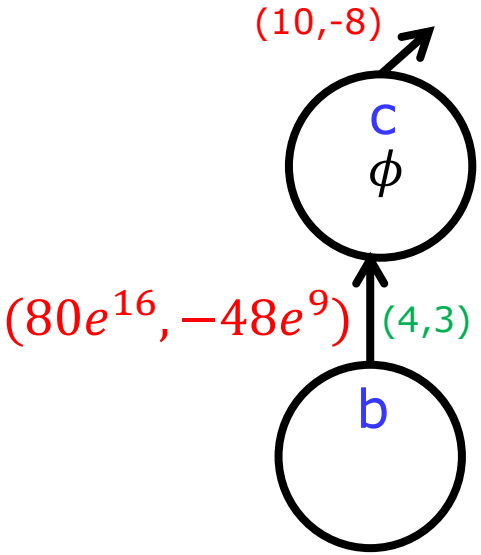
Vectorized Computation Graph

- Let's try same approach as before:
  - Name nodes.
  - Forward pass.
  - Backwards pass.

- Activations.
  - Just memorize:

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial b} = \frac{\partial f}{\partial c}\begin{pmatrix} \phi'(b_1) & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & \phi'(b_k) \end{pmatrix}$$

Exercise: Prove this!



$$\phi(x) = e^{x^2}$$

$$\phi'(x) = e^{x^2}2x$$

$$(10, -8)\begin{pmatrix} e^{16}8 & 0 \\ 0 & e^{9}6 \end{pmatrix}$$

Stochastic Gradient Descent

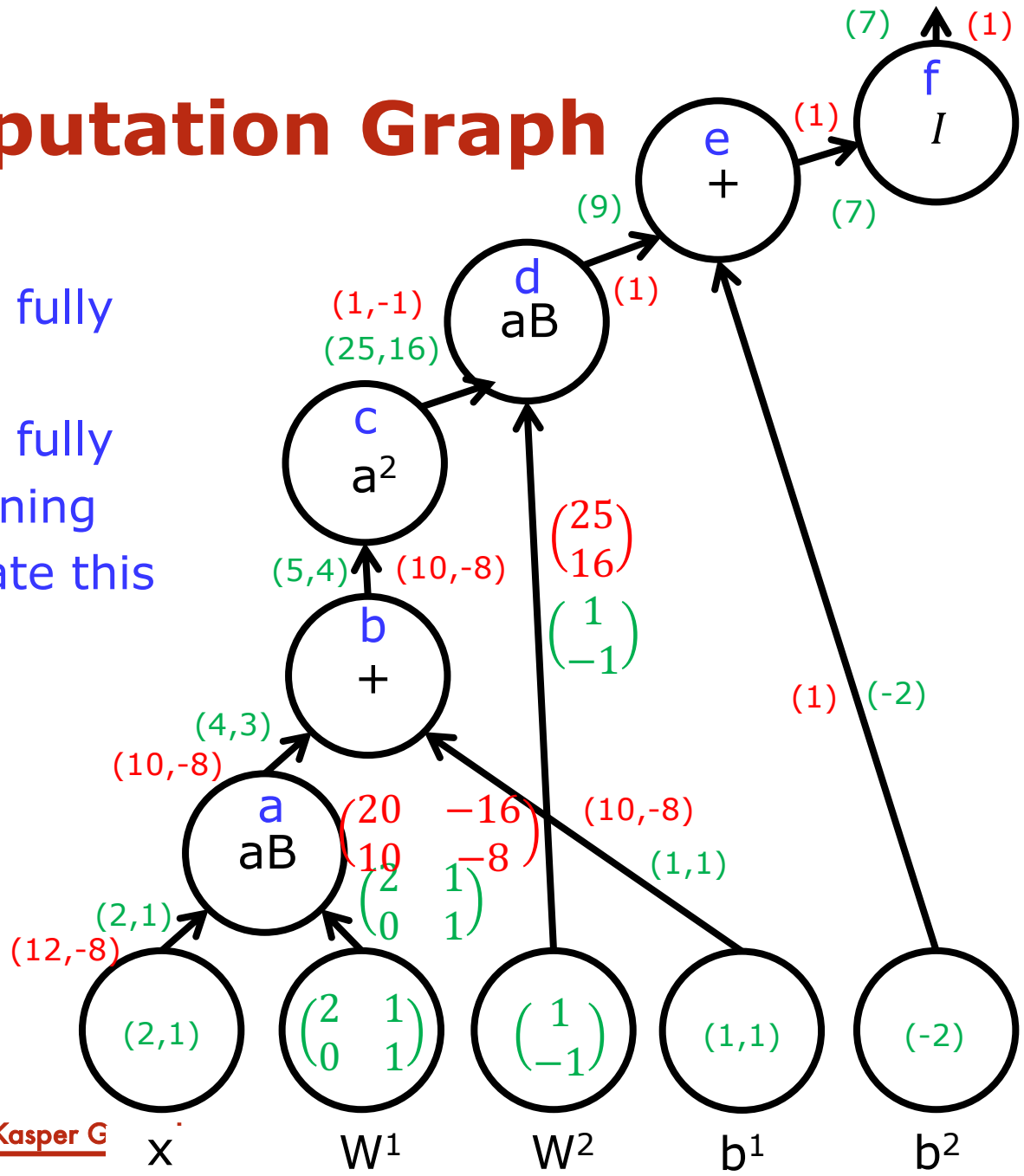
- Any point-wise loss:
  - $E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n} L(w(x_i), y_i)$
- Mini-batch SGD:
  - Initialize w (e.g. random N(0,1) distributed er
  - While more time:
    - Permute $(x_1,y_1),\ldots,(x_n,y_n)$ randomly
    - For i=1,...,n/B:
      - Let $(x_1',y_1'),\ldots,(x_B',y_B')$ be next B data items.
      - $g = \frac{1}{B}\sum_{i=1}^{B} \nabla L(w(x_i'), y_i')$
      - $w = w - \eta g$
  - Return w.

$w(x_i)$ output of neural net with weights and biases w

Computation graph must also include L

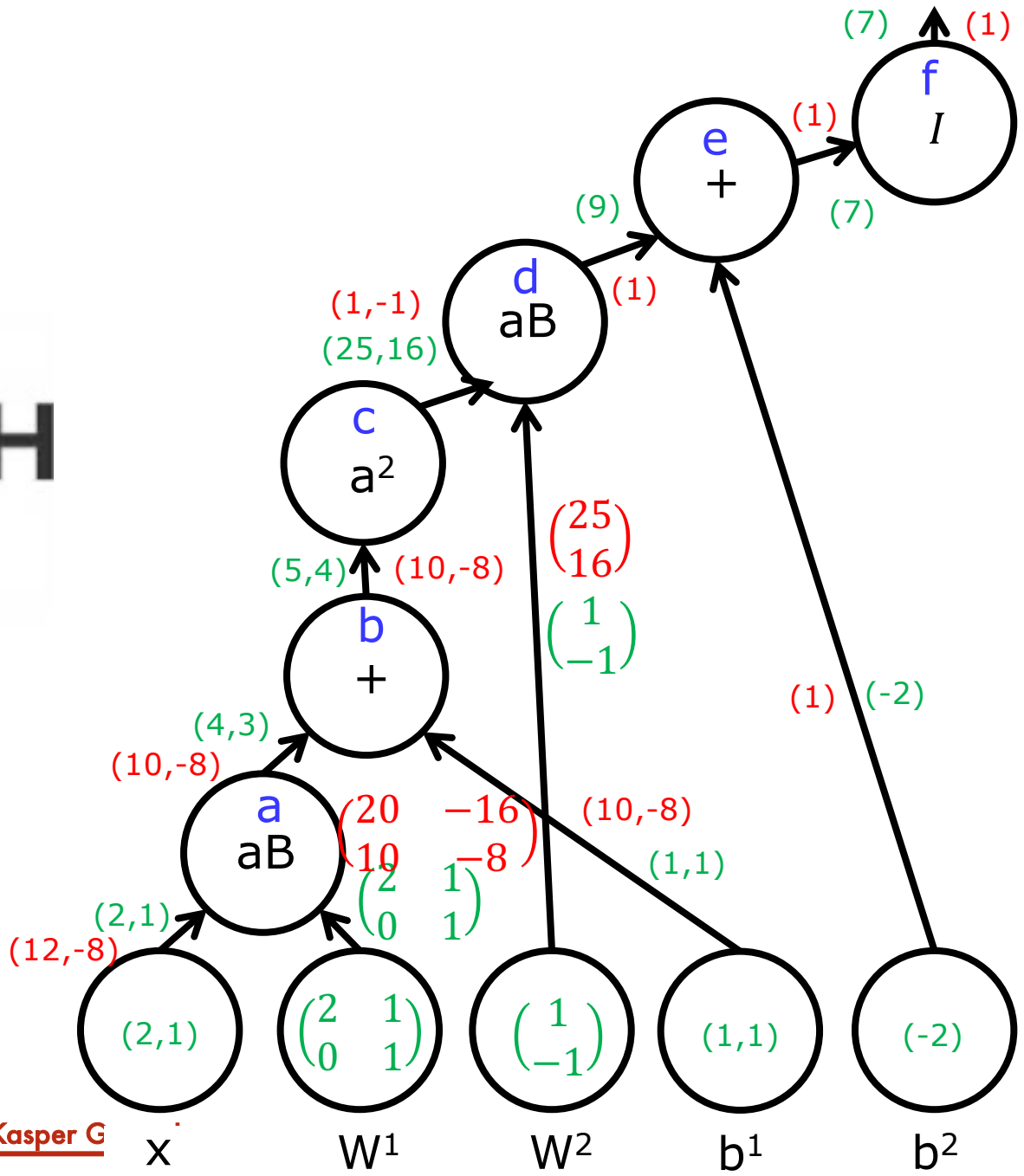
Vectorized Computation Graph

- Very tedious.
  - Try it at least once to fully understand it.
  - Try it at least once to fully appreciate deep learning libraries that automate this process.



PyTorch


PYTORCH


f
I
e
+
d
aB
c
a²
b
+
a
aB
(7)
(1)
(1)
(7)
(9)
(1)
(1,-1)
(25,16)
(5,4)
(10,-8)
(4,3)
(10,-8)
(12,-8)
(2,1)
(10,-8)
(1,1)
(1)
(-2)
(2,1)
(2 1 / 0 1)
(1 / −1)
(1,1)
(-2)
x
W¹
W²
b¹
b²

```python
import torch

x = torch.tensor([2.0, 1.0], dtype=torch.double, requires_grad=True)
W1 = torch.tensor([[2.0, 1.0],[0.0, 1.0]], dtype=torch.double, requires_grad=True)
W2 = torch.tensor([[1.0],[-1.0]], dtype=torch.double, requires_grad=True)
b1 = torch.tensor([1.0, 1.0], dtype=torch.double, requires_grad=True)
b2 = torch.tensor([-2.0], dtype=torch.double, requires_grad=True)

a=x@W1
b=a+b1
c=b**2
d=c@W2
e=d+b2
f=e

a.retain_grad()
b.retain_grad()
c.retain_grad()
d.retain_grad()
e.retain_grad()
f.backward()
print('df_dx',x.grad)
print('df_dW1',W1.grad)
print('df_dW2',W2.grad)
print('df_db1',b1.grad)
print('df_db2',b2.grad)
print('df_da',a.grad)
print('df_db',b.grad)
print('df_dc',c.grad)
print('df_dd',d.grad)
print('df_de',e.grad)
```

```
d39655:Week5 au203538$ python3 pytorch.py
df_dx tensor([12., -8.], dtype=torch.float64)
df_dW1 tensor([[ 20., -16.],
        [ 10.,  -8.]], dtype=torch.float64)
df_dW2 tensor([[25.],
        [16.]], dtype=torch.float64)
df_db1 tensor([10., -8.], dtype=torch.float64)
df_db2 tensor([1.], dtype=torch.float64)
df_da tensor([10., -8.], dtype=torch.float64)
df_db tensor([10., -8.], dtype=torch.float64)
df_dc tensor([ 1., -1.], dtype=torch.float64)
df_dd tensor([1.], dtype=torch.float64)
df_de tensor([1.], dtype=torch.float64)
```

